

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF INDUSTRIAL ENGINEERING AND MANAGEMENT**



**Demand-Weighted Optimization of First-Mile Feeder Bus
Routes for Metro Station Access: A Case Study of Ho Chi
Minh City Metro**

Submitted in partial fulfilment of the requirements for the Degree of
Bachelor of Engineering in Logistics and Supply Chain Management

Student: TRẦN DUY QUÂN

ID: IELSIU21046

Thesis Advisor: DR. PHẠM HUỲNH TRÂM

Ho Chi Minh City, Vietnam
June 2026

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF INDUSTRIAL ENGINEERING AND MANAGEMENT**



**Demand-Weighted Optimization of First-Mile Feeder Bus
Routes for Metro Station Access: A Case Study of Ho Chi
Minh City Metro**

Submitted in partial fulfillment of the requirements for the Degree of
Bachelor of Engineering in Logistics and Supply Chain Management/

Student: TRẦN DUY QUÂN

ID: IELSIU21046

Thesis Advisor: DR. PHẠM HUỖNH TRÂM

Ho Chi Minh City, Vietnam
June 2026

Demand-Weighted Optimization of First-Mile Feeder Bus Routes for Metro Station Access: A Case Study of Ho Chi Minh City Metro

By

TRẦN DUY QUÂN

Submitted in partial fulfillment of the requirements for the Degree of
Bachelor of Engineering in Logistics and Supply Chain Management
International University, Ho Chi Minh City
June 2026

Signature of Student: _____
Tran Duy Quan

Certified by _____
Dr. Pham Huynh Tram
Thesis Advisor

Approved by _____
Dr. Nguyen Van Hop
Dean of IEM school

Abstract

This thesis addresses the persistent challenge of "first-mile" connectivity in Ho Chi Minh City, specifically focusing on the Vietnam National University (VNU-HCM) urban area's integration with the HCMC Metro Line 1. Despite the introduction of feeder bus services, low-capacity utilization and a heavy reliance on private motorbikes during peak hours (7:00 AM – 9:00 AM) indicate a gap in current route efficiency. This research proposes a data-driven optimization framework using the Pulse Algorithm to design demand-weighted feeder bus loops. By utilizing secondary geospatial data from Google Maps and population density proxies, the model maximizes a "Priority Index" (PI) to capture high-demand clusters—such as student dormitories and faculty campuses—under strict travel-time constraints. Unlike traditional heuristic methods, this exact optimization approach ensures a global optimum, providing a robust decision-support tool for urban planners to enhance metro ridership and support Transit-Oriented Development (TOD). The result is an operationally feasible, non-overlapping loop structure that significantly improves the Total Served Area for the National University Metro Station.

Keywords: First-mile connectivity, Pulse Algorithm, Feeder Bus Network Design Problem, Meta-Heuristics, Transit-oriented development.

Acknowledgements

I would like to express my profound gratitude to my advisor, Dr. Pham Huynh Tram, for her detailed technical instruction on the statistical collection and cleaning of real-world datasets and for guiding me through a complete research framework to derive meaningful insights from the proposed models. My sincere thanks to my fellow peers at the School of Industrial Engineering & Management for their constant support throughout our logistics and supply chain major. I am further indebted to my friends in the Faculty of Environment and Natural Resources at Ho Chi Minh City University of Technology (HCMUT) for their invaluable insights into the academic Geographic Information System (GIS) tools and to those in the Faculty of Information Systems at the University of Information Technology (UIT) for recommending state-of-the-art IT resources of their field for my dissertation development. Their interdisciplinary insights were vital in constructing an optimized first-mile feeder bus network for the National University station of HCMC Metro Line 1.

Table of Contents

Abstract	ii
Acknowledgements	iii
Table of Contents.....	iv
List of Tables	v
List of Figures.....	vi
List of Abbreviations	vii
Chapter 1: INTRODUCTION.....	9
1.1. Background:.....	9
1.2. Problem Statement – The Need for Study:	10
1.3. Objectives of Study:	14
1.4. Scope and Limitation:.....	16
Chapter 2: RELATED WORKS.....	19
2.1 Overview:.....	19
2.1.1 Theoretical foundations	19
2.1.2 Current System Investigation.....	21
2.2. Literature Review:	26
2.2.1 Investigating Current Development Approaches	26
2.2.2 Design Concepts Consideration	27
2.3 Key references/ Candidate Solution Methods.....	28
Chapter 3: METHODOLOGY	31
3.1 Approaches Comparison and Selection:	31
3.2 Proposed System Design	33
Chapter 4: SOLUTION DEVELOPMENT.....	37
4.1 Data Collection & Model Construction	37
4.2 Mathematical Model.....	48
4.3 Optimization Process Flowchart.....	55
Chapter 5: RESULT ANALYSIS.....	58
5.1 Sensitivity Analysis	58
5.2 Scenario Analysis	61
Chapter 6: CONCLUSIONS	74
6.1 Results Discussion & Implications.....	74
6.2 Recommendations for Future Research.....	77
References	79
Appendix A Google Earth Engine Code.....	82
Appendix B TomTom Data Retrieve – Google Colab Script	91
Appendix C Grid Link Assignment – Colab Script.....	114
Appendix D PULSE ALGORITHM	132

List of Tables

Table 1. Methodologies Comparison Table	31
Table 2. Sample data table of 92 stations.....	38
Table 3. Input Data Field Demonstration.....	39
Table 4. Weekly Speed Distribution Table of Ho Chi Minh City	43
Table 5. Table of Indices & Sets	48
Table 6. Table of Parameters	49
Table 7. Table of Decision Variables.....	52
Table 8. Grid size Sensivity Analysis Table.....	59
Table 9. Scenario Analysis Table	61
Table 10. Table of Optimized Parameters	64
Table 11. Discussion Summary.....	76

List of Figures

Figure 1. VNU Metro Station Motorbike Parking Lot	10
Figure 2. Feeder bus network at VNU	12
Figure 3. Bus Route 166 Map	22
Figure 4. Bus Route 164V Map	23
Figure 5. Bus Route 167 Map	24
Figure 6. Bus Route 170 Map	25
Figure 7. Research Framework	33
Figure 8. VNU Map Polygon with station marks	37
Figure 9. Link type description	39
Figure 10. Valid/Invalid Links	41
Figure 11. VNU Polygon with Grid network	45
Figure 12. Population Distribution in VNU area	46
Figure 13. Model Optimization Flowchart	55
Figure 14. Comparison of 18-min output maps with existing route 166	67
Figure 15. Route map of PA 45-min model	69
Figure 16. Route map of GA/ACO 45-min model	70
Figure 17. Route map of existign route 170	70
Figure 18. Pulse & ACO/ GA route difference	71

List of Abbreviations

1. Transit Oriented Development = TOD
2. Vietnam National University = VNU
3. Pulse Algorithm = PA
4. Genetic Algorithm = GA
5. Ant Colony Optimization = ACO

Chapter 1: INTRODUCTION

1.1. Background

Transit-oriented development (TOD) is commonly defined as an integrated planning approach that aligns land use with high-capacity public transport by shaping station areas to improve connectivity, accessibility, and place quality, thereby reducing dependence on private vehicles (World Bank, 2015). This approach has become central in urban and public transport planning because it links infrastructure investment with sustainable travel behavior. In Ho Chi Minh City, where congestion and air-pollution pressures are persistent, the opening of Metro Line 1 (Ben Thanh–Suoi Tien) marks a major step toward a mass rapid transit backbone intended to support a shift away from private vehicle use and ease pressure on the road network (Japan International Cooperation Agency [JICA], 2024).

Despite this progress, Ho Chi Minh City does not yet have a fully mature MRT network with strong functional connectivity to surrounding residential areas, and station access remains a binding constraint on metro ridership potential. International energy and transport policy evidence emphasizes that achieving high passenger throughput on urban rail requires integrated first/last-mile solutions, including well-designed bus networks and other access modes, as part of TOD-oriented implementation (International Energy Agency [IEA], n.d.). In response, the city has introduced electric feeder bus services specifically to enhance access to Metro Line 1, but feeder operations can still face under-capacity and inefficiency when route structures are not optimized for spatially uneven local demand and operational constraints (Ministry of Construction of Vietnam, 2025). Therefore, this thesis focuses on demand-weighted optimization of first-mile feeder bus loop routes for metro station access, building on constrained feeder-route optimization principles in the recent academic literature (Hemdan et al., 2026)

1.2. Problem Statement – The Need for Study:

The structural integration of high-capacity mass rapid transit backbones, such as the Ho Chi Minh City Metro Line 1, depends heavily on the efficiency of localized feeder systems to resolve the critical first-mile accessibility gap. At the microscopic scale, this systemic friction is manifested daily within the Vietnam National University (VNU-HCM) urban area. During the primary morning peak-hour window from 7:00 AM to 9:00 AM, the motorcycle parking infrastructure serving the National University Metro Station experiences complete physical saturation, as documented by field observations.



Figure 1. VNU Metro Station Motorbike Parking Lot

This hyper-localized travel behavior—characterized by students choosing personal motorbikes over public transit even for short-distance intra-campus trips—indicates that the existing feeder

bus network is profoundly uncompetitive. From a transit planning perspective, the current feeder framework fails to minimize the commuter's generalized access cost, which encompasses in-vehicle travel time, scheduling reliability, transfer friction, and out-of-vehicle convenience. This localized observation of parking lot saturation is directly reflective of broader, macro-level urban transport realities across Vietnam. Personal vehicle availability is decoupled from economic constraints in major Vietnamese metropolitan centers, where motorcycles consistently constitute between 90% and 96% of the national vehicle fleet (Huu & Ngoc, 2021). Long-term empirical transit data demonstrates that motorcycle trips command a stubborn 85% to 87% of total urban travel demand (Ngoc et al., 2021). Driven by an exceptionally high vehicle ownership rate of approximately 565 units per 1,000 persons, a private motorized vehicle is almost universally accessible to the student demographic (Huu & Ngoc, 2021). Consequently, the default mode choice automatically skews toward private transport because alternative transit choices fail to offer comparable utility.

Other modes of travel, such as conventional buses, cycling, and especially walking, are completely lackluster compared to personal motorized vehicles, as confirmed by empirical urban data where walking accounts for less than 25% of aggregate trips (Ngoc et al., 2021). The persistence of this travel asymmetry is deeply rooted in the transport psychology of Vietnamese students. Numerical behavioral metrics indicate that the threshold for acceptable walking distances among the student demographic is highly restricted, demonstrating a steep psychological decay in the willingness to walk beyond minimal distances (Linh Le, 2024). Private motorcycles are overwhelmingly selected because they align with established behavioral habits, maximizing perceived absolute temporal efficiency while minimizing the cognitive and physical effort required by active travel modes (Linh Le, 2024). Without a highly attractive, walk-friendly feeder bus network designed to directly

intercept these short-distance trips, this systemic overreliance on personal vehicles will continue unabated.

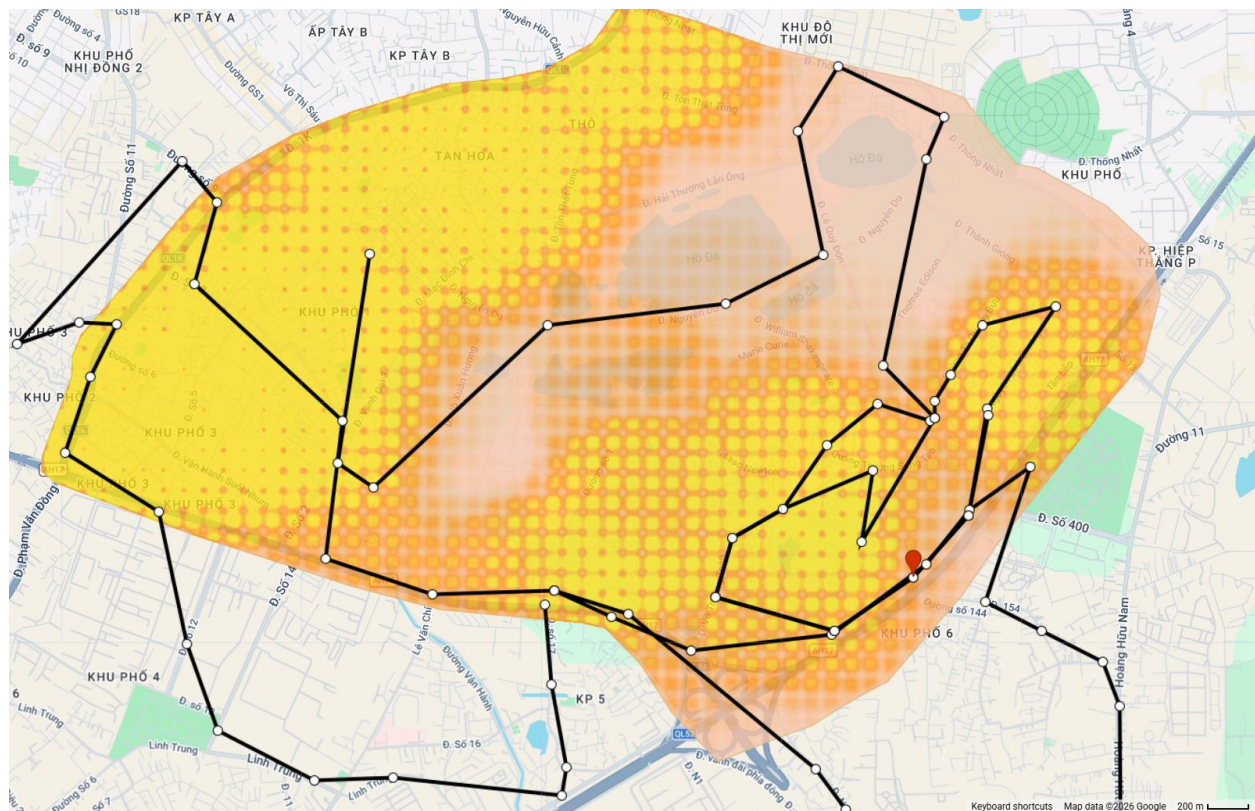


Figure 2. Feeder bus network at VNU

The operational failure of the current public transport ecosystem within the campus is fundamentally a problem of spatial mismatch. The map of feeder bus network evidence provided by the VNU population heatmap overlay with the current active feeder bus networks exposes a severe demand-capture deficit. The layout of the active routes indicates a critical planning gap: the physical trajectories of existing buses do not align with the asymmetric distribution of campus commuters. Large, high-intensity population zones—marked by intense yellow density clusters representing major student residential dormitories and key faculty compounds—are completely bypassed by the network. Conversely, several active route segments are inefficiently projected

over spaces with negligible or entirely absent population densities, wasting operational mileage on low-demand corridors.

This localized spatial misalignment mirrors city-wide trends, where public bus transit commands a historically low share of overall ridership, hovering at a microscopic 4.7% to 5% of the regional modal split (Ngoc et al., 2021). To address this structural crisis and improve public transport utility, the urgency of implementing a demand-weighted bus system is further emphasized by recent regulatory shifts.

According to urban transit evaluations from the Ho Chi Minh City Department of Construction, while the municipality's public passenger transport network has undergone recent structural expansion, its aggregate modal share remains critically suppressed, satisfying only approximately 7% of commuter travel demand in 2025 (Ho Chi Minh City People's Council, 2026). To remedy this baseline deficit, the Department of Construction proposed a pilot framework in mid-2026 for a Demand Responsive Transport (DRT) system—a flexible public transport mechanism engineered to dynamically adapt routes based on real-world commuter demand profiles rather than fixed, rigid timetables (Ministry of Construction of Vietnam, 2025). This institutional shift underscores an immediate, recognized need to pivot toward dynamically optimized, demand-weighted transit loops. Developing an exact optimization workflow for a demand-focused feeder bus system—modelled on exact mathematical solution frameworks capable of navigating highly constrained network topologies to achieve global optimality—directly answers this regulatory mandate. This approach provides an operationally feasible mechanism to maximize spatial demand capture and establish a competitive, high-utility first-mile gateway to Metro Line 1.

1.3. Objectives of Study:

This study aims to address the underutilization of feeder buses connecting to Ho Chi Minh City's MRT stations by designing demand-weighted first-mile feeder bus loop routes that (i) expand effective coverage area and (ii) capture higher potential ridership under strict travel-time constraints. The objective is to improve first-mile accessibility in a way that is operationally feasible for real deployment, where passenger acceptance is sensitive to door-to-door travel time, waiting time, and transfer friction.

The results of this research are expected to benefit multiple stakeholder groups. Public transport planners and urban planners can use the proposed approach to support network integration and station catchment planning in the context of transit-oriented development. TOD researchers can apply the demand-weighted framework to better quantify and operationalize “accessibility” around rail stations. From a green logistics and operations research perspective, the study also contributes to researchers working on vehicle routing and constrained network design problems, where the feeder-bus loop is a real-world variant of demand-weighted routing under time limits. In addition, electric feeder-bus operators and deployment enterprises may use the outputs to prioritize service corridors and operational plans aligned with demand density. Finally, the Ho Chi Minh City Urban Railway Company (HURC) can leverage the proposed optimized routes as decision support for improving multimodal integration and ridership performance of the current and future metro network.

In terms of outputs and applications, the study is expected to deliver at least one optimized feeder bus loop route that demonstrates higher demand coverage within a fixed time budget when compared with the limited set of existing feeder routes. Practically, this provides an implementable candidate route design that can be evaluated for coverage, feasibility, and demand capture. Scientifically, the study contributes a data-driven, demand-weighted optimization workflow for

first-mile access—bridging TOD concepts (accessibility and station-area integration) with constrained routing methods from operations research. This foundation is also transferable to future network expansion, offering methodological support for planning first-mile connectivity for upcoming lines such as Ho Chi Minh City Metro Line 2, where early-stage integration planning can materially affect future ridership and sustainability outcomes.

1.4. Scope and Limitation:

1.4.1. Scope of the study

This thesis is bounded to a single-station, first-mile feeder design problem in Ho Chi Minh City Metro Line 1. The geographic scope is restricted to the Vietnam National University (VNU) Urban Area and its immediate surrounding catchment, with the optimization targeting only one metro access point: National University Metro Station. The study focuses on designing an operationally feasible circular feeder bus loop serving station access demand within this defined area. To maintain a clear and implementable routing structure, the scope considers one directional movement only (i.e., forward/clockwise operation) for the circular loop.

In terms of service scope, the research explicitly addresses first-mile travel—collecting passengers from demand zones and delivering them to the metro station—while excluding last-mile services (distributing passengers from the metro station back to neighborhoods). The problem is framed as a demand-weighted network design and routing task under time constraints, where the primary performance target is to increase effective coverage and potential captured demand for station access within a route time budget. The research is conducted as a case study, and results are interpreted as decision support for route planning in similar station-area contexts rather than as a full-city network redesign.

1.4.2. Data scope, sources, and primary assumptions

The dataset is based on secondary geospatial data collected from Google Maps and Google Earth. Data inputs include: (i) the GIS coordinates and topology of candidate bus stops/road links within the study area, (ii) the polygon boundary representing the VNU Urban Area research region, and (iii) estimated population distribution within the boundary used as a proxy for potential feeder demand. Because GIS features and demographic conditions evolve over time, the outputs of this thesis are most meaningful under the assumption that the urban form, station access structure, and

major land-use patterns remain stable or experience only minor changes relative to the study period, which is Q1/2026.

To convert population topology into a tractable mathematical representation while preserving practical interpretability, the study approximates the research area using a polygon boundary and discretizes it into equal-sized grid cells. Each grid cell is assigned a demand weight (proxy population), and the optimization model uses these weights to evaluate coverage and served demand under route time constraints. A critical assumption concerns the study boundary itself: there is currently no single official administrative resolution that defines the exact geographic boundary of the VNU Urban Area. Therefore, the research boundary is constructed by synthesizing the most recent credible references and available descriptions that mention the area's extent. This boundary definition is treated as an analytical approximation and is documented transparently as part of model inputs.

1.4.3. Practical constraints considered

The model incorporates practical constraints to ensure feasibility and policy relevance, including:

- Single-station focus: all routes are designed to connect demand zones to National University Metro Station.
- Directional loop structure: the bus loop operates in one direction (forward/clockwise) to avoid operational complexity.
- Time budget constraint: the loop must satisfy a strict route time limit (representing service reliability and passenger tolerance for in-vehicle access time).
- First-mile only: routing decisions are optimized for station access; last-mile distribution is excluded from the objective and constraints.

- Spatial discretization: demand is represented at grid-cell level; route evaluation is based on grid coverage and demand weights.

1.4.4. Limitations

Several limitations may affect the interpretation and generalizability of results. First, the study relies on secondary data sources (Google Maps/Earth) rather than primary operational data (e.g., smartcard records, bus AVL/GPS, on-board counts). This may introduce measurement uncertainty in road attributes, stop locations, and travel-time realism. Second, the use of estimated population weights as a demand proxy does not fully capture behavioral factors such as willingness to transfer, fare sensitivity, service perception, or competing access modes (e.g., motorbike parking availability). Third, the grid-cell discretization simplifies continuous spatial demand patterns and can create sensitivity to grid size and boundary definition; results may vary if different resolutions or boundary assumptions are used.

Finally, the single-station and first-mile-only scope improves analytical clarity but limits the ability to infer network-wide impacts such as inter-station interactions, system-level bus fleet allocation, or corridor-level coordination across multiple metro stations. These factors are beyond the current study's control but may influence real-world effectiveness during deployment. Accordingly, findings should be interpreted as station-area decision support and a methodological foundation for scaling to multi-station or integrated first–last-mile planning in future work.

Chapter 2: RELATED WORKS

2.1. Overview:

2.1.1 Theoretical foundations

The Feeder Bus Network Design Problem (FBNDP) is traditionally grounded in the foundational mathematical formulation introduced by (Kuah & Perl, 1989), which frames feeder services as a strategic network design task: selecting feeder routes (and often their service levels) to connect dispersed demand zones to one or multiple rapid-transit stations under operational and service constraints while minimizing system cost and/or maximizing accessibility. Building on this origin, the FBNDP constitutes a strategic layer in urban mobility planning and serves as the critical “first and last mile” link that determines whether high-capacity transit investments—such as HCMC Metro Line 1—can effectively attract riders. In practical terms, FBNDP solutions seek to create seamless connectivity between residential catchments and the metro “spine,” thereby improving station access, reducing generalized travel cost, and supporting a mode shift away from private vehicles toward public transit. Mathematically, the FBNDP is classified as an NP-hard, non-convex optimization problem. This classification stems from the immense computational complexity required to navigate multi-dimensional search spaces where objectives—such as maximizing service coverage and minimizing operational costs—often conflict. Because of this complexity, the problem necessitates advanced mathematical frameworks and innovative solution methods beyond static models.

The FBNDP fundamentally derives from the classical Vehicle Routing Problem (VRP). In this context, the objective is to determine the optimal set of routes for a fleet of vehicles to visit specific locations while adhering to various operational constraints. These constraints typically include:

- **Time Windows:** Ensuring service delivery within set time frames to maintain schedule reliability.
- **Demand Indicators:** Incorporating potential passenger demand, often modeled through accessibility indicators or priority indices derived from GIS-based demographic data.
- **Operational Factors:** Accounting for vehicle capacity, total travel time, and the distance between urban links and the target metro station

2.1.2 Current System Investigation

The Vietnam National University, Ho Chi Minh City (VNU-HCM) urban area encompasses a vast expanse of 643.7 hectares located in Linh Xuan Ward, Ho Chi Minh City. Strategically situated at the eastern gateway of the city, the VNU-HCM urban area serves not only as a crucial center for higher education and academic research but also as the nucleus of the highly interactive innovative urban area in Eastern Ho Chi Minh City (Vietnam National University Ho Chi Minh City [VNU-HCM], 2023). This strategic positioning provides a significant driving force for regional socio-economic development and expands opportunities for domestic and international collaboration in education, science, and technology.

To accommodate the extensive daily commuting demands of tens of thousands of students, staff, and faculty members, the internal public transportation system currently maintains an estimated network of 4 feeder bus routes. These routes' transit points are systematically distributed along the main transport corridors and adjacent to the campuses of member universities. With the implementation of the HCMC Metro Line 1 (Ben Thanh – Suoi Tien), establishing a seamless "first and last mile" connection between the university campus and the mass rapid transit spine has become a critical mobility objective. Accordingly, a network of feeder bus routes has been designated to connect the VNU-HCM area directly to the metro stations situated along the Hanoi Highway (Ho Chi Minh City Management Centre of Public Transport [MCPT], 2024).

By investigating the official public transport application from Ho Chi Minh City Department of Transport, Key feeder routes integrating the campus with the rapid transit network include:

Route 166: Serves as a high-frequency transit link dedicated to transferring passengers from the core VNU-HCM village directly to the National University Station, with the following stations below:

To Metro Station:	National University Station
Total number of bus station	15 stations
Total Distance	6.60 km
Total Route time	18 minutes

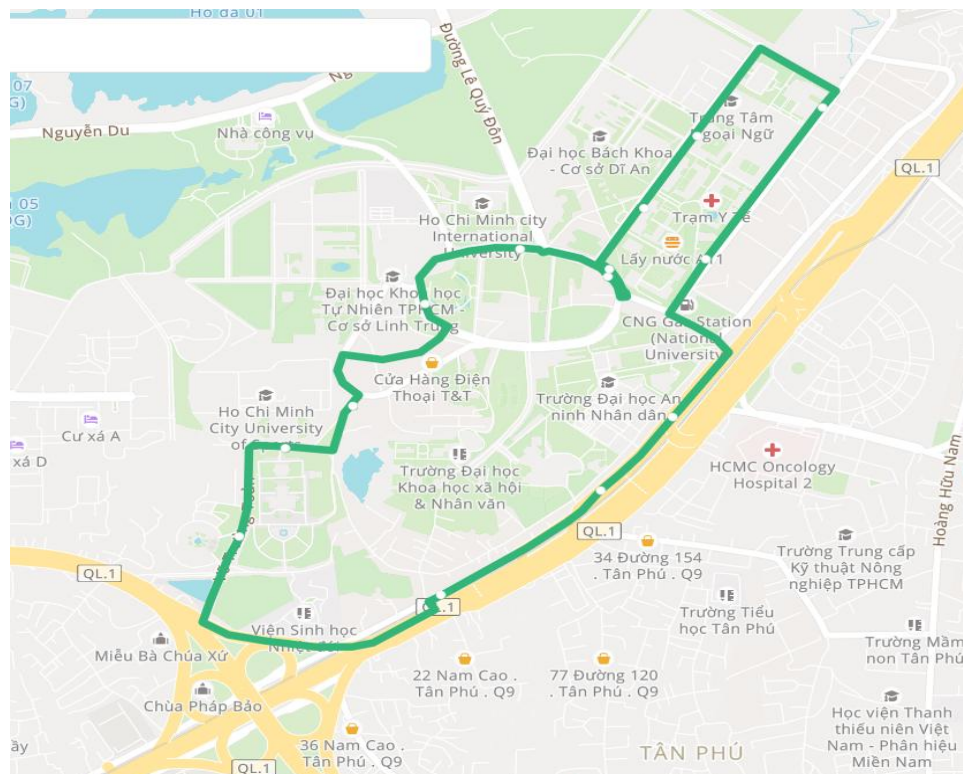


Figure 3. Bus Route 166 Map

Route 164V: Connects the Nong Lam University axis and nearby residential apartments to the High-Tech Park Station and National University Station, as shown below:

To Metro Station:	National University Station
Total number of bus station	18 stations
Total Distance	10.0 km
Total Route time	35 minutes

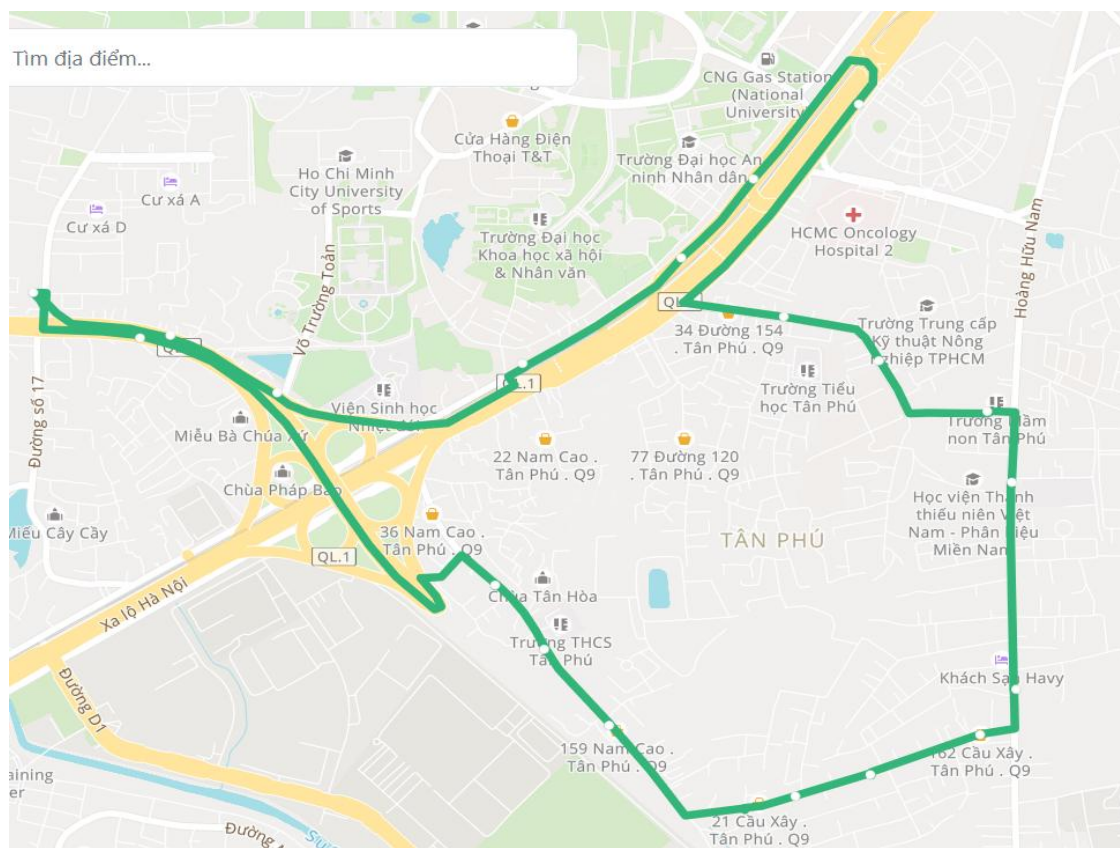


Figure 4. Bus Route 164V Map

Route 167: Links Nong Lam University to the High-Tech Park Station, passing through the Linh Trung 1 Export Processing Zone.

To Metro Station:	High-Tech Park Station
Total number of bus station	20 stations
Total Distance	13.00 km
Total Route time	55 minutes

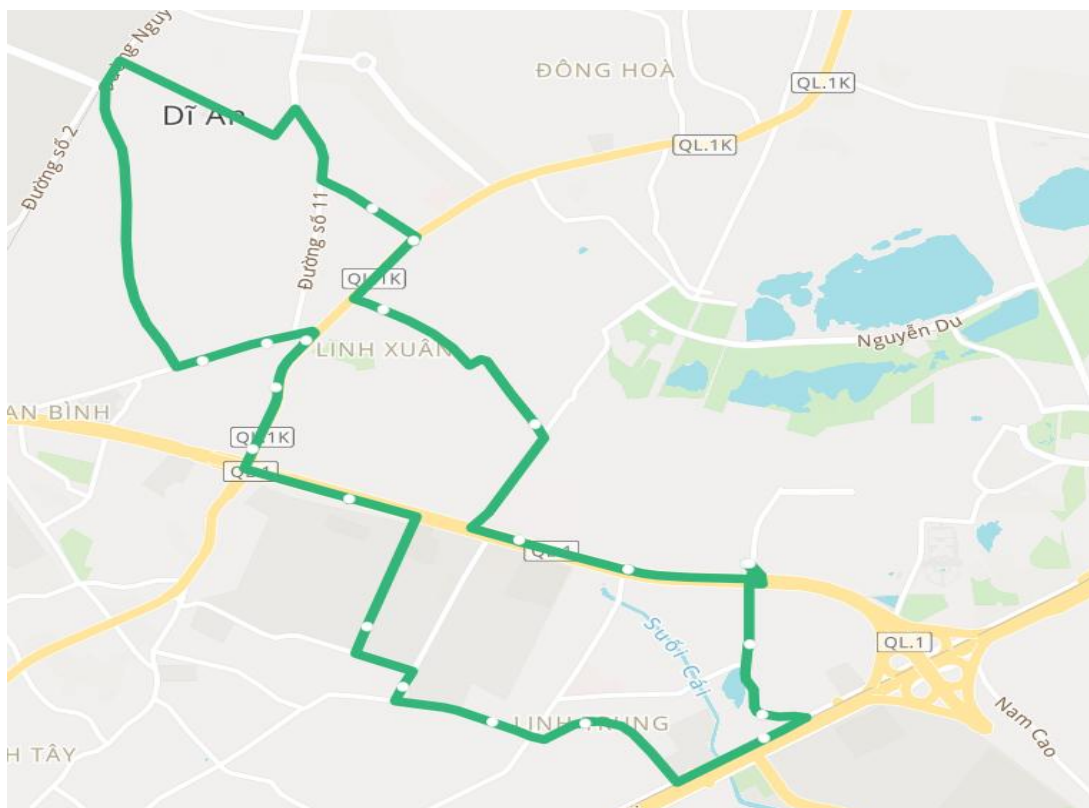


Figure 5. Bus Route 167 Map

Route 170: connects key locations within National University Area, especially student dormitories and big universities to the nearest metro station - National University Station.

To Metro Station:	National University Station
Total number of bus station	21 stations
Total Distance	14.5 km
Total Route time	45 minutes

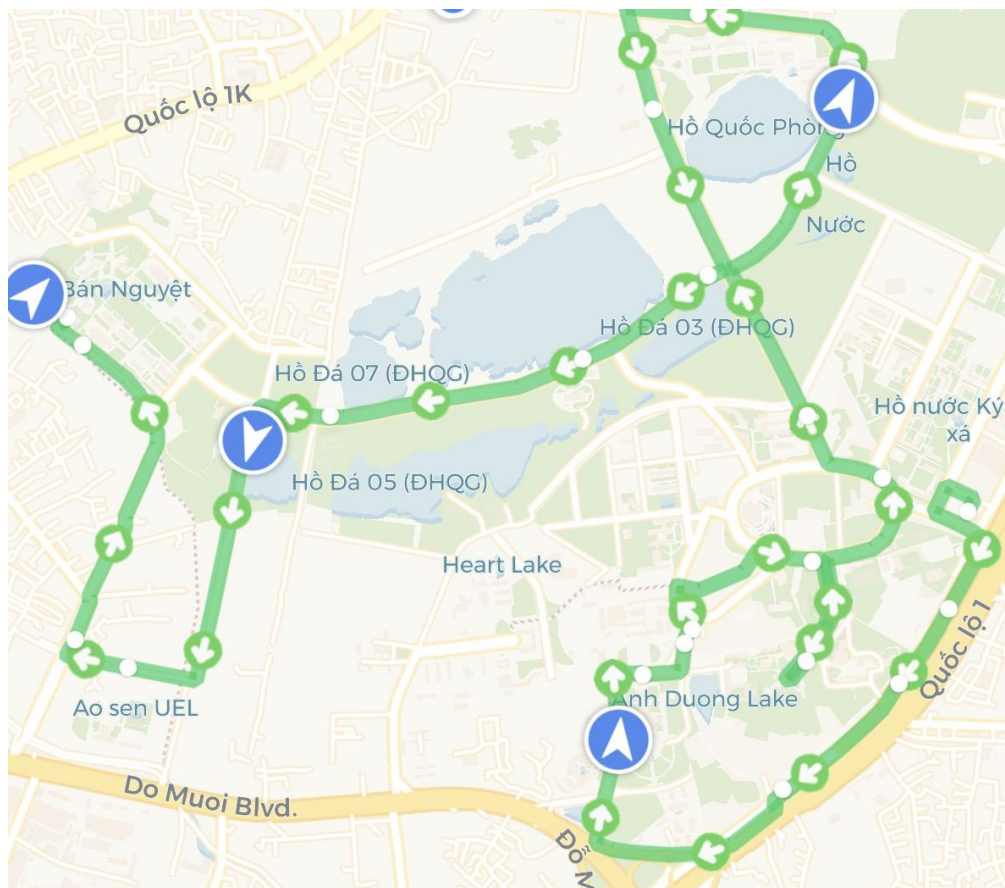


Figure 6. Bus Route 170 Map

2.2. Literature Review:

2.2.1 Investigating Current Development Approaches

The Feeder Bus Network Design Problem (FBNDP) connecting to urban railway systems has evolved significantly from static representations to dynamic, spatially aware frameworks. Current development approaches in the literature can be broadly categorized into analytical continuum models, metaheuristic optimizations, and advanced machine learning simulations.

Analytical approaches, such as the Continuum Approximation (CA) model proposed by Zhen and Gu (2024), address spatially heterogeneous demand over large service regions. Their methodology jointly optimizes stop spacings, line spacings, and headways by treating discrete variables as continuous density functions, while explicitly accounting for passenger boarding numbers and transfer delays at rail terminals. The most dominant methodological stream involves metaheuristic algorithms integrated with spatial and demographic data. Wei et al. (2021) formulated a bi-level programming model that leverages open-source GIS and aggregated cellular data. Their approach uses a "stops-first" design to minimize passenger walking time before executing an Ant Colony Optimization (ACO) heuristic to determine the optimal route. Similarly, Calabrò et al. (2023) combined GIS-based demographic data with an ACO algorithm embedded within an agent-based modeling environment. Their model relies on gravity-based active and passive accessibility indicators to estimate potential demand at each stop. Cao et al. (2022) utilized a Genetic Algorithm (GA) to solve a multi-objective optimization problem that balances bus operation costs against passenger travel costs. Their model specifically limits generalized travel costs by introducing penalty functions for excessive station transfers. Furthermore, Hemdan et al. (2026) utilized dynamic programming alongside a pulse algorithm; their approach maximizes the serviced population by implementing an edge-repetition elimination technique, which drastically increases the network's spatial coverage.

Finally, microscopic operational validation and competitive modeling represent the latest advancements in the field. Ding et al. (2017) utilized a GA based on route properties to optimize a campus bus network,

subsequently evaluating its operational feasibility through VISSIM discrete-event simulation. For highly complex transit environments, Su and Yang (2025) proposed a framework based on Multi-Agent Reinforcement Learning (MARL). They introduced a Cost-based Competitive Coverage (CCC) model to dynamically adapt feeder routes by evaluating them against competing travel modes, such as direct walking or ordinary bus services.

2.2.2 Design Concepts Consideration

Drawing from the reviewed literature, the proposed feeder bus network design for the VNU-HCM area is structured around the conceptual integration of three key references: Hemdan et al. (2026), Calabrò et al. (2023), and Cao et al. (2022).

First, the design adopts the concept of a demand-weighted spatial network established by Calabrò et al. (2023). By utilizing GIS-derived socio-demographic data to calculate active and passive accessibility indicators, the proposed model ensures that the routing algorithm prioritizes high-demand clusters. For the VNU-HCM context, this means strategically weighting specific nodes like university dormitories and high-density faculties, avoiding the unrealistic assumption of uniform student distribution across the campus.

Second, the system's operational coverage draws directly upon the optimization techniques of Hemdan et al. (2026). The conceptual design incorporates their strategy of maximizing the serviced population under strict travel-time constraints by explicitly preventing unnecessary edge repetition along the transit graph. This topological rule ensures that the generated circular feeder routes efficiently expand the effective catchment area connecting to the HCMC Metro Line 1, preventing vehicles from wasting operational time traversing the same internal campus links.

Finally, the formulation of the multi-objective cost function is heavily guided by the station transfer constraints identified by Cao et al. (2022). Recognizing that the transition between the campus feeder bus and the mass transit spine introduces significant temporal and psychological friction, the proposed design integrates specific transfer penalties into the generalized passenger travel cost calculations. By explicitly

balancing these weighted passenger costs against the operator's distance-based routing costs, the VNU-HCM model guarantees a realistic, user-centric approach that actively incentivizes students to shift toward public transit ridership.

2.3. Key references/ Candidate Solution Methods

1. Dynamic Programming and the Pulse Algorithm

The Pulse Algorithm, integrated within a dynamic programming framework, represents a robust exact optimization method typically applied to the Constrained Shortest Path Problem (CSP) by Lozano and Medaglia (2012). As utilized by Hemdan et al. (2026), the algorithm operates by propagating exploration "pulses" through a network graph starting from a designated source node (e.g., a metro station). As these pulses traverse the graph's edges toward various destination nodes, they accumulate both costs (such as travel time or distance) and rewards (such as the number of passengers served). To prevent the combinatorial explosion typical of NP-hard routing problems, the algorithm employs aggressive bounding and pruning strategies; if a pulse reaches a node with accumulated costs that violate predefined constraints (like a maximum allowable route duration), or if it is strictly dominated by another path, the pulse is terminated (pruned).

Rationale: The primary rationale for selecting the Pulse Algorithm in feeder bus network design lies in its ability to guarantee an optimal solution under strict operational constraints, avoiding the trap of local optima that often plagues heuristic methods. Hemdan et al. (2026) specifically adapted this algorithm to solve the feeder bus routing problem by introducing an edge-repetition elimination technique. In the context of the VNU-HCM to Metro Line 1 connection, this exact method is highly valuable because it computationally forces the generation of loop or circular routes that maximize the spatial coverage of the serviced student population without wasting operator time traversing the same road segments twice.

2. Ant Colony Optimization (ACO)

Ant Colony Optimization (ACO) is a swarm intelligence metaheuristic inspired by the foraging behavior of natural ant colonies. In mathematical routing, "artificial ants" construct solutions probabilistically by moving through a graph of potential bus stops. The probability of an ant choosing the next stop is determined by two factors: heuristic information (a greedy preference for shorter distances or higher demand) and a pheromone trail (historical memory of successful routes). Once all ants complete their routes, the pheromone trails are updated; paths that yield higher-quality solutions (e.g., lower travel time, higher ridership) receive a stronger pheromone deposit, guiding subsequent iterations toward the global optimum (Wei et al., 2021).

Rationale: The Feeder Bus Network Design Problem is inherently multi-objective, requiring a delicate balance between operator costs and passenger convenience. The rationale for employing ACO, particularly within the agent-based modeling (NetLogo) environment proposed by Calabrò et al. (2023), is its superior capacity to dynamically adapt to spatially distributed weights. Calabrò et al. (2023) utilized gravity-based accessibility indicators derived from GIS demographic data to influence the ACO's heuristic parameters. This makes ACO an exceptional candidate for the VNU-HCM case study, as the algorithm can organically prioritize high-density student nodes (like KTX Khu A) and organically "discover" optimal feeder paths that static analytical models might overlook, significantly enhancing the "demand-weighted" aspect of the network design.

3. Genetic Algorithms (GA)

Genetic Algorithms (GA) are evolutionary search heuristics grounded in the principles of natural selection and genetics. In the context of transit routing, a GA encodes a potential solution (a complete feeder bus network) into a "chromosome," typically represented as an array or string of node sequences (bus stops). An initial population of these chromosomes is generated, and each is evaluated using a fitness function. The algorithm then simulates evolution through selection (favoring high-fitness networks), crossover (combining segments of two parent routes to create a novel offspring route), and mutation (randomly altering a stop sequence to maintain genetic diversity and prevent premature convergence).

Rationale: The primary rationale for utilizing a Genetic Algorithm is its exceptional flexibility in handling highly complex, non-linear, and multi-objective fitness functions. Cao et al. (2022) specifically leveraged a GA framework because their model introduced unique penalty functions for station transfers, evaluating the "generalized travel cost" which includes the psychological and temporal friction of moving from a bus to a rail system. Furthermore, Ding et al. (2017) demonstrated that GA is highly effective for campus bus networks when evaluated against route properties and variable passenger demands. For the VNU-HCM thesis, GA provides a robust framework capable of mathematically penalizing excessive travel times and transfer delays, ensuring the final network configuration aligns with the commuters' true cost of travel while maintaining feasible fleet sizes for the operator.

Chapter 3: METHODOLOGY

3.1. Approaches Comparison and Selection:

To establish a robust methodological framework for optimizing feeder bus routes in the VNU-HCM urban area, it is necessary to conduct a comparative analysis of the leading algorithmic candidates identified in the literature. This section evaluates the Pulse Algorithm, Ant Colony Optimization (ACO), and Genetic Algorithms (GA) based on their mathematical logic, operational advantages, and suitability for high-precision Geographic Information System (GIS) integration.

Qualitative Comparison of Optimization Approaches:

The following table synthesizes the core rationale and qualitative characteristics of the three primary approaches considered for this research:

Table 1. Methodologies Comparison Table

Attribute	Pulse Algorithm (Hemdan et al., 2026)	Ant Colony Optimization (Calabrò et al., 2023)	Genetic Algorithm (Cao et al., 2022)
Key Reference	<i>Hemdan et al. (2026)</i>	<i>Calabrò et al. (2023)</i>	<i>Cao et al. (2022)</i>
Core Rationale (Objectives & Variables)	Objective: Maximize serviced population within travel time limits. Variables: Binary decision variables for link selection and pulse flow.	Objective: Multi-objective (ridership vs. distance). Variables: Pheromone intensity and heuristic desirability on edges.	Objective: Minimize generalized travel cost + transfer penalties. Variables: Chromosome encoding of stop sequences and headways.
Primary Advantages	Exact Method: Guarantees global optimum for loop searches. GIS Efficiency: Simplifies spatial data by attaching "Grid Cells" to links for direct processing.	Adaptive Search: Excellent for dynamic agent-based environments. Incorporates gravity-based accessibility.	Flexibility: Handles non-linear cost functions and psychological transfer penalties effectively.
Primary Disadvantages	High computational demand if pruning strategies are not aggressively optimized.	Heuristic nature; potential for local optima and high sensitivity to parameter tuning.	Computational complexity in encoding spatial constraints into fixed-length chromosomes.

Based on the comparative analysis, this thesis selects the Pulse Algorithm as the primary methodological framework for optimizing the feeder bus routes connecting VNU-HCM to Metro Line 1.

Unlike metaheuristics (ACO and GA) which provide "near-optimal" solutions based on stochastic search, the Pulse Algorithm—as an exact method—ensures the identification of the absolute global optimum for the defined objective function. In the context of large-scale infrastructure like HCMC Metro Line 1, a guaranteed optimal solution is essential for justifying fleet allocation and route efficiency to urban planners.

A definitive advantage of the Pulse Algorithm is its ability to simplify complex GIS data into mathematical variables understandable by the decision-making engine. By attaching demographic Grid Cells directly to the physical links (edges) of the road network, the algorithm can process spatial demand weights without the abstraction layers required by heuristic methods. This creates a direct pipeline between Geographic Spatial Analysis and the optimization variables, ensuring that the decision to include a specific road segment is based on real-world population density. As demonstrated by Hemdan et al. (2026), the Pulse Algorithm is specifically optimized for the "First-Mile" problem through its edge-repetition elimination logic. This is highly relevant for the VNU-HCM campus, where feeder routes must be designed as efficient, non-overlapping loops that maximize student pick-up points across disparate faculties and dormitories while maintaining strict travel-time synchronization with the Metro schedule.

3.2. Proposed System Design

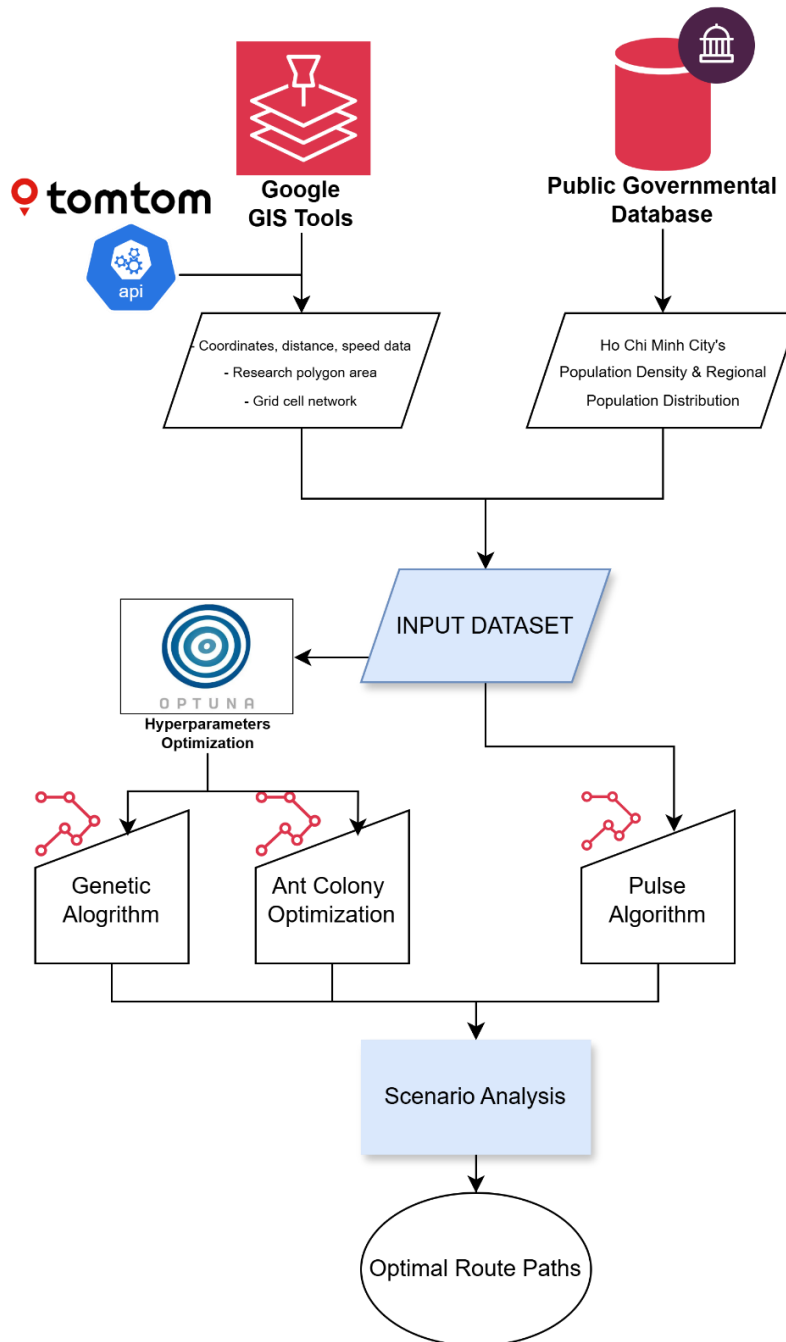


Figure 7. Research Framework

The research framework for this study integrates advanced Geographic Information Systems (GIS), public socio-demographic data, and heuristic optimization to design an efficient feeder bus network for the Vietnam National University (VNU) area. The initial phase of the methodology employs Google GIS tools, specifically **Google My Maps**, to delineate the research boundary by mapping a polygon area defined by connected vertices. Within this defined spatial scope, all existing and active bus stations are marked with precise coordinates to retrieve a comprehensive GIS data profile of the VNU area and its current infrastructure. Following the establishment of these spatial nodes, stations are manually connected to construct a comprehensive Origin-Destination (OD) matrix of all feasible links. To populate this matrix with real-world traffic dynamics, the **TomTom API** is utilized to request access to GIS routing services, providing critical parameters such as link-specific routing distance and average travel speed.

The second component of the framework involves the synthesis of population demand data with the physical network topography. Socio-demographic indicators, including population density and regional distribution over time in Ho Chi Minh City, are retrieved from the **public database of the National Statistics Office (NSO) of Viet Nam**. To localize these broad statistics, **Google Earth Engine** is used to overlay the VNU polygon area with a variable-sized grid network. This technique allows for the disaggregation of the total population into smaller, discrete grid cells, each filled with calculated population values to reflect heterogeneous demand across the study area. Once the spatial and demographic data are synchronized, the resulting information is assembled and cleaned to form a complete input dataset, which serves as the foundation for training and executing the proposed optimization models.

The final phase of the framework focuses on the comparative optimization of the network through sophisticated algorithmic tuning. To achieve peak performance for the Genetic Algorithm (GA)

and Ant Colony Optimization (ACO) models, the Optuna library is implemented in a Python environment. As proposed by Akiba et al. (2019), Optuna serves as a next-generation hyperparameter optimization framework, utilizing automated tuning to determine the most effective parameters for each heuristic approach. The study then executes a rigorous comparison between three primary algorithms: the tuned Genetic Algorithm, the tuned Ant Colony Optimization, and the Pulse Algorithm. By running these models across diverse contexts and operational scenarios, the framework evaluates their performance metrics—such as captured demand and travel time—to determine the optimal feeder bus route paths for the VNU urban rail interface.

Before executing the optimization algorithms, it is essential to define the core drivers of the optimization model: the Priority Index and the Served Area. The exact mathematical formulations for these concepts will be fully detailed in **Chapter 4: Solution Development**.

- **Priority Index (PI):** The primary objective of the algorithm is to Maximize the Priority Index under the network's time constraints. The PI is defined as an integer-based ranking of the potential demand for each link between two random nodes. By converting raw potential demand into an integer ranking, the model emphasizes the priority differences between specific edges, significantly improving computational speed and guiding the algorithm to target the most critical demand corridors.
- **Served Area as another final evaluation metric:** While the algorithm optimizes for the highest PI, the ultimate measure of the route's success is its physical geographic coverage. The Served Area metric evaluates the total coverage area of the selected route path. It calculates the sum of the inhabited grid areas assigned to the links in the final path , allowing us to evaluate how effectively the network expands its footprint to serve the population.

Chapter 4: SOLUTION DEVELOPMENT

4.1 Data Collection & Model Construction



Figure 8. VNU Map Polygon with station marks

The map figure illustrates the spatial framework established for the study, in which a total of 92 bus stations were georeferenced and mapped within the VNU-HCM vicinity using the GIS system *Google Earth Engine*. Each station was assigned a unique numerical identifier to support consistent spatial referencing, network construction, and subsequent link generation in the feeder bus modeling process. The distribution of these stations indicates coverage along the main internal and surrounding road corridors, thereby reflecting the potential access structure for first-mile connectivity to the metro system. In parallel, a polygon representing the defined VNU study area was delineated to specify the geographical boundary of analysis. Based on the mapping interface, this polygon covers an area of approximately 13 km² and has an estimated perimeter of 14.4 km.

The boundary polygon serves an important analytical function by constraining the study scope, ensuring that station selection, feasible link identification, and route optimization are conducted within a consistent and relevant service area.

Table 2. Sample data table of 92 stations

station_id	longitude (decimal degree)	latitude (decimal degree)	station_name
1	106.8029	10.87711	International University, VNU-HCM
2	106.8028	10.87813	International University, VNU-HCM - Le Quy Don Intersection - Thomas Edison Street Street - Linh Trung Ward
3	106.8025	10.87663	International University, VNU-HCM - Creative Square - Linh Trung Ward
4	106.8025	10.87641	International University, VNU-HCM - Opposite International University - Creative Square - Linh Trung Ward
5	106.8049	10.87564	VNU-HCM Area A Dormitory - Creative Square Intersection (Outbound) - Creative Square - Linh Trung Ward
6	106.8051	10.87578	VNU-HCM Area A Dormitory - Creative Square Intersection (Inbound) - Creative Square - Linh Trung Ward
7	106.8061	10.87465	University of Science, VNU-HCM - Opposite CNG Station - Road 621
8	106.8071	10.87431	VNU-HCM Ring Road - VNU-HCM Utility Pole 6 (Mua Bui Cafe) - Road 621 - Linh Trung Ward
9	106.8076	10.8759	Area A Bus Terminal - No. 28-30 Nguyen Binh Khiem Street - Linh Trung Ward
...	...	...	

The primary spatial dataset for the study area comprises a network of 92 candidate bus stops within the Vietnam National University (VNU) Urban Area. These locations were meticulously identified and marked using Google My Maps to ensure comprehensive coverage of the "first-mile" catchment area surrounding the HCMC Metro Line 1 terminus. This digitization process involved mapping the physical locations of existing transit infrastructure alongside high-density residential and academic hubs, effectively establishing the nodal basis required for the subsequent Feeder Bus Network Design Problem (FBNDP) optimization. To ensure high-precision spatial analysis and interoperability with Geographic Information System (GIS) environments, the geographical coordinates for all 92 stations were recorded using the EPSG:4326 coordinate reference system, adhering to the World Geodetic System 1984 (WGS 84) standard. The location data, stored as latitude and longitude in decimal degrees, was retrieved from the GIS service and transformed into a structured tabular format.

Table 3. Input Data Field Demonstration

No.	Data Field	Sample Value
1	link_name	1_3, 1_2, 1_25,
2	link_type	Forward, U-turn, Backward
3	Distance (in metre)	142, 111, 308,
4	Average_travel_time (in seconds)	23, 16, 47,
5	Average_speed (in kmh)	22.22608696, 24.975, 23.59148936,....
6	Assigned_grid_area (in m2)	630, 0, 6300,
7	Pop _{ij} (in citizens)	1.429816771, 0, 43.30485724,...
8	S _{ij} (in metre)	4.713433499, 0
9	forward_source_link_name	1_3, 1_2, 1_25,

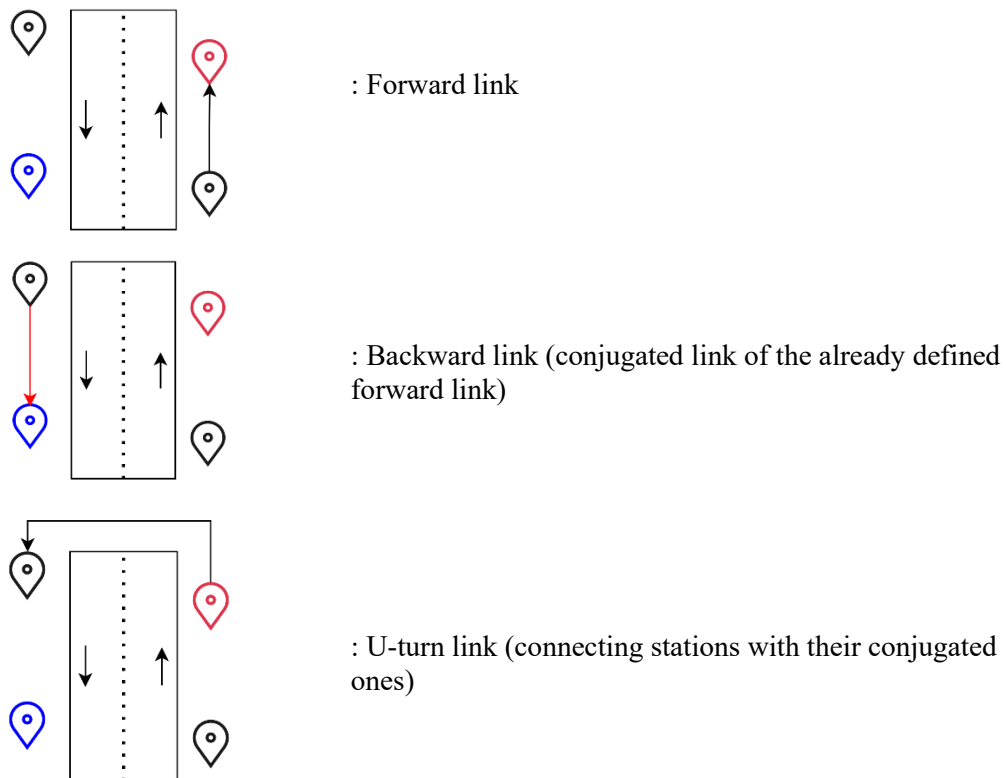


Figure 9. Link type description

The table of model input data fields as in table 3 show all the essential data to feed in the optimization model. For the *No. 1*, link_name is created based on the station_id, where the number before “_” represents the Origin station_id standing for “i” and the number after “_” represents the Destination station_id standing for “j”, thereby forming a connect link call (i,j) with the total number of feasible links – 271 links . For *data field No.2*, each directional edge is classified using the variable link_type, as demonstrated in the figure 8 to distinguish its operational and demand-assignment role. Each specific link (i,j)’s link_type or directions are manually recorded from the 1st link to the 271st link. A Forward link represents the primary directional connection between two base station nodes and follows the original physical traffic direction of the road segment. Since the population-grid assignment process is first performed on the original road network, Forward links serve as the main demand-carrying links. Therefore, all population grids, potential demand values, served area values, and related priority-index attributes are initially assigned to this link type.

A Backward link represents the conjugated counterpart of an already defined Forward link. It connects the conjugated station nodes in the opposite physical traffic direction. Rather than receiving a separate population-grid assignment, each Backward link inherits the number of assigned grids and demand-related attributes from its corresponding Forward link, as recorded in the field forward_source_link_name. This treatment prevents double-counting of demand while still allowing the optimization algorithm to consider movement in the opposite direction of the same physical road segment.

A U-turn link is defined as a virtual connector between a station node and its conjugated node. Its function is mainly topological: it allows the route to transfer between the original and mirrored layers of the network when constructing a circular feeder bus route. Unlike Forward and Backward links, U-turn links do not represent demand-serving road segments. Therefore, they are assigned

zero population grids, zero potential demand, and zero priority index. This logic is consistent with the mirrored-network structure, where virtual connections are used to link original and conjugated nodes while carrying no demand value.

The variables Distance, Average_travel_time, and Average_speed were incorporated to represent the physical and operational characteristics of each feasible link in the constructed feeder bus network. For each candidate link(i,j), the road-based distance was retrieved using the TomTom API with bus mode configuration. However, a candidate connection was only accepted as a feasible link when the calculated route between the origin and destination station did not pass through any intermediate station among the remaining 90 station nodes. This condition was applied to ensure that each accepted link represents a direct and indivisible station-to-station connection. If a route passed through another station before reaching its destination, the connection was excluded because it would overlap with another possible link and could distort the structure of the network as shown in figure 9. After this screening process, 271 feasible links were retained for the final feeder bus optimization graph.

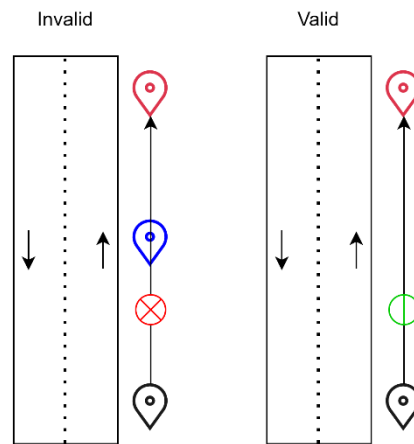


Figure 10. Valid/Invalid Links

The Average_speed field was also obtained using the TomTom API. For each feasible link, TomTom Routing API (Calculate Route | Routing API, n.d.) for bus mode was used to estimate the historical average speed along the corresponding road path. Compared with a single instantaneous speed observation, the historical average speed provides a more representative measure of normal traffic conditions because it reduces the influence of temporary congestion, incidents, or short-term fluctuations at the moment of data collection. After the distance and average speed of each feasible link were obtained, the average_travel_time was calculated from these two variables. In this dataset, average_travel_time represents the expected time required to traverse a link under the average traffic-speed condition provided by TomTom. It serves as the main time-based impedance variable in the optimization model and is used to check whether a candidate feeder bus route satisfies the maximum allowable route duration.

However, due to the geospatial complexity of the VNU study area, the TomTom API did not return complete route information for all feasible candidate links. Among the final set of 271 feasible links, only 186 links were successfully matched with both route distance and average speed from TomTom. The remaining 85 links were still topologically valid within the constructed network, but their feasible route distance and speed attributes could not be detected by the API. To avoid excluding these links from the optimization model, their route distances were manually collected using Google My Maps by tracing the corresponding station-to-station road paths.

For the 85 links without available TomTom speed outputs, a supplementary speed estimation procedure was applied. The hourly speed distribution of Ho Chi Minh City from the TomTom

Traffic Index in the figure below was used as the reference traffic-speed profile.

Table 4. Weekly Speed Distribution Table of Ho Chi Minh City

Time	Mon	Tue	Wed	Thu	Fri	Sat	Sun
12:00am	26.00	26.00	25.00	26.00	26.00	26.00	25.00
01:00am	27.00	27.00	25.00	26.00	26.00	26.00	26.00
02:00am	27.00	27.00	27.00	27.00	27.00	27.00	27.00
03:00am	28.00	28.00	27.00	28.00	28.00	28.00	27.00
04:00am	28.00	27.00	27.00	27.00	27.00	28.00	28.00
05:00am	26.00	26.00	26.00	26.00	26.00	26.00	27.00
06:00am	22.00	22.00	22.00	23.00	23.00	25.00	26.00
07:00am	18.00	18.00	18.00	18.00	18.00	23.00	24.00
08:00am	17.00	17.00	17.00	17.00	17.00	21.00	23.00
09:00am	18.00	17.00	18.00	17.00	17.00	19.00	21.00
10:00am	19.00	18.00	18.00	18.00	18.00	18.00	21.00
11:00am	19.00	18.00	18.00	18.00	17.00	18.00	21.00
12:00pm	20.00	19.00	19.00	19.00	18.00	18.00	21.00
01:00pm	20.00	20.00	20.00	19.00	19.00	19.00	22.00
02:00pm	19.00	19.00	19.00	19.00	18.00	19.00	21.00
03:00pm	19.00	19.00	19.00	18.00	17.00	19.00	21.00
04:00pm	18.00	17.00	17.00	17.00	16.00	19.00	20.00
05:00pm	15.00	15.00	15.00	15.00	14.00	18.00	19.00
06:00pm	15.00	15.00	15.00	15.00	14.00	17.00	18.00
07:00pm	18.00	18.00	18.00	17.00	16.00	18.00	19.00
08:00pm	20.00	20.00	20.00	20.00	18.00	18.00	19.00
09:00pm	21.00	21.00	21.00	21.00	19.00	19.00	21.00
10:00pm	23.00	22.00	22.00	22.00	21.00	20.00	22.00
11:00pm	25.00	24.00	24.00	24.00	24.00	23.00	24.00

Legend

14 km/h	21 km/h	28 km/h
---------	---------	---------

Lower speed = heavier congestion; higher speed = freer traffic flow.

Since speed is a rate variable, the harmonic mean was adopted instead of the arithmetic mean. The harmonic mean is more suitable for aggregating traffic speed because it gives greater influence to lower-speed periods, which are more representative of travel-time burden under congested urban conditions. In this study, the hourly speed values from **05:00 to 23:00, Monday to Sunday**, were used to represent the operational traffic condition of feeder bus services. The time-based harmonic average speed was calculated as follows:

$$V^H = \frac{N}{\sum_{d \in D} \sum_{h \in H} v_{dh}}$$

where V^H is the harmonic average speed of Ho Chi Minh City, v_{dh} is the observed speed for day d and hour h , D is the set of weekdays from Monday to Sunday, H is the hourly period from 05:00 to 23:00, and N is the total number of valid hourly observations. With seven days and nineteen hourly observations per day, the maximum number of observations is 133. The result of harmonic means average speed applied universally to all 85 unavailable links is

$$V^H = 19.17 \text{ km/h}$$

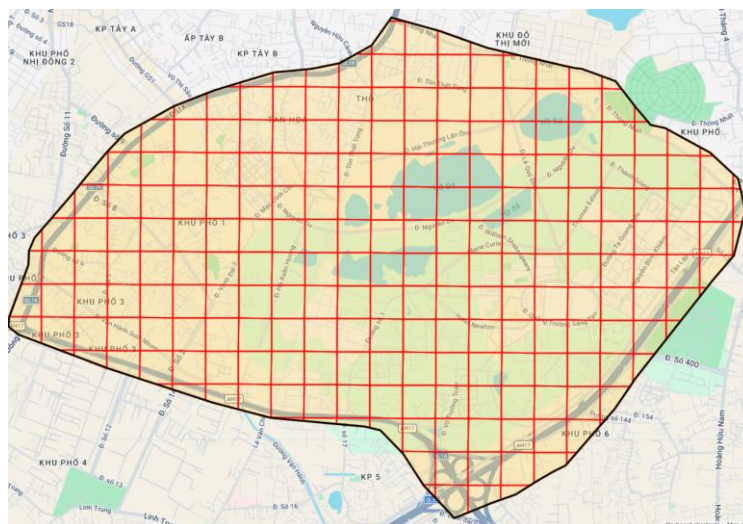
After obtaining the harmonic average speed, the average_travel_time of each manually completed link was calculated from the manually measured route distance and the estimated harmonic speed:

$$t_{i,j} = \frac{3.6D_{ij}}{V^H}$$

where D_{ij} is the manually measured route distance of link (i,j) in meters, V^H is the harmonic average speed in km/h, and average_travel_time or $t_{i,j}$ is expressed in seconds.

This procedure ensured that all 271 feasible links could be retained in the final optimization dataset.

The 186 links with complete TomTom outputs preserve link-specific API-based distance and speed information, while the 85 unavailable links are completed through manually measured route distance and a city-level harmonic speed profile.



Population estimation for each grid was developed using two official data sources. First, the 2024

population density of Ho Chi Minh City was retrieved from the National Statistics Office of Vietnam database. This value provided the baseline population intensity for estimating the total population within the VNU polygon area. Second, household distribution data were retrieved from the GIS portal of the General Statistics Office of Vietnam for the three wards forming the study area, namely Linh Trung, Linh Xuan, and Dong Hoa. These household distribution points were used to spatially adjust the population allocation across the grid system rather than assuming that population is uniformly distributed over the entire polygon. As a result, grids located closer to observed household clusters received higher population weights, while grids with limited or no household presence received lower estimated population values as shown in the figure 11.

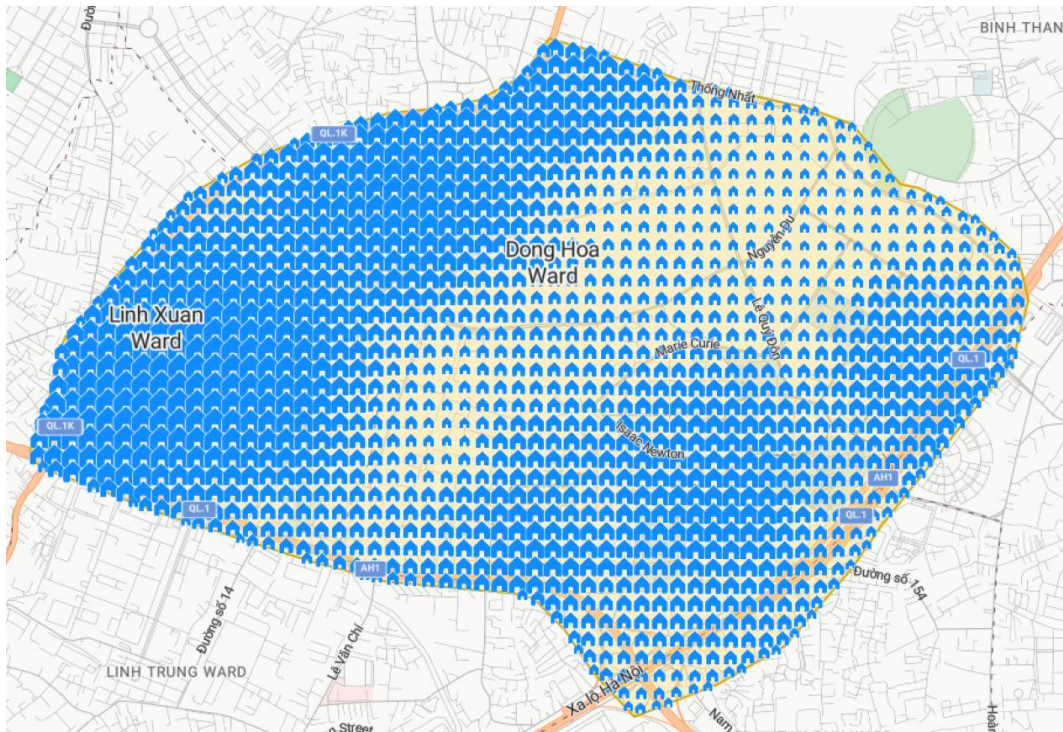


Figure 12. Population Distribution in VNU area

After the grid population was estimated, each grid centroid was spatially matched to the feasible network links. The assignment process follows the logic that passengers within the same grid are

represented by the grid centroid and are assigned to one corresponding link based on spatial proximity. This treatment follows the simplifying assumption used in feeder bus route design studies, where individuals within the same grid are assumed to share similar access characteristics, and the centroid-to-link distance is used to represent the passenger access distance from that grid to the feeder bus link (Zhu et al., 2017). The field `Assigned_grid_area` represents the total area of all grid cells assigned to a specific link(i, j). This variable is used to quantify the spatial service coverage of each candidate link. A larger assigned grid area indicates that the link is spatially associated with a wider portion of the study area, while a zero value indicates that no grid was assigned to that link. This field is also later used to calculate the served area of the optimized feeder bus route. The field `Popij` represents the estimated population assigned to link(i, j). It is obtained by aggregating the population values of all grid cells assigned to that link. In the feeder bus optimization model, `Popij` functions as the link-level population demand indicator. A higher `Popij` value implies that the link is expected to serve a larger number of potential passengers and therefore has stronger demand relevance in the route design process. The field `Sij` represents the average passenger access distance to link (i, j), measured in meters. It is calculated from the distances between the assigned grid centroids and the corresponding link. This variable reflects how far the assigned population is, on average, from the feeder bus link. A lower `Sij` value indicates better spatial accessibility because the assigned passengers are located closer to the link, while a higher value suggests weaker access convenience. In the mathematical model, `Sij` is essential because potential demand is not determined by population alone; it also depends on how easily that population can reach the candidate bus link.

4.2 Mathematical Model:

4.2.1 Indices and sets

Table 5. Table of Indices & Sets

Indices & Sets	Description
$G = (N, A)$	directed graph representing the feeder bus network
N	set of nodes in the transportation network.
$s \in N$	Target metro station, where $s = 31$ represents National University Metro Station
A	set of directed links (arcs) in the transportation network.
$(i, j) \in A$	directed link from node i to node j
Z	set of inhabited grids generated inside the study area
$g \in Z$	index of grid cell
$A_P \subseteq A$	set of links contained in a selected path P
$L_{i,j} = \{0,1,2\} \in L$	set of link types where: U-turn = 0; Forward = 1; Backward = 2
$K \subseteq A^1$	Set of Forward source links that have at least one Backward counterpart
$k \in K$	Index of a Forward source link
$A_k^2 \subseteq A^2$	Set of Backward links whose forward_source_link_name is Forward source link k
P	Selected circular feeder bus path
$A_P \subseteq A$	Set of directed links contained in path P

4.2.2 Parameters

The main parameters used in the model are defined as follows.

Table 6. Table of Parameters

Symbol	Description
pop_z	Total population of the inhabited zone.
ΔZ	Area of the inhabited zone.
λ	Population density of the inhabited zone (persons per square meter).
Δg	Area of grid g .
Pop_g	Population assigned to grid g .
$s_{g,ij}$	Shortest distance from the midpoint of grid g to link (i, j) .
$P_{g,ij}$	Probability that grid g is associated with link (i, j) .
Pop_{ij}	Population served by link (i, j) .
S_{ij}	Average passenger distance to link (i, j) .
L_{ij}	Distance between the midpoint of link (i, j) and the target station.
PD_{ij}	Potential demand of link (i, j) .
ω	Ordered array of nonzero potential demand values.
$n_{pd_{ij,n}}$	Rank order of a nonzero potential demand value in ω .
PI_{ij}	Priority index of link (i, j) .
t_{ij}	Travel time associated with link (i, j) .
d_{ij}	Road-based travel distance of link (i, j) .
T	Maximum allowable trip time for one feeder bus route.
$\Delta_{i,j}$	Area of grids assigned to link (i, j) .

The population density of the inhabited study zone is computed as:

$$\lambda = \frac{Pop_z}{\Delta_z}$$

The population assigned to each grid cell is then estimated by:

$$Pop_g = \lambda \Delta_g, \forall g \in Z$$

The distance between each grid centroid and each feasible link is denoted by $s_{g,ij}$. Following the grid-to-link assignment logic, grids located closer to a link have a higher probability of being associated with that link. The assignment probability is expressed as:

$$P_{g,ij} = \frac{e^{-s_{g,ij}}}{\sum_{(m,n) \in A} e^{-s_{g,mn}}}, \forall g \in Z, \quad (i,j) \in A$$

The population assigned to link (i,j) is calculated by aggregating the population values of all grids assigned to that link:

$$Pop_{ij} = \sum_{g \in Z} Pop_g * F_{g,ij}, \forall (i,j) \in A$$

The average passenger access distance to link (i,j) is calculated as the population-weighted average distance between assigned grid centroids and the corresponding link:

$$S_{ij} = \frac{\sum_{g \in Z} Pop_g * F_{g,ij} * s_{g,ij}}{\sum_{g \in Z} Pop_g * F_{g,ij}}, \forall (i,j) \in A$$

If no grid is assigned to link (i,j) , then:

$$Pop_{ij} = 0, S_{ij} = 0$$

The potential demand of each candidate feeder link is defined as:

$$PD_{i,j} = \begin{cases} \frac{Pop_{ij}}{S_{ij}} \cdot L_{ij}, & Pop_{ij} > 0, S_{ij} > 0 \\ 0, & otherwise \end{cases}, \quad \forall (i,j) \in A$$

This formulation indicates that a link becomes more attractive when it is associated with a larger assigned population and is located farther from the target metro station, while its attractiveness decreases when the average access distance from passengers to the link becomes larger. This structure is consistent with the potential-demand concept used in feeder bus route design, where link attractiveness depends on population demand, passenger access distance, and distance to the metro station.

The nonzero potential demand values are arranged in ascending order as:

$$\omega = [PD_1, PD_2, \dots, PD_n], PD_n > PD_{n-1}, PD_n \neq 0$$

The priority index of each link is then obtained from the rank order of its potential demand:

$$PI_{ij} = \begin{cases} 0, & PD_{ij} = 0 \\ rank(PD_{ij}), & PD_{ij} > 0 \end{cases} \forall (i,j) \in A$$

For U-turn links, no demand is assigned because they are only virtual topological connectors:

$$Pop_{ij} = 0, PD_{ij} = 0, PI_{ij} = 0, \forall (i,j) \in A^0$$

The total assigned grid area of each link is calculated as:

$$\Delta_{i,j} = \sum_{g \in Z} \Delta_g F_{g,i,j}, \forall (i,j) \in A$$

4.2.3 Decision Variables

The model uses the following binary decision variables.

Table 7. Table of Decision Variables

Symbol	Type	Description
Y_{ij}	Binary	Equals 1 if link (i, j) is selected as part of the feeder route, and 0 otherwise.
$F_{g,ij}$	Binary	Equals 1 if grid g is assigned to link (i, j) , and 0 otherwise.

The variable Y_{ij} is the principal route-selection variable. The variable $F_{g,ij}$ is supporting binary variables used to compute, respectively, the population captured by each link and the served area of the final route.

4.2.4 Objective Functions

The objective of the model is to maximize the total priority index collected by the selected circular feeder bus route:

$$\max Z = \sum_{(i,j) \in A} P I_{ij} Y_{ij}$$

The use of PI_{ij} rather than raw PD_{ij} transforms continuous potential demand values into integer-based priority values. This ranking structure improves route discrimination and reduces numerical complexity during the search process. The objective therefore prioritizes links that are more relevant to demand-weighted first-mile access, while the route remains constrained by operational travel time and network feasibility.

Although the objective function is based on total priority index, the final solution is also evaluated by total potential demand, total served population, total served area, total travel time, and route distance. These indicators are treated as performance measures rather than primary optimization objectives. The served area of a selected route can be computed as:

$$SAp = \sum_{(i,j) \in A} \Delta_{i,j} Y_{ij}$$

Or in the percentage form:

$$SAp\% = \frac{\sum_{(i,j) \in A} \Delta_{i,j} Y_{ij}}{\Delta_Z} * 100$$

4.2.5 Constraints

1. The maximum route travel time constraint, where T is a constant to be adjusted based on different scenarios with $T = 18, 30, 45, \dots$ minutes.

$$t_{ij} Y_{ij} \leq T$$

2. The selected route must both start and return to the same target station $s = 31$:

$$\sum_{j:(s,j) \in A} Y_{sj} = \sum_{i:(i,s) \in A} Y_{is} = 1$$

3. For every intermediate node, the number of selected incoming links must equal the number of selected outgoing links:

$$\sum_{j:(i,j) \in A} Y_{ij} = \sum_{h:(h,i) \in A} Y_{hi}, \quad \forall i \in N, i \neq s$$

4. To prevent route redundancy and repeated internal station visits, each intermediate node can be entered and exited at most once:

$$\sum_{j:(i,j) \in A} Y_{ij} \leq 1, \quad \forall i \in N, i \neq s$$

$$\sum_{h:(h,i) \in A} Y_{hi} \leq 1, \quad \forall i \in N, i \neq s$$

5. The model also prevents simultaneous selection of a Forward source link and its Backward counterpart. Since every Backward link has a Forward source link, but not every Forward link has a Backward counterpart, the mutual-exclusion rule is formulated from the set of Forward source links that are referenced by Backward links:

$$Y_k + \sum_{(i,j) \in A_k^2} Y_{ij} \leq 1, \forall k \in K$$

6. A grid is assigned to a link(i,j) on the condition of the shortest distance from its midpoint to the nearest point on the vector line of the link(i,j) within the catchment radius purely – assumed as the preferable maximum walking distance = 950 metre.

$$s_{g,ij} \leq 950 \text{ (m)}$$

4.3 Optimization Process Flowchart

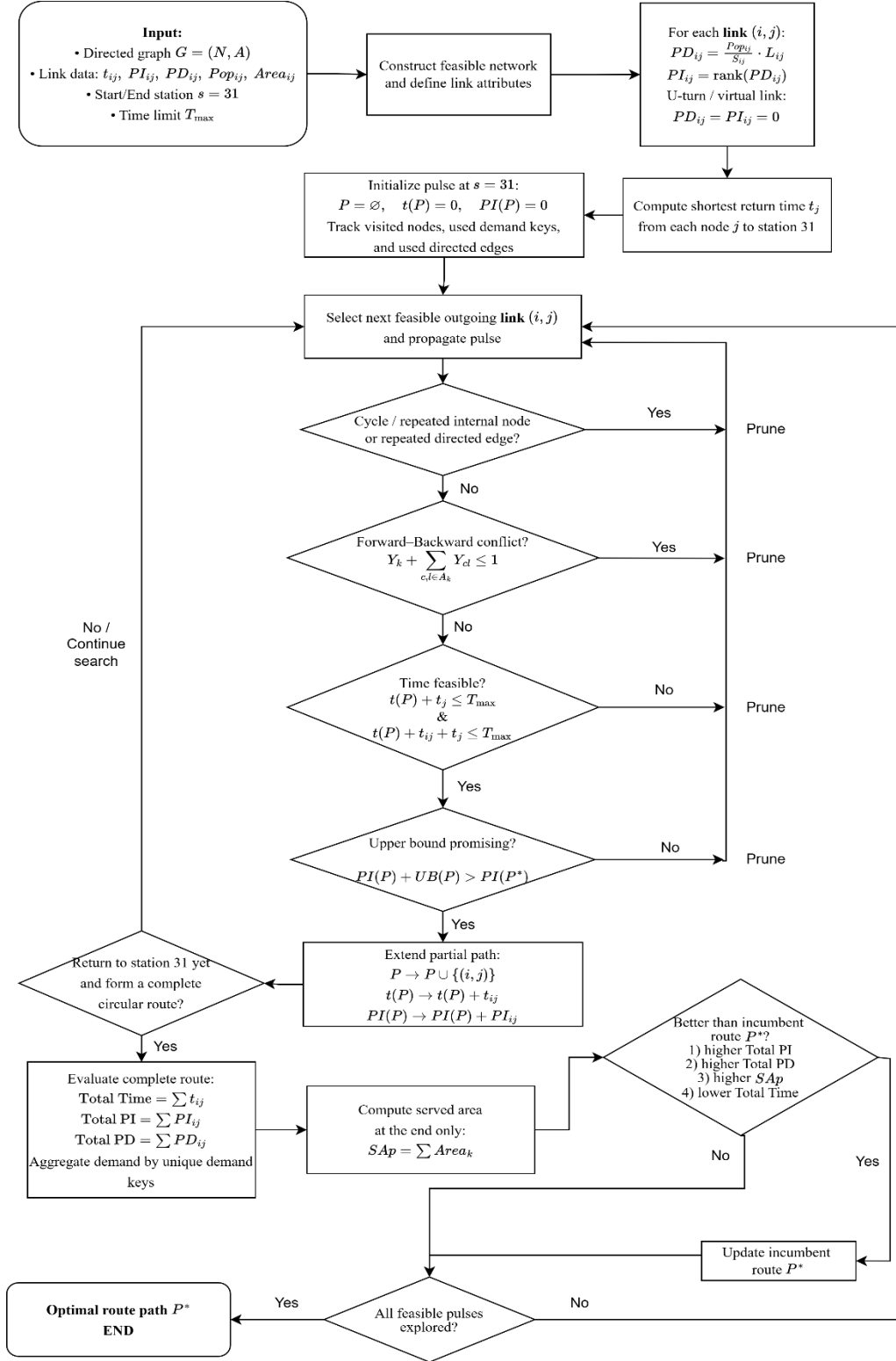


Figure 13. Model Optimization Flowchart

The proposed optimization process formulates the feeder bus network as a directed graph $G = (N, A)$, where each feasible arc $(i, j) \in A$ is associated with travel time t_{ij} , priority index PI_{ij} , potential demand PD_{ij} , population Pop_{ij} , and assigned service area $Area_{ij}$. Starting from the metro station $s = 31$, the algorithm propagates a partial route P through feasible outgoing arcs while updating $t(P)$, $PI(P)$, $PD(P)$, and the visited-node set. Each pulse is pruned if it violates elementary-path conditions, repeats a directed edge, creates a forward–backward link conflict, duplicates an already covered demand key, or fails the time feasibility condition $t(P) + t_{ij} + r_j \leq T_{\max}$, where r_j is the shortest return time from node j to station 31. Complete circular routes are evaluated only when the pulse returns to s , after which total travel time, total PI , total PD , and SAP are computed and compared with the incumbent best route P^* . This structure follows the pulse-search logic for constrained path problems, in which partial paths are recursively extended and infeasible or nonpromising paths are eliminated before full enumeration becomes computationally excessive (Lozano, 2012).

In contrast to the original pulse framework, the present implementation removes Q dominance-labelling memory and relies instead on explicit feasibility, structural, and upper-bound pruning. This adjustment is motivated by the trade-off discussed by Lozano and Medaglia (2012), where storing all labels (c, t) for paths reaching node v_k would require substantial memory in large networks, while limiting the memory to Q labels improves scalability but still allows some dominated paths to pass through some nodes v_k . In the VNU feeder bus case, this trade-off was considered inefficient because the added memory structure increases implementation and storage burden without guaranteeing that all dominated pulses are eliminated. Therefore, the thesis algorithm prioritizes low-memory recursive exploration with strict route-validity rules, including

no cycle, no repeated internal node, no repeated directed edge, no forward–backward conflict, and no duplicated demand coverage. This design is consistent with the feeder-route objective of maximizing demand-oriented service coverage under a maximum travel-time constraint, while maintaining computational practicality for the 271-link real network.

Chapter 5: RESULT ANALYSIS

5.1 Sensitivity Analysis

Grid centroid coordinates constitute a core feed-in dataset for the proposed mathematical model because they support the calculation of spatial demand, passenger–link distance, assigned service area, and subsequent objective-function components. To obtain these coordinates consistently, the Vietnam National University polygon was processed in Google Earth Engine and overlaid with equal-sized grid cells, from which the centroid of each grid was extracted. This grid-based representation follows feeder bus demand modeling studies in which the study area is divided into traffic-zone grids and passenger demand is allocated to candidate links based on centroid-to-link distance; Zhu et al. (2017) treated each grid as a traffic zone and used the midpoint-to-link distance for passenger assignment, while Hemdan et al. (2026) similarly divided the inhabited area into grids to estimate grid population and link-level potential demand. Although a $100\text{ m} \times 100\text{ m}$ grid size is commonly adopted in previous feeder bus optimization models by both Zhu et al. (2017) and Hemdan et al. (2026), this resolution did not provide a sufficiently compatible output for the mathematical model and dataset structure of the present VNU case. Therefore, alternative grid resolutions, by scaling down by half till $50\text{ m} \times 50\text{ m}$ and $25\text{ m} \times 25\text{ m}$ at least, were configured and tested to evaluate the sensitivity of the Pulse model to spatial discretization and to identify the grid size that best supports the subsequent optimization analysis in the table below:

Table 8. Grid size Sensivity Analysis Table

Indicators	Pulse Algorithm Models		
Grid Size	25m x 25m	50m x 50m	100m x 100m
Model Runtime	5.02 mins	3.73 mins	3.76 mins
<i>Google Earth Engine</i> runtime	2.6 mins	48s	19s
Percent of grid-assigned links (%)	100%	97%	88%
Total PI	2851	2836	2430
SAP%	24.45132	21.6213	22.1842
Total travelling time (minutes)	44.944	43.953844925	44.75820
Total Distance (km)	16.348	15.626	16.213

The grid-size sensitivity test was conducted using the Pulse Algorithm under a maximum travel-time constraint of **T=45 minutes**. The results show that the 50 m × 50 m grid resolution provides the most balanced computational and optimization performance. Compared with the 25 m × 25 m grid, the 50 m × 50 m configuration reduces the model runtime from 5.02 minutes to 3.73 minutes and substantially decreases the Google Earth Engine processing time from 2.6 minutes to 48 seconds, while maintaining a highly comparable total priority index (2836 compared with 2851). Its total travelling time and total distance also remain within a similar range, indicating that the route structure is not significantly weakened by using a moderately coarser grid resolution. In contrast, the 100 m × 100 m grid produces the lowest total PI (2430) and the lowest percentage of grid-assigned links (88%), suggesting that this resolution is too coarse to adequately represent the

spatial distribution of demand in the VNU study area.

However, the 25 m × 25 m grid remains important from a methodological perspective. It achieves 100% grid-assigned links, meaning that all feasible links receive spatial demand information from the grid-assignment process. This complete assignment can provide richer routing alternatives, especially when the dataset or study area is expanded in future model applications. The main trade-off is its substantially longer preprocessing time in Google Earth Engine and higher model runtime. Therefore, while the 50 m × 50 m grid is selected as the best-performing grid size for the Pulse Algorithm in the current experiment due to its strong balance between solution quality and computational efficiency, the 25 m × 25 m grid is retained for universal testing across the Pulse Algorithm, Genetic Algorithm, and Ant Colony Optimization models. This allows the cross-model comparison to be conducted under the most spatially complete demand representation, even at the cost of longer computation.

5.2 Scenario Analysis

There are 2 scenarios of value for T = the maximum route travel time based on existing real-life feeder bus routes in the VNU area, Route 166 & 170 both starting and finishing at target station = 31, which is the National University Metro Station. The aim is to create two different maximum route time travel (T) to benchmark the performance of 3 models: Pulse Algorithm, Genetic Algorithm and Ant Colony Optimization all configured with grid size = 25m x 25m as discussed above in sensitivity analysis section, compared to the existing route performance based on the common performance indicators.

Table 9. Scenario Analysis Table

Maximum Route Travel Time, $T = 18$ mins				
Indicators	Pulse Algorithm	Genetic Algorithm	Ant Colony Optimization	Route 166
Circular route Start/end at 1 station?	Yes	Yes	Yes	Yes
Model Runtime	0.0151 sec	2.2145 sec	5.0785 sec	N/A
Hyperparameter Optimization time	N/A	23.26 minutes	25.05 minutes	N/A
Total PI	749	749	749	658
No. of stations passed	14	14	14	15
SAP%	9.7351	9.7351	9.7351	5.9369
Total travelling time (minutes)	17.56	17.56	17.56	20.47
Total Distance (km)	7.261	7.261	7.261	6.888

Maximum Route Travel Time, T = 45 mins

Indicators	Pulse Algorithm	Genetic Algorithm	Ant Colony Optimization	Route 170
Circular route Start/end at 1 station?	Yes	Yes	Yes	No
Model Runtime	5.02 mins	36.25 sec	65.87 sec	N/A
Hyperparameter Optimization time	N/A	1.823 hrs	1.435 hrs	N/A
Total PI	2851	2815	2815	1,574
No. of stations passed	34	33	33	21
SAP%	24.4513	22.8784	22.8784	20.95%
Total travelling time (minutes)	44.944	44.488	44.488	42.12
Total Distance (km)	16.348	16.198	16.198	15.59

The two comparative tables evaluate the performance of the proposed feeder bus route optimization models under two maximum route travel time scenarios, namely T=18 minutes and T=45 minutes. These two values were selected based on the operating characteristics of existing feeder bus routes in the VNU area, where Route 166 represents the shorter benchmark route and Route 170 represents the longer benchmark route. All three optimization models, including the Pulse Algorithm, Genetic Algorithm (GA), and Ant Colony Optimization (ACO), were implemented using the selected grid cell size of 25 m × 25 m from the sensitivity analysis. The main purpose of this comparison is to examine whether the optimized routes can improve route performance over existing real-life routes while satisfying the same maximum travel time condition. The methodological basis of this comparison is consistent with previous feeder bus

network design studies. Zhu et al. (2017) developed a circular feeder bus route model that starts and ends at an urban rail transit station and maximizes potential demand under a maximum travel time constraint. Their model was solved using a genetic algorithm, which supports the use of GA as one of the candidate metaheuristic methods in this study. Similarly, Wei et al. (2022) proposed an improved ant colony algorithm for urban bus network optimization based on existing bus routes, which supports the inclusion of ACO as another population-based search approach for route network optimization. The Pulse Algorithm is also theoretically relevant because the feeder bus route design problem can be formulated as a constrained path problem, where the objective is to maximize the priority index while satisfying the travel time limit. Hemdan et al. (2026) applied the pulse-based approach to feeder bus route design and emphasized the use of a priority index to improve route selection under a fixed travel time frame. The pulse method itself is based on propagating partial paths through a network and pruning infeasible or dominated paths using dominance, bounds, and resource feasibility rules.

For the Genetic Algorithm and Ant Colony Optimization models, hyperparameter optimization was required before final model execution. This is because both GA and ACO are metaheuristic algorithms whose solution quality depends strongly on parameters such as population size, mutation rate, crossover rate, pheromone influence, heuristic influence, evaporation rate, and number of iterations. Therefore, Optuna library (Akiba et al., 2019) was implemented with 100 trials as the fixed tuning budget for each algorithm and each travel time scenario. Optuna is suitable for this purpose because it provides an automatic hyperparameter optimization framework with flexible search-space construction and efficient search/pruning mechanisms especially for NP – hard constrained problems. After the tuning process, different optimal parameter sets were selected

for the 18-minute and 45-minute scenarios as shown in these tables below, reflecting the fact that algorithm behavior changes when the feasible search space becomes larger.

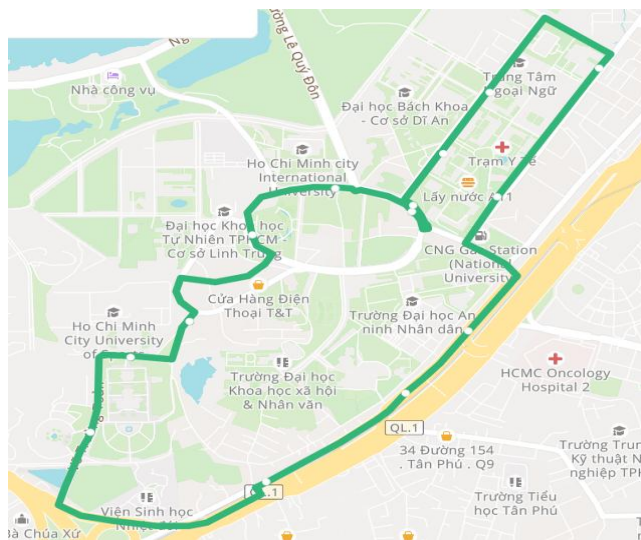
Ant Colony Optimization	
18-minute T max travel time	45-minute T max travel time
'n_ants': 160	'n_ants': 120
'iterations': 180	'iterations': 120
'alpha': 0.571440722658989	'alpha': 0.520903583142434
'beta': 1.72829504724664	'beta': 1.07737743917036
'pi_gamma': 1.06550475252494	'pi_gamma': 1.65306234334256
'time_beta': 1.22452545130217	'time_beta': 1.44814513284064
'closure_gamma': 0.0609214916103877	'closure_gamma': 0.0944079291717722
'rho': 0.319311283805087	'rho': 0.107668965995643
'Q': 0.0072116047926307	'Q': 0.00605324350869886
'tau0': 1.18794922928538	'tau0': 0.748028955063651
'tau_min': 0.00001	'tau_min': 0.00001
'tau_max': 30.7319518134835	'tau_max': 23.625323296202
'softmax_temperature': 1.35358290350908	'softmax_temperature': 0.725018156132352
'close_prob': 0.0383863176548441	'close_prob': 0.0514935417626857
'n_elite_ants': 2	'n_elite_ants': 7
'max_steps': 34	'max_steps': 35

Genetic Algorithm	
18-minute T max travel time	45-minute T max travel time
'population_size': 120	'population_size': 160
'generations': 120	'generations': 180
'elite_size': 13	'elite_size': 19
'tournament_size': 2	'tournament_size': 2
'crossover_rate': 0.487228079161762	'crossover_rate': 0.432591268
'mutation_rate': 0.801437179149482	'mutation_rate': 0.610768446
'construct_alpha': 1.59552780081384	'construct_alpha': 0.831968822
'construct_beta': 0.76735979883162	'construct_beta': 0.804562617
'close_prob': 0.0791510633903858	'close_prob': 0.027575611
'temperature': 0.766813754635238	'temperature': 1.252181045
'max_steps': 35	'max_steps': 35
'population_size': 120	'population_size': 160
'generations': 120	'generations': 180
'elite_size': 13	'elite_size': 19
'tournament_size': 2	'tournament_size': 2
'crossover_rate': 0.487228079161762	'crossover_rate': 0.432591268

Table 10. Table of Optimized Parameters

Under the $T=18$ minutes scenario, the three optimization models produced exactly the same route performance. The Pulse Algorithm, Genetic Algorithm, and Ant Colony Optimization all obtained a total PI of 749, passed 14 stations, achieved an SAp of 9.7351%, required 17.56 minutes of total travelling time, and covered 7.261 km. This complete convergence across all performance indicators suggests that the 18-minute constraint creates a highly restricted feasible solution space. When the allowable travel time is tight, only a limited number of circular routes can satisfy the requirement of starting and ending at station 31. Therefore, all three models converged to the same best feasible route. The second map presents the direct visualization output of the three optimization models on OpenStreetMap. In this output, the route is represented by straight-line connections between selected stations, meaning that the correct guided road path is not fully determined. Therefore, this visualization is useful for identifying the selected station sequence, but it does not accurately represent the real bus movement along the road network. To address this limitation, the third map was manually reconstructed in Google My Maps using the same 18-minute optimized route output. This guided-route visualization follows the actual road geometry more closely and therefore provides a more realistic representation of the route path. The first map shows the existing Route 166, which was used as the benchmark route in the current system investigation section. Compared with the existing Route 166 benchmark, the optimized route under $T=18$ minutes achieved stronger demand-oriented performance. The optimized route obtained a total PI of 749, while Route 166 obtained a total PI of 658. This represents an improvement of 91 PI units, equivalent to approximately 13.83%. The optimized route also achieved an SAp value of 9.7351%, compared with 5.9369% for Route 166, corresponding to an increase of 3.7982 percentage points. In relative terms, the optimized route improved the served area indicator by approximately 63.98%. Although the optimized route passed one fewer station than Route 166,

with 14 stations compared with 15 stations, it produced a stronger demand-weighted result. This indicates that the optimization model did not merely maximize the number of stops, but instead selected links with stronger spatial demand contribution. The travel time and distance comparison provides additional insight into the operational trade-off. The optimized route required 17.56 minutes, which satisfies the imposed maximum travel-time constraint. Its distance was 7.261 km, slightly longer than the modelled distance of Route 166 at 6.888 km. The additional distance of 0.373 km, or approximately 5.42%, is relatively small, yet it contributed to a substantially higher PI and SAp value. However, Route 166 showed a modelled travel time of 20.47 minutes in the recalculated performance table, which is higher than the nominal investigated value of 18 minutes. This difference should be interpreted carefully. The official investigated travel time reflects the scheduled or reported route duration, while the table reports route performance recalculated using the study's link-based distance and travel-time indicators. Therefore, the optimized route and Route 166 should be compared primarily under the same modelling framework rather than only against the official schedule value.



Existing Route 166

Route path (in station_id):

31 → 27 → 23 → 20 → 19 → 4 → 5 → 6
 → 17 → 11 → 14 → 15 → 10 → 33 →
 32 → 31

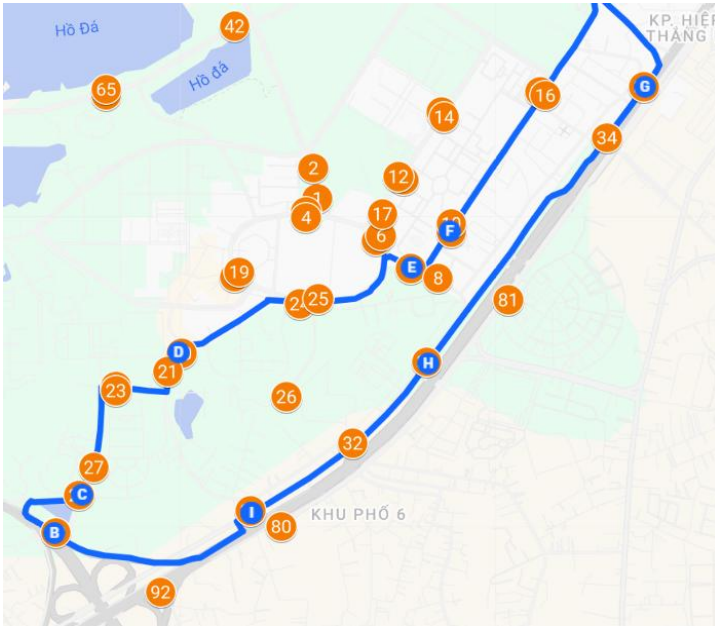
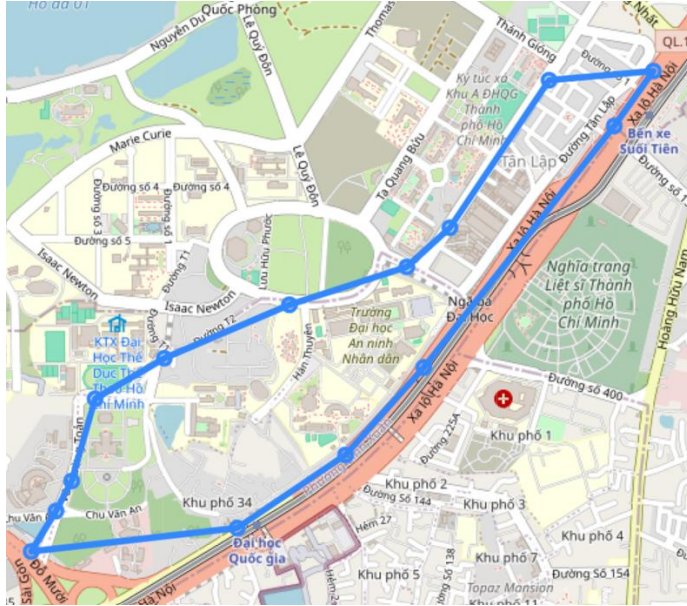


Figure 14. Comparison of 18-min output maps with existing route 166

Pulse Algorithm, Genetic Algorithm & Ant Colony Optimization

Route path (in station_id):

31 → 30 → 28 → 27 → 23 → 20 → 24
→ 7 → 9 → 16 → 35 → 34 → 33 →
32 → 31

The main discrepancy under the 18-minute scenario appears in computational runtime. The Pulse Algorithm solved the problem in only 0.0151 seconds, while the Genetic Algorithm required 2.2145 seconds and ACO required 5.0785 seconds. This means that GA was approximately 146.66 times slower than the Pulse Algorithm, while ACO was approximately 336.32 times slower. ACO was also around 2.29 times slower than GA in direct model runtime. This result is consistent with

the structure of the Pulse Algorithm under tight constraints. Since infeasible partial routes can be pruned early when the time budget is restrictive, the Pulse Algorithm can eliminate many candidate paths quickly. In contrast, GA and ACO still need to evaluate multiple candidate solutions through population-based or pheromone-based iterations, even when the feasible region is small. The hyperparameter optimization time further highlights the computational cost of metaheuristic models. Under the 18-minute scenario, GA required 23.26 minutes for hyperparameter tuning, while ACO required 25.05 minutes. Although their final model runtimes were only a few seconds, the full experimental workflow was much more time-consuming once tuning was included. ACO required 1.79 minutes more tuning time than GA, equivalent to approximately 7.70% longer. Therefore, for the short-route scenario, the Pulse Algorithm was clearly superior in computational efficiency because it required no hyperparameter optimization and still obtained the same optimal performance indicators as GA and ACO.

Under the T=45 minutes scenario, the performance difference among the optimization models became more visible. The Pulse Algorithm achieved the highest total PI of 2851, while GA and ACO both obtained a total PI of 2815. The difference of 36 PI units is relatively small, equal to approximately 1.28% improvement over GA and ACO. However, it indicates that the Pulse Algorithm was able to identify a slightly better demand-weighted route when the travel time constraint became more relaxed. The Pulse route passed 34 stations, while GA and ACO passed 33 stations. The Pulse Algorithm also achieved the highest SAp value at 24.4513%, compared with 22.8784% for GA and ACO. This represents an increase of 1.5729 percentage points, or approximately 6.88% relative improvement in served area over the two metaheuristic models. The total travel time values show that all optimized models remained feasible under the 45-minute constraint. The Pulse Algorithm produced a total travel time of 44.944 minutes, while GA and

ACO produced 44.488 minutes. The Pulse route used 0.456 minutes more than GA and ACO, but this additional time was converted into higher PI, one additional station, and higher SAP coverage. This result suggests a trade-off between route compactness and demand capture. GA and ACO produced a slightly shorter and faster route, while the Pulse Algorithm used more of the available time budget to extract additional demand value from the network.

PULSE ALGORITHM

Route Path (in station_id): 31 → 30 → 29 → 54 → 56 → 55 → 52 → 53 → 57 → 66 → 67 → 68 → 69 → 70 → 71 → 76 → 77 → 62 → 59 → 58 → 61 → 47 → 64 → 1 → 3 → 18 → 24 → 7 → 9 → 16 → 35 → 34 → 33 → 32 → 31

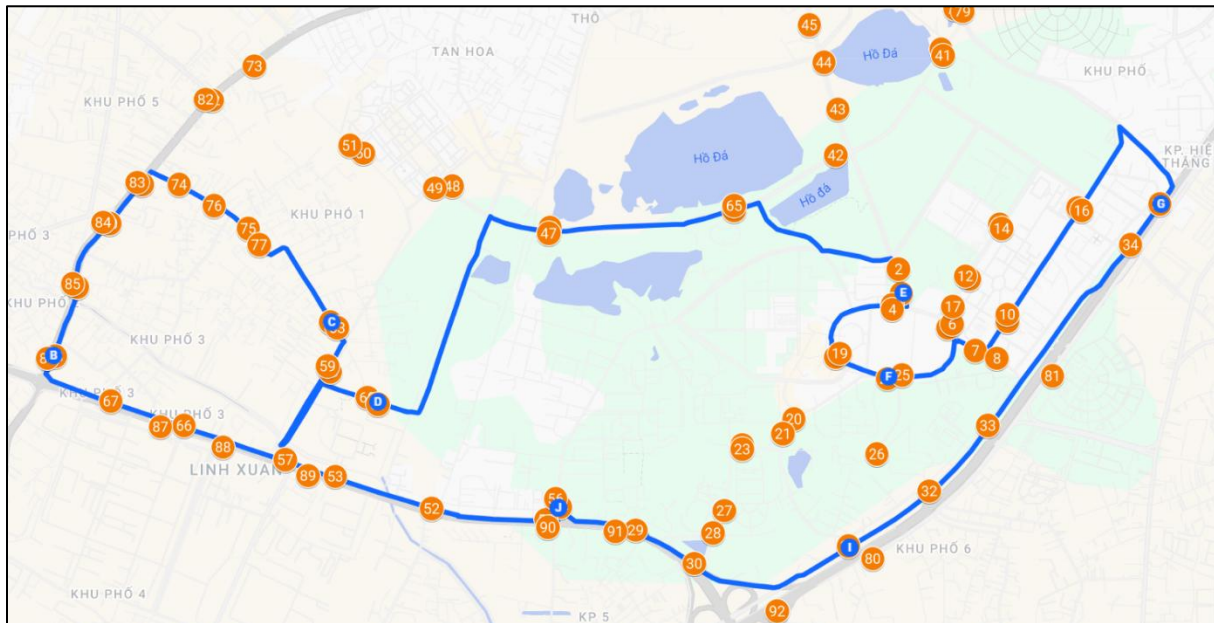


Figure 15. Route map of PA 45-min model

ACO & GA

Route Path (in station_id): 31 → 30 → 29 → 54 → 55 → 52 → 53 → 57 → 66 → 67 → 68 → 69 → 70 → 71 → 76 → 77 → 62 → 59 → 58 → 61 → 47 → 64 → 1 → 3 → 18 → 24 → 7 → 9 → 16 → 35 → 34 → 33 → 32 → 31

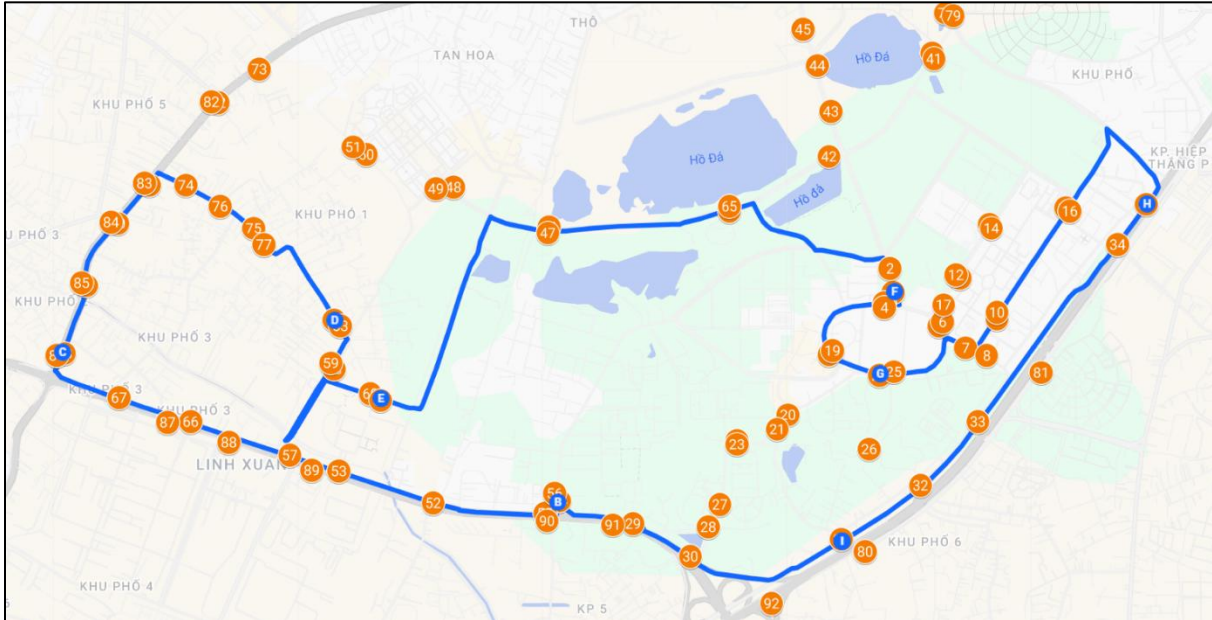


Figure 16. Route map of GA/ACO 45-min model

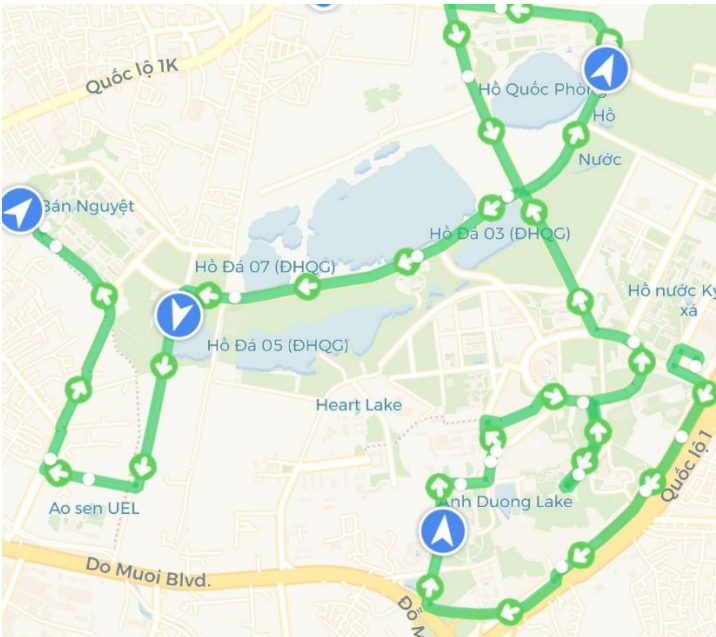


Figure 17. Route map of existign route 170

Existing Route 170

Route Path (in station_id): 10 → 33 → 32 → 31 → 27 → 23 → 20 → 24 → 26 → 6 → 2 → 41 → 79 → 37 → 45 → 42 → 65 → 46 → 60 → 58 → 50

The minor difference is barely visible in the route maps above. The spatial alignments of the Pulse route and the GA/ACO route are nearly identical across most of the network; however, the Pulse Algorithm selected the additional segment 54→56→55 including a U-turn from 54→56 while adding in additional PI from 56→55, whereas GA and ACO selected the more direct segment 54→55. The improvement suggests that the additional segment 54→56→55 did not merely increase the route length, but also contributed meaningful demand coverage. In this case, the Pulse Algorithm used a slightly less compact route structure to capture a higher-priority link and expand the served area. Therefore, the only structural difference between the optimized solutions is the inclusion of station 56 on the Pulse route. This explains why the Pulse Algorithm passed 34 stations, compared with 33 stations for GA and ACO, which is illustrated as below:



Figure 18. Pulse & ACO/ GA route difference

Compared with the existing Route 170, all optimized models substantially improved the demand-oriented indicators. Unlike the optimized routes, Route 170 is not represented as a circular route

starting and ending at station 31 in the modeled network. This structural difference partly explains its weaker performance under the same evaluation indicators. Route 170 achieved a total PI of 731, whereas the Pulse Algorithm achieved 2851. This is an improvement of 2120 PI units, or approximately 290.01%. GA and ACO also achieved 2815 PI, which is 2084 PI units higher than Route 170. In terms of SAp, Route 170 obtained 20.95%, while the Pulse Algorithm achieved 24.4513%. This represents an improvement of 3.5 percentage points, or approximately 34.33%. GA and ACO also improved SAp by 8.2948 percentage points over Route 170. These results indicate that the optimized routes are much more effective in selecting high-priority and spatially valuable links than the existing route structure. The route map also supports this interpretation, as the optimized routes form a wider circular coverage around station 31 and use the available route budget to serve more demand-relevant sections of the VNU network. The Route 170 comparison also shows an important difference between real-life route characteristics and model-based evaluation. The investigated real-life data indicate that Route 170 contains 27 stations, has a distance of 15.45 km, and operates with a route time of 45 minutes. However, when recalculated using the problem's modeled indicators, Route 170 in the table passes 21 stations, covers 15.59 km, and has a total travelling time of 42.12 minutes. Therefore, Route 170 should be interpreted as a benchmark reference under the same model evaluation framework, rather than a complete reproduction of the full official route record. The optimized Pulse route covered 16.348 km, which is 4.728 km longer than the modeled Route 170 distance, but this additional route coverage produced a much higher PI and SAp value. This indicates that the optimized route uses the available 45-minute budget more fully than the existing benchmark route.

The runtime pattern in the 45-minute scenario differs sharply from the 18-minute case. The Pulse Algorithm required 5.02 minutes, while GA required 36.25 seconds and ACO required 65.87 seconds. In this case, the Pulse Algorithm was approximately 8.31 times slower than GA and 4.57 times slower than ACO. This result is expected because a longer maximum travel time greatly expands the feasible search space. Under a relaxed constraint, fewer partial paths can be eliminated early by time infeasibility, so the Pulse Algorithm must explore a much larger number of feasible route combinations. GA and ACO, by contrast, remain bounded by their configured population size, number of iterations, and selected stopping criteria, which explains their shorter final runtime. However, once hyperparameter tuning is considered, GA required 1.823 hours and ACO required 1.435 hours. Therefore, the direct runtime advantage of GA and ACO must be balanced against their much higher pre-optimization cost.

Chapter 6: CONCLUSIONS

6.1 Results Discussion & Implications

This thesis established a data-driven framework for the Feeder Bus Network Design Problem (FBNDP). By shifting from traditional coverage-centric designs to a demand-weighted paradigm, this research optimized feeder loops using a composite Priority Index (\$PI\$) to bridge the first-mile accessibility gap at the National University Metro Station. The empirical results reveal a stark performance divergence between exact mathematical approaches and metaheuristic approximations under varying time thresholds.

The Pulse Algorithm (PA) demonstrated definitive computational superiority. It mapped the absolute global optimum in a mere 0.0151 seconds, completely bypassing the iterations required by the Genetic Algorithm (GA) (2.2145 seconds) and Ant Colony Optimization (ACO) (5.0785 seconds). As the search space expanded, an aggressive computational trade-off emerged. The Pulse Algorithm achieved the absolute global optimum ($PI = 2851$; $SAP\% = 24.4513\%$), capturing critical additional demand zones such as station 56. However, this exactness triggered a combinatorial explosion, raising its execution runtime to 5.02 minutes. The stark contrast in performance curves between the exact Pulse Algorithm and the approximate metaheuristics yields critical implications for scalability and deployment in transit-oriented development (TOD). If the underlying input dataset is scaled up from the isolated VNU-HCM urban area to a comprehensive, macro-level metropolitan network encompassing an entire major city, the computational behaviour of these algorithms will diverge dramatically. The Pulse Algorithm, despite its sophisticated pruning mechanisms, is fundamentally governed by the network's topological complexity and constraint thresholds. On a massive, city-wide directed graph containing thousands of transit nodes and tens of thousands of directed arcs, the execution runtime of the Pulse Algorithm is projected

to escalate rapidly. It could scale in an extreme logarithmic or exponential fashion if the time constraint T remains loose, potentially rendering real-time routing adjustments computationally intractable without massive parallel computing infrastructure. Conversely, metaheuristic models exhibit superior structural scalability regarding runtime. For both GA and ACO, scaling the dataset up to a metropolitan scale will inevitably increase the pre-operational hyperparameter optimization overhead, as engineers must invest more time into calibrating mutation rates, population sizes, and pheromone dynamics to avoid premature convergence. However, once those hyperparameters are locked in, the actual model execution runtime for GA and ACO will scale gracefully within manageable polynomial limits. This makes metaheuristics highly attractive for massive urban networks where a minor sacrifice in absolute global optimality (settling for a 95–98% near-optimal solution) is a highly acceptable trade-off for near-instantaneous route generation and flexible operational scheduling. When evaluated against the active municipal transit framework of existing routes 166 and 170, the optimized loops yielded a transformative leap in transit efficacy. The optimized 18-minute loop outperformed Route 166 by 13.83% in total PI and expanded the served area coverage by 63.98% in relative terms. The optimized 45-minute loop outpaced the circuitous baseline of Route 170, demonstrating how strategic, non-overlapping circular loops can intercept personal motorized trips and significantly mitigate parking yard saturation at the metro terminus. All the discussed implications for each model for this thesis Feeder Network Design Problem is summarized as in this table below.

Table 11. Discussion Summary

Benchmark	Pulse Algorithm (PA)	Genetic Algorithm (GA)	Ant Colony Optimization (ACO)
Optimality Guarantee	Mathematically guaranteed absolute global optimum.	Stochastic near-optimal approximation.	Stochastic near-optimal approximation.
Short-Horizon Efficiency (Tmax is small)	Instantaneous convergence; completely outpaces heuristic methods due to strict bounding constraints.	Rapid convergence; incurs minor evolutionary generation and chromosomal loop overhead.	Moderate convergence; delayed by structural initialization and agent population management.
Extended-Horizon Efficiency (Tmax is large)	Highly delayed execution due to massive network expansion and recursive path processing.	Exceptionally rapid execution; highly resilient to network path and graph expansion.	Moderate execution velocity; slower than alternative metaheuristics but vastly faster than exact methods.
Hyperparameter Calibration Overhead	Completely absent due to its deterministic structural formulation.	Low to moderate; exhibits rapid parameter landscape stabilization and stable convergence paths.	Highly intensive; characterized by extreme sensitivity to pheromone deposition and evaporation coefficients.
Macro-Urban Scalability Potential	Restricted; highly vulnerable to severe exponential computational degradation on dense city networks.	High; final execution runtimes scale gracefully within highly predictable polynomial limits.	Moderate; manageable scale transition but bottlenecked by structural node selection step costs.
Vulnerability	Extreme runtime inflation under loose travel-time constraints and high vertex-to-arc ratios.	Susceptibility to premature convergence within localized sub-optimal traps if diversity falls.	Elevated algorithmic cost per operational loop during step-by-step probabilistic arc evaluation.

6.2 Recommendations for Future Research

While this study establishes a robust, data-driven optimization paradigm for first-mile transit design, certain boundaries were maintained to preserve computational tractability. To advance the scientific maturity and real-world applicability of this framework, two primary vectors for future research are highly recommended.

Stochastic Multi-Objective Optimization Model Under Demand and Travel-Time Uncertainties for both First – Last Mile Travels

The deterministic assumptions embedded within the current model—specifically assigning fixed link travel times (t_{ij}) from historical Thursday profiles and static population proxies—ignore the volatile operational realities of urban transport networks. Future iterations should reformulate the Feeder Bus Network Design Problem into a stochastic, multi-objective mathematical framework that explicitly acknowledges systemic uncertainties. The objective functions should be expanded to simultaneously minimize total operator costs (fleet size, fuel consumption, and driver shifts) and passenger generalized travel costs (in-vehicle travel time, transfer penalties, and schedule delay costs), while maximizing the demand captured. Crucially, the travel times across links must be treated as stochastic variables governed by continuous probability distributions (e.g., log-normal or Gamma distributions) to simulate the disruptive impacts of peak-hour traffic congestion, adverse weather anomalies, and traffic incidents. Simultaneously, commuter demand rates should be modeled as a non-homogeneous Poisson process to capture the dynamic variability of passenger arrivals at feeder nodes. To solve this highly complex formulation, researchers should investigate hybrid exact-metaheuristic algorithms, where the Pulse Algorithm's pruning capability is integrated into the multi-objective Pareto-archiving frameworks of algorithms like NSGA-II or NSGA-III, thereby rendering robust solutions that remain stable under varying urban conditions.

This stochastic model should be expanded into a unified bidirectional framework that optimizes first-mile collection and last-mile distribution simultaneously. Future formulations must isolate drop-off demand from pick-up demand to capture asymmetric commuter workflows across peak periods. To mirror real-world volatility, the model should treat passenger arrival rates at metro platforms as a non-homogeneous Poisson process, mathematically synchronizing feeder bus arrival windows with scheduled rail headways to minimize intermodal transfer friction.

Multi-Modal Microscopic Transport Simulation

Future research should also transition from isolated bus routing to an integrated, multi-modal transit ecosystem. Developing a conceptual model within microscopic traffic simulation environments and softwares—such as VISSIM or AIMSUN—will allow investigators to evaluate the competitive and complementary dynamics between optimized feeder loops, private motorbikes, and bicycle-sharing networks. This simulation layer is critical for quantifying the psychological utility thresholds governing commuter modal shifts within dense campus environments.

References

- Hemdan, S., Ramadan, M., Alsultan, A., & Othman, A. (2026). Optimization of feeder buses route to connect high-speed railway stations with urban areas. *Engineering Proceedings*, 121(6). <https://doi.org/10.3390/engproc2025121006>.
- Zhu, Z., Guo, X., Zeng, J., & Zhang, S. (2017). Route design model of feeder bus service for urban rail transit stations. *Mathematical Problems in Engineering*, 2017, Article 1090457. <https://doi.org/10.1155/2017/1090457>
- Wei, Y., Jiang, N., Li, Z., Zheng, D., Chen, M., & Zhang, M. (2022). An improved ant colony algorithm for urban bus network optimization based on existing bus routes. *ISPRS International Journal of Geo-Information*, 11(5), Article 317. <https://doi.org/10.3390/ijgi11050317>
- Ozaki, Y., Watanabe, S., & Yanase, T. (2025). *OptunaHub: A Platform for Black-Box Optimization* (arXiv:2510.02798). arXiv. <https://doi.org/10.48550/arXiv.2510.02798>
- International Energy Agency. (n.d.). Transport (Energy System). Retrieved February 25, 2026, from <https://www.iea.org/energy-system/transport>.
- Huu, D. N., & Ngoc, V. N. (2021). Analysis study of current transportation status in Vietnam's urban traffic and the transition to electric two-wheelers mobility. *Sustainability*, 13(10), Article 5577. <https://doi.org/10.3390/su13105577>
- Linh Le, T. (2024). Why is Vietnam a motorcycle nation? A transport psychology study. *European Transport/Trasporti Europei*, 99(6), 1–17. <https://doi.org/10.48295/et.2024.99.6>
- Ngoc, A. M., Nishiuchi, H., Van Truong, N., & Huyen, L. T. (2021). A comparative study on travel mode share, emission, and safety in five Vietnamese cities. *International Journal of Intelligent Transportation Systems Research*, 20, 157–169. <https://doi.org/10.1007/s13177-021-00283-0>
- Japan International Cooperation Agency. (2024, December). Inauguration of Vietnam's first underground urban metro line in Ho Chi Minh City (Press release). https://www.jica.go.jp/english/information/press/2024/20241225_21.html
- Ministry of Construction of Vietnam. (2025, March 14). Developing environmentally friendly transportation system. <https://moc.gov.vn/en/news/84190/developing-environmentally-friendly-transportation-system.aspx>
- World Bank. (2015). Transforming the urban space through transit-oriented development: The 3V approach. <https://documents.worldbank.org/en/publication/documents-reports/documentdetail/647351490648306084>

- Calabrò, G., Le Pira, M., Inturri, G., Ignaccolo, M., & Pluchino, A. (2023). A simulation-optimization approach to solve the first and last mile of mass rapid transit via feeder services. *Transportation Research Procedia*, 69, 767–774. <https://doi.org/10.1016/j.trpro.2023.02.234>
- Cao, Y., Jiang, D., & Wang, S. (2022). Optimization for feeder bus route model design with station transfer. *Sustainability*, 14(5), 2780. <https://doi.org/10.3390/su14052780>
- Ding, J., Feng, S., Li, L., & Zhang, Y. (2017). Campus bus network design and evaluation based on the route property. *Tsinghua Science and Technology*, 22(5), 539–550. <https://doi.org/10.23919/TST.2017.8030543>
- Su, Y., & Yang, H. (2025). Enhancing feeder bus service coverage with Multi-Agent Reinforcement Learning: A case study in Hong Kong. *Transportation Research Part E: Logistics and Transportation Review*, 196, 103997. <https://doi.org/10.1016/j.tre.2025.103997>
- Wei, M., Liu, T., & Sun, B. (2021). Optimal routing design of feeder transit with stop selection using aggregated cell phone data and open-source GIS tool. *IEEE Transactions on Intelligent Transportation Systems*, 22(4), 2452–2464. <https://doi.org/10.1109/TITS.2020.3042014>
- Zhen, L., & Gu, W. (2024). Feeder bus service design under spatially heterogeneous demand. *Transportation Research Part A: Policy and Practice*, 189, 104214. <https://doi.org/10.1016/j.tra.2024.104214>
- Lozano, L., & Medaglia, A. L. (2012). On an exact method for the constrained shortest path problem. *Computers & Operations Research*, 40(1), 378–384. <https://doi.org/10.1016/j.cor.2012.07.008>
- Calculate Route | Routing API. (n.d.). TomTom Developer Portal. https://developer.tomtom.com/routing-api/documentation/tomtom-maps/calculate-route?utm_source=chatgpt.com
- TomTom. (2026). Ho Chi Minh traffic report. TomTom Traffic Index. Retrieved May 6, 2026, from <https://www.tomtom.com/traffic-index/city/ho-chi-minh>
- Ministry of Construction of Vietnam. (2025). *Đề xuất thí điểm vận tải công cộng theo nhu cầu tại TP. Hồ Chí Minh* [Proposal for piloting demand-responsive public transport in Ho Chi Minh City]. Portal of the Ministry of Construction. <https://moc.gov.vn/tl/tin-tuc/94232/de-xuat-thi-diem-van-tai-cong-cong-theo-nhu-cau-tai-tp--ho-chi-minh.aspx>
- Ho Chi Minh City People's Council. (2026, May 26). *TP.HCM đề xuất thí điểm xe buýt nhỏ đón khách theo nhu cầu* [HCMC proposes a pilot for demand-responsive mini-bus passenger collection]. Ho Chi Minh City Web Portal. <https://hdnd.hochiminhcity.gov.vn/tin-tuc/tin-tong-hop/tp-hcm-de-xuat-thi-diem-xe-buyt-nho-don-khach-theo-nhu-cau>

Kuah, G. K., & Perl, J. (1989). The feeder-bus network-design problem. *Journal of the Operational Research Society*, 40(8), 751–767. <https://doi.org/10.1057/jors.1989.122>

Appendix A

Google Earth Engine Code

```
/*****
=====
=====
* USER PARAMETERS
*
=====
===== ****/

var POLYGON_ASSET = "projects/charged-city-423909-
e5/assets/Pulse_FDNDP_Polygon_Area";

var LINH_XUAN_ASSET = "projects/charged-city-423909-e5/assets/Pulse-
FDNDP-LinhXuan";
var LINH_TRUNG_ASSET = "projects/charged-city-423909-e5/assets/Pulse-
FDNDP-LinhTrung";
var THIRD_WARD_ASSET = "projects/charged-city-423909-e5/assets/Pulse-
FDNDP-DongHoa";

var GRID_SIZE_M = 25;
var RADIUS_M = 500;

var HCM_DENSITY_2024 = 4554.6; // citizens/km2
var TARGET_AREA_KM2 = 13.10875; // VNU polygon area
assumption
var MIN_DENSITY = 20; // citizens/km2 for no-household-proxy
areas

var EXPORT_NAME = "VNU_" + GRID_SIZE_M +
"m_Grid_Centroids_500m_PopDensity";

/*****
=====
=====
* 1. LOAD VNU POLYGON AND PROJECTION
```

```

*
=====

===== ****/

var studyArea = ee.FeatureCollection(POLYGON_ASSET).geometry();
var utm48n = ee.Projection("EPSG:32648");

Map.centerObject(studyArea, 15);
Map.addLayer(studyArea, {color: "red"}, "VNU Polygon");

/****
=====

=====
* 2. LOAD HOUSEHOLD PROXY POINTS
*
=====

===== ****/

function loadPoints(asset, wardName) {
  return ee.FeatureCollection(asset)
    .filter(ee.Filter.notNull(["Longitude", "Latitude"]))
    .filter(ee.Filter.gt("Longitude", 100))
    .filter(ee.Filter.gt("Latitude", 1))
    .map(function(f) {
      return ee.Feature(
        ee.Geometry.Point([
          ee.Number(f.get("Longitude")),
          ee.Number(f.get("Latitude"))
        ]),
        {
          source_ward: wardName,
          household_proxy: 1
        }
      );
    });
}

```

```

var householdPoints = loadPoints(LINH_XUAN_ASSET, "Linh Xuan")
    .merge(loadPoints(LINH_TRUNG_ASSET, "Linh Trung"))
    .merge(loadPoints(THIRD_WARD_ASSET, "Third Ward"))
    .filterBounds(studyArea);

print("Household proxy points inside VNU:", householdPoints.size());
Map.addLayer(householdPoints, {color: "black"}, "Household Proxy Points");

/****
=====
=====
* 3. CREATE GRID CELLS
*
=====
===== *****/

var bounds = studyArea.bounds().transform(utm48n, 1);
var coords = ee.List(bounds.coordinates().get(0));

var xs = coords.map(function(c) { return ee.Number(ee.List(c).get(0)); });
var ys = coords.map(function(c) { return ee.Number(ee.List(c).get(1)); });

var xmin = ee.Number(xs.reduce(ee.Reducer.min()));
var xmax = ee.Number(xs.reduce(ee.Reducer.max()));
var ymin = ee.Number(ys.reduce(ee.Reducer.min()));
var ymax = ee.Number(ys.reduce(ee.Reducer.max()));

var xSeq = ee.List.sequence(xmin, xmax.subtract(GRID_SIZE_M), GRID_SIZE_M);
var ySeq = ee.List.sequence(ymin, ymax.subtract(GRID_SIZE_M), GRID_SIZE_M);

var grid = ee.FeatureCollection(
  xSeq.map(function(x) {
    return ySeq.map(function(y) {
      x = ee.Number(x);
      y = ee.Number(y);

      var cell = ee.Geometry.Rectangle(

```

```

        [x, y, x.add(GRID_SIZE_M), y.add(GRID_SIZE_M)],
        utm48n,
        false
    );

    var clipped = cell.intersection(studyArea, 1);
    var areaKm2 = clipped.area(1).divide(1000000);

    return ee.Feature(clipped, {
        grid_size_m: GRID_SIZE_M,
        grid_area_km2: areaKm2
    });
});
}).flatten()
).filterBounds(studyArea);

/****
=====
=====
* 4. CONVERT GRID CELLS TO CENTROIDS
*
=====
===== *****/

var centroidGrid = grid.map(function(f) {
    var centroid = f.geometry().centroid(1);
    var centroidUTM = centroid.transform(utm48n, 1);

    var lonLat = centroid.coordinates();
    var xy = centroidUTM.coordinates();

    return ee.Feature(centroid, {
        grid_size_m: GRID_SIZE_M,
        grid_area_km2: f.get("grid_area_km2"),
        longitude_epsg4326: lonLat.get(0),
        latitude_epsg4326: lonLat.get(1),
        x_epsg32648: xy.get(0),

```

```

        y_epsg32648: xy.get(1)
    });
});

var gridList = centroidGrid.toList(centroidGrid.size());

var indexedGrid = ee.FeatureCollection(
  ee.List.sequence(1, centroidGrid.size()).map(function(id) {
    id = ee.Number(id);
    return ee.Feature(gridList.get(id.subtract(1))).set("grid_id", id);
  })
);

/****
=====
=====
* 5. 500 m HOUSEHOLD PROXY INTENSITY
*
=====
===== *****/

var pointImage = householdPoints
  .reduceToImage({
    properties: ["household_proxy"],
    reducer: ee.Reducer.sum()
  })
  .unmask(0)
  .reproject({
    crs: utm48n,
    scale: GRID_SIZE_M
  });

var localCountImage = pointImage
  .reduceNeighborhood({
    reducer: ee.Reducer.sum(),
    kernel: ee.Kernel.circle({
      radius: RADIUS_M,

```



```

        units: "meters",
        normalize: false
    }),
    skipMasked: false
})
.rename("local_household_proxy_count")
.clip(studyArea);

var sampledGrid = localCountImage.sampleRegions({
    collection: indexedGrid,
    properties: [
        "grid_id",
        "grid_size_m",
        "grid_area_km2",
        "longitude_epsg4326",
        "latitude_epsg4326",
        "x_epsg32648",
        "y_epsg32648"
    ],
    scale: GRID_SIZE_M,
    projection: utm48n,
    geometries: true,
    tileScale: 4
});

var maxLocalCount =
ee.Number(sampledGrid.aggregate_max("local_household_proxy_count"));

print("Maximum 500 m local household count:", maxLocalCount);

/****
=====
=====
* 6. MASS-BALANCED POPULATION DENSITY
*
=====
===== ****/

```

```

var withRatio = sampledGrid.map(function(f) {
  var localCount = ee.Number(f.get("local_household_proxy_count"));
  var areaKm2 = ee.Number(f.get("grid_area_km2"));

  var ratio = ee.Number(
    ee.Algorithms.If(
      maxLocalCount.gt(0),
      localCount.divide(maxLocalCount),
      0
    )
  );

  return f.set({
    density_ratio_to_vnu_max: ratio,
    ratio_area_km2: ratio.multiply(areaKm2)
  });
});

var totalGridAreaKm2 =
ee.Number(withRatio.aggregate_sum("grid_area_km2"));
var totalRatioAreaKm2 =
ee.Number(withRatio.aggregate_sum("ratio_area_km2"));

var targetPopulation =
ee.Number(HCM_DENSITY_2024).multiply(TARGET_AREA_KM2);
var backgroundPopulation =
ee.Number(MIN_DENSITY).multiply(totalGridAreaKm2);

var densityScale = ee.Number(
  ee.Algorithms.If(
    totalRatioAreaKm2.gt(0),

    targetPopulation.subtract(backgroundPopulation).divide(totalRatioAreaKm2),
    0
  )
);

```

```

var finalGrid = withRatio.map(function(f) {
  var ratio = ee.Number(f.get("density_ratio_to_vnu_max"));
  var areaKm2 = ee.Number(f.get("grid_area_km2"));

  var density = ee.Number(MIN_DENSITY).add(ratio.multiply(densityScale));
  var population = density.multiply(areaKm2);

  return f.set({
    grid_population_density: density,
    grid_population: population
  });
}).sort("grid_id");

/****
=====
=====
* 7. VALIDATION CHECK
*
=====
===== *****/

print("Grid size used:", GRID_SIZE_M, "m");
print("Number of grid centroids:", finalGrid.size());
print("Total clipped grid area km2:", totalGridAreaKm2);
print("Target population:", targetPopulation);
print("SUM(grid_population):", finalGrid.aggregate_sum("grid_population"));
print("Difference:",
ee.Number(finalGrid.aggregate_sum("grid_population")).subtract(targetPopulati
on));
print("Sample output:", finalGrid.limit(10));

Map.addLayer(localCountImage, {
  min: 0,
  max: maxLocalCount,
  palette: ["white", "yellow", "orange", "red"]
}, "500 m Household Proxy Count");

```

```
Map.addLayer(finalGrid, {color: "blue"}, "Grid Centroids");
```

```
/*  
=====
```

* 8. EXPORT TO GOOGLE DRIVE

```
*  
=====
```

===== */

```
Export.table.toDrive({  
  collection: finalGrid,  
  description: EXPORT_NAME,  
  fileNamePrefix: EXPORT_NAME,  
  fileFormat: "CSV",  
  selectors: [  
    "grid_id",  
    "grid_size_m",  
    "grid_area_km2",  
    "longitude_epsg4326",  
    "latitude_epsg4326",  
    "x_epsg32648",  
    "y_epsg32648",  
    "local_household_proxy_count",  
    "density_ratio_to_vnu_max",  
    "grid_population_density",  
    "grid_population"  
  ]  
});
```

Appendix B

TomTom Data Retrieve – Google Colab Script

```
# =====
# 1. Install required packages
# =====
!pip -q install pandas openpyxl requests shapely pyproj folium tqdm
# =====
# 2. Import libraries
# =====
import os
import time
import json
import math
import zipfile
import xml.etree.ElementTree as ET
from pathlib import Path
from typing import Dict, List, Optional, Tuple, Any

import pandas as pd
import numpy as np
import requests
from tqdm.auto import tqdm

from shapely.geometry import Point, LineString, Polygon, mapping
from shapely.ops import transform
from pyproj import Transformer

import folium
# =====
# 3. User configuration
# =====

# --- Input files ---
# Upload these files to /content/ in Colab before running the notebook.
OD_LINKS_FILE = "/content/OD_matrix_links.xlsx"
STATIONS_FILE = "/content/vnu_bus_stations.csv"
POLYGON_KMZ_FILE = "/content/vnu_polygon_area.kmz"
```

```

# --- TomTom API key ---
# Paste your TomTom Routing API key here.
TOMTOM_API_KEY = "hUxzA1Ph66qpTDYofMGwVGtqLJ8ReB7Z"

# --- Routing parameters ---
ROUTE_TYPE = "shortest"
COMPUTE_TRAVEL_TIME_FOR = "all"
TRAFFIC = "true"
MAX_ALTERNATIVES = 5

# Attempt order:
# 1) bus mode,
# 2) car fallback,
# 3) None = omit travelMode parameter and let TomTom use default mode.
TRAVEL_MODE_ATTEMPTS = ["bus", None]

# --- Spatial validation parameters ---
# EPSG:4326 = WGS84 lon/lat.
# EPSG:32648 = WGS84 / UTM zone 48N, suitable for Ho Chi Minh City distance
calculations.
WGS84_CRS = "EPSG:4326"
METRIC_CRS = "EPSG:32648"

# Route may be very close to polygon boundary due to map-matching.
# A small tolerance avoids rejecting valid boundary-following routes.
POLYGON_TOLERANCE_M = 5.0

# If a station point is within this distance from the route line and lies between
# the route endpoints, it is treated as an intermediate station conflict.
STATION_INTERSECTION_BUFFER_M = 18.0

# Stations close to origin or destination are not treated as intermediate conflicts.
ENDPOINT_EXCLUSION_RADIUS_M = 45.0

# API safety controls
REQUEST_TIMEOUT_SECONDS = 30
API_SLEEP_SECONDS = 1.5

```

```

# Run switch.
# Keep False for the first run. After the single API test works, change to True.
RUN_FULL_COLLECTION = True

# --- Output folder ---
OUT_DIR = Path("/content/tomtom_vnu_outputs")
OUT_DIR.mkdir(parents=True, exist_ok=True)

print("Output folder:", OUT_DIR)
# =====

# 4. Optional: upload files directly in Colab
# =====
# Run this cell only if the files are not already in /content/.

try:
    from google.colab import files
    print("Colab upload tool is available.")
    print("Upload OD_matrix_links.xlsx, vnu_bus_stations.csv, and
vnu_polygon_area.kmz if needed.")
    # uploaded = files.upload()
except Exception:
    print("Not running in Colab, or google.colab is not available.")
# =====

# 5. Helper functions: input loading and standardization
# =====

def normalize_columns(df: pd.DataFrame) -> pd.DataFrame:
    """Standardize column names while keeping readable field names."""
    df = df.copy()
    df.columns = [
        str(c).strip()
        .replace("\uffff", "")
        .replace("\n", " ")
        .replace("\r", " ")
        for c in df.columns
    ]
    return df

```

```

def read_od_links(path: str) -> pd.DataFrame:
    """Read OD matrix links from Excel. Uses the first sheet by default."""
    xl = pd.ExcelFile(path)
    df = pd.read_excel(path, sheet_name=xl.sheet_names[0])
    df = normalize_columns(df)

    # Accept both naming styles if files are updated later.
    rename_map = {
        "Link Type": "link_type",
        "Link_name": "link_name",
        "from_station_id": "from_station_id",
        "to_station_id": "to_station_id",
    }
    df = df.rename(columns=rename_map)

    required = ["from_station_id", "to_station_id"]
    for col in required:
        if col not in df.columns:
            raise ValueError(f"Missing required OD link column: {col}")

    if "link_name" not in df.columns:
        df["link_name"] = df["from_station_id"].astype(str) + "_" +
df["to_station_id"].astype(str)

    if "link_type" not in df.columns:
        df["link_type"] = ""

    df["from_station_id"] = df["from_station_id"].astype(int)
    df["to_station_id"] = df["to_station_id"].astype(int)

    return df

def read_stations(path: str) -> pd.DataFrame:
    """Read station coordinates."""
    df = pd.read_csv(path)
    df = normalize_columns(df)

```



```

required = ["station_id", "longitude", "latitude"]
for col in required:
    if col not in df.columns:
        raise ValueError(f"Missing required station column: {col}")

if "station_name" not in df.columns:
    df["station_name"] = ""

df["station_id"] = df["station_id"].astype(int)
df["longitude"] = pd.to_numeric(df["longitude"], errors="coerce")
df["latitude"] = pd.to_numeric(df["latitude"], errors="coerce")

if df[["longitude", "latitude"]].isna().any().any():
    bad = df[df[["longitude", "latitude"]].isna().any(axis=1)]
    raise ValueError(f"Station file contains invalid coordinates:\n{bad}")

return df

def attach_station_coordinates(links_df: pd.DataFrame, stations_df: pd.DataFrame) -
> pd.DataFrame:
    """Attach from/to longitude and latitude to each OD link."""
    stations_small = stations_df[["station_id", "longitude", "latitude",
"station_name"]].copy()

    out = links_df.merge(
        stations_small.rename(columns={
            "station_id": "from_station_id",
            "longitude": "from_lon",
            "latitude": "from_lat",
            "station_name": "from_station_name",
        }),
        on="from_station_id",
        how="left",
    )

    out = out.merge(
        stations_small.rename(columns={
            "station_id": "to_station_id",

```

```

        "longitude": "to_lon",
        "latitude": "to_lat",
        "station_name": "to_station_name",
    }},
    on="to_station_id",
    how="left",
)

missing = out[out[["from_lon", "from_lat", "to_lon", "to_lat"]].isna().any(axis=1)]
if len(missing) > 0:
    raise ValueError(
        "Some links cannot be matched with station coordinates. Check station
IDs.\n"
        + str(missing[["from_station_id", "to_station_id"]].head(20))
    )

return out

links_raw = read_od_links(OD_LINKS_FILE)
stations = read_stations(STATIONS_FILE)
links = attach_station_coordinates(links_raw, stations)

print("OD links:", links.shape)
display(links.head())

print("Stations:", stations.shape)
display(stations.head())
# =====
# 6. Helper functions: KMZ polygon loading
# =====

def extract_kml_text_from_kmz(kmz_path: str) -> str:
    """Extract the first KML text file from a KMZ archive."""
    with zipfile.ZipFile(kmz_path, "r") as z:
        kml_files = [name for name in z.namelist() if
name.lower().endswith(".kml")]
        if not kml_files:
            raise ValueError("No .kml file found inside the KMZ.")

```

```

        return z.read(kml_files[0]).decode("utf-8", errors="ignore")

def parse_first_polygon_from_kml(kml_text: str) -> Polygon:
    """Parse the first polygon coordinates from KML text into a Shapely polygon."""
    root = ET.fromstring(kml_text)

    # KML namespace handling
    ns = {"kml": "http://www.opengis.net/kml/2.2"}

    # Search standard KML polygon coordinates first.
    coord_nodes =
root.findall("./kml:Polygon//kml:outerBoundaryIs//kml:LinearRing//kml:coordinates
", ns)

    # Fallback: any coordinates node.
    if not coord_nodes:
        coord_nodes = root.findall("./kml:coordinates", ns)

    if not coord_nodes:
        raise ValueError("No polygon coordinates found in KML.")

    coord_text = coord_nodes[0].text.strip()

    coords = []
    for item in coord_text.split():
        parts = item.split(",")
        if len(parts) >= 2:
            lon = float(parts[0])
            lat = float(parts[1])
            coords.append((lon, lat))

    if len(coords) < 3:
        raise ValueError("Polygon has fewer than 3 coordinate points.")

    poly = Polygon(coords)

    if not poly.is_valid:
        poly = poly.buffer(0)

```

```

    if poly.is_empty:
        raise ValueError("Parsed polygon is empty or invalid.")

    return poly

kml_text = extract_kml_text_from_kmz(POLYGON_KMZ_FILE)
study_area_wgs84 = parse_first_polygon_from_kml(kml_text)

print("Polygon loaded.")
print("Polygon bounds:", study_area_wgs84.bounds)

# =====
# 7. Coordinate transformation and spatial objects
# =====

to_metric_transformer = Transformer.from_crs(WGS84_CRS, METRIC_CRS,
always_xy=True)
to_wgs84_transformer = Transformer.from_crs(METRIC_CRS, WGS84_CRS,
always_xy=True)

def to_metric_geom(geom):
    return transform(to_metric_transformer.transform, geom)

def to_wgs84_geom(geom):
    return transform(to_wgs84_transformer.transform, geom)

study_area_utm = to_metric_geom(study_area_wgs84)
study_area_utm_buffered = study_area_utm.buffer(POLYGON_TOLERANCE_M)

station_points_wgs84 = {
    int(row.station_id): Point(float(row.longitude), float(row.latitude))
    for row in stations.itertuples(index=False)
}

station_points_utm = {
    sid: to_metric_geom(pt)
    for sid, pt in station_points_wgs84.items()
}

```

```

}

print("Metric polygon area, km²:", study_area_utm.area / 1_000_000)
print("Number of station points:", len(station_points_utm))
# =====
# 8. TomTom API functions
# =====

def travel_mode_label(mode: Optional[str]) -> str:
    if mode is None:
        return "tombom_default"
    return str(mode)

def build_tombom_params(api_key: str, travel_mode: Optional[str]) -> Dict[str, Any]:
    """Build TomTom request parameters.

    If travel_mode is None, the travelMode parameter is omitted.
    """
    params = {
        "key": api_key,
        "routeType": ROUTE_TYPE,
        "traffic": TRAFFIC,
        "computeTravelTimeFor": COMPUTE_TRAVEL_TIME_FOR,
        "routeRepresentation": "polyline",
        "maxAlternatives": MAX_ALTERNATIVES,
    }

    if travel_mode is not None:
        params["travelMode"] = travel_mode

    return params

def calculate_tombom_route(
    from_lat: float,
    from_lon: float,
    to_lat: float,
    to_lon: float,
    travel_mode: Optional[str],

```

```

    api_key: str,
) -> Tuple[Optional[Dict[str, Any]], Optional[str]]:
    """Call TomTom Calculate Route API for one OD pair.

    Returns:
        response_json, error_message
    """

    if not api_key or api_key == "PASTE_YOUR_TOMTOM_API_KEY_HERE":
        return None, "TOMTOM_API_KEY is missing. Paste your API key in the
configuration cell."

    url = (
        "https://api.tomtom.com/routing/1/calculateRoute/"
        f"{from_lat},{from_lon}:{to_lat},{to_lon}/json"
    )

    params = build_tomtom_params(api_key, travel_mode)

    # Retry logic for HTTP 429 Too Many Requests.
    response = None
    last_429_message = ""

    for retry in range(5):
        try:
            response = requests.get(url, params=params,
timeout=REQUEST_TIMEOUT_SECONDS)
        except requests.RequestException as e:
            return None, f"REQUEST_EXCEPTION: {e}"

        if response.status_code == 429:
            wait_time = 10 * (retry + 1)
            try:
                last_429_message = json.dumps(response.json())[:500]
            except Exception:
                last_429_message = response.text[:500]

            print(f"HTTP 429 Too Many Requests. Waiting {wait_time} seconds...")
            time.sleep(wait_time)

```

```

        continue

    break

if response is None:
    return None, "NO_RESPONSE"

if response.status_code == 429:
    return None, f"HTTP_429_AFTER_5_RETRIES: {last_429_message}"

if response.status_code != 200:
    try:
        err_json = response.json()
        err_msg = json.dumps(err_json)[:500]
    except Exception:
        err_msg = response.text[:500]
    return None, f"HTTP_{response.status_code}: {err_msg}"

try:
    return response.json(), None
except Exception as e:
    return None, f"JSON_PARSE_ERROR: {e}"

def route_json_to_linestring(route: Dict[str, Any]) -> Optional[LineString]:
    """Convert TomTom route geometry to Shapely LineString in WGS84 lon/lat."""
    points_lonlat = []

    for leg in route.get("legs", []):
        for p in leg.get("points", []):
            if "longitude" in p and "latitude" in p:
                lonlat = (float(p["longitude"]), float(p["latitude"]))
                if not points_lonlat or lonlat != points_lonlat[-1]:
                    points_lonlat.append(lonlat)

    if len(points_lonlat) < 2:
        return None

    return LineString(points_lonlat)

```

```

def get_average_time_seconds(summary: Dict[str, Any]) -> Tuple[Optional[float], str]:
    """Get historical average travel time if available; otherwise use best-estimate
    travel time."""
    if "historicTrafficTravelTimeInSeconds" in summary:
        return float(summary["historicTrafficTravelTimeInSeconds"]),
        "historicTrafficTravelTimeInSeconds"

    if "noTrafficTravelTimeInSeconds" in summary:
        # If historical traffic is unavailable, no-traffic time is at least a stable
        average-like reference.
        return float(summary["noTrafficTravelTimeInSeconds"]),
        "noTrafficTravelTimeInSeconds"

    if "travelTimeInSeconds" in summary:
        return float(summary["travelTimeInSeconds"]), "travelTimeInSeconds"

    return None, "missing_time_field"
# =====

# 9. Route validation functions
# =====

def check_route_inside_polygon(route_line_wgs84: LineString) -> Tuple[bool, str]:
    """Check whether route is inside the VNU polygon with tolerance."""
    route_line_utm = to_metric_geom(route_line_wgs84)

    if study_area_utm_buffered.covers(route_line_utm):
        return True, "inside_polygon"

    outside_length_m =
route_line_utm.difference(study_area_utm_buffered).length
    return False, f"outside_polygon_length_m={outside_length_m:.2f}"

def find_intermediate_station_conflicts(
    route_line_wgs84: LineString,
    from_station_id: int,
    to_station_id: int,
) -> List[Dict[str, Any]]:

```


"""Find stations located in the middle of the route.

The origin and destination stations are excluded.

Stations near the route endpoints are also ignored because they are not middle-route conflicts.

```
"""
route_line_utm = to_metric_geom(route_line_wgs84)
route_len = route_line_utm.length

conflicts = []
exclude_ids = {int(from_station_id), int(to_station_id)}

for sid, pt_utm in station_points_utm.items():
    if sid in exclude_ids:
        continue

    dist_m = route_line_utm.distance(pt_utm)

    if dist_m <= STATION_INTERSECTION_BUFFER_M:
        along_m = route_line_utm.project(pt_utm)

        is_middle = (
            along_m > ENDPOINT_EXCLUSION_RADIUS_M
            and along_m < route_len - ENDPOINT_EXCLUSION_RADIUS_M
        )

        if is_middle:
            station_name = stations.loc[stations["station_id"] == sid,
"station_name"]
            station_name = station_name.iloc[0] if len(station_name) else ""

            conflicts.append({
                "station_id": sid,
                "station_name": station_name,
                "distance_to_route_m": round(float(dist_m), 3),
                "position_along_route_m": round(float(along_m), 3),
            })
```

```

    return conflicts

def validate_route(
    route_line_wgs84: Optional[LineString],
    from_station_id: int,
    to_station_id: int,
) -> Tuple[bool, str, List[Dict[str, Any]]]:
    """Apply polygon and intermediate-station validation."""
    if route_line_wgs84 is None:
        return False, "missing_route_geometry", []

    inside, polygon_msg = check_route_inside_polygon(route_line_wgs84)
    if not inside:
        return False, polygon_msg, []

    conflicts = find_intermediate_station_conflicts(
        route_line_wgs84,
        from_station_id=from_station_id,
        to_station_id=to_station_id,
    )

    if conflicts:
        conflict_ids = [str(c["station_id"]) for c in conflicts[:10]]
        msg = "intermediate_station_conflict=" + ",".join(conflict_ids)
        if len(conflicts) > 10:
            msg += f"_and_{len(conflicts)-10}_more"
        return False, msg, conflicts

    return True, "valid_route", []

# =====
# 10. Select the first valid TomTom route among alternatives
# =====

def select_valid_route_from_response(
    response_json: Dict[str, Any],
    from_station_id: int,
    to_station_id: int,
) -> Tuple[Optional[Dict[str, Any]], Optional[LineString], str, List[Dict[str, Any]]]:

```

```

"""Return the first route alternative satisfying all GIS validation rules."""
routes = response_json.get("routes", [])

if not routes:
    return None, None, "no_routes_returned", []

validation_messages = []

for idx, route in enumerate(routes):
    route_line = route_json_to_linestring(route)

    valid, msg, conflicts = validate_route(
        route_line,
        from_station_id=from_station_id,
        to_station_id=to_station_id,
    )

    if valid:
        return route, route_line, f"selected_candidate_index={idx}", []

    validation_messages.append(f"candidate_{idx}:{msg}")

return None, None, " | ".join(validation_messages), []

def build_result_row_from_route(
    link_row: pd.Series,
    selected_route: Dict[str, Any],
    route_line_wgs84: LineString,
    travel_mode_used: Optional[str],
    validation_detail: str,
) -> Dict[str, Any]:
    """Build one accepted output record."""
    summary = selected_route.get("summary", {})

    distance_m = summary.get("lengthInMeters", np.nan)
    distance_km = float(distance_m) / 1000 if pd.notna(distance_m) else np.nan

    avg_time_s, avg_time_source = get_average_time_seconds(summary)

```

```

avg_time_min = avg_time_s / 60 if avg_time_s is not None else np.nan

avg_speed_kmh = np.nan
if avg_time_s is not None and avg_time_s > 0 and pd.notna(distance_km):
    avg_speed_kmh = distance_km / (avg_time_s / 3600)

return {
    "from_station_id": int(link_row["from_station_id"]),
    "to_station_id": int(link_row["to_station_id"]),
    "link_name": link_row.get("link_name", ""),
    "link_type": link_row.get("link_type", ""),
    "from_lat": float(link_row["from_lat"]),
    "from_lon": float(link_row["from_lon"]),
    "to_lat": float(link_row["to_lat"]),
    "to_lon": float(link_row["to_lon"]),
    "from_station_name": link_row.get("from_station_name", ""),
    "to_station_name": link_row.get("to_station_name", ""),

    "validation_status": "VALID",
    "validation_detail": validation_detail,
    "travel_mode_used": travel_mode_label(travel_mode_used),

    "gis_route_distance_m": distance_m,
    "gis_route_distance_km": distance_km,

    "average_travel_time_s": avg_time_s,
    "average_travel_time_min": avg_time_min,
    "average_travel_time_source": avg_time_source,

    "average_speed_kmh": avg_speed_kmh,

    "tomtom_travel_time_s": summary.get("travelTimeInSeconds", np.nan),
    "tomtom_no_traffic_time_s":
summary.get("noTrafficTravelTimeInSeconds", np.nan),
    "tomtom_historic_traffic_time_s":
summary.get("historicTrafficTravelTimeInSeconds", np.nan),
    "tomtom_live_traffic_time_s":
summary.get("liveTrafficIncidentsTravelTimeInSeconds", np.nan),

```

```

        "tomtom_traffic_delay_s": summary.get("trafficDelayInSeconds", np.nan),

        "route_wkt": route_line_wgs84.wkt,
        "route_geojson": json.dumps(mapping(route_line_wgs84)),
        "api_error": "",
    }

def build_blank_result_row(
    link_row: pd.Series,
    validation_detail: str,
    api_error: str = "",
) -> Dict[str, Any]:
    """Build one unresolved output record with blank numerical fields."""
    return {
        "from_station_id": int(link_row["from_station_id"]),
        "to_station_id": int(link_row["to_station_id"]),
        "link_name": link_row.get("link_name", ""),
        "link_type": link_row.get("link_type", ""),
        "from_lat": float(link_row["from_lat"]),
        "from_lon": float(link_row["from_lon"]),
        "to_lat": float(link_row["to_lat"]),
        "to_lon": float(link_row["to_lon"]),
        "from_station_name": link_row.get("from_station_name", ""),
        "to_station_name": link_row.get("to_station_name", ""),

        "validation_status": "NO_VALID_ROUTE_AFTER_ALL_MODES",
        "validation_detail": validation_detail,
        "travel_mode_used": "",

        "gis_route_distance_m": np.nan,
        "gis_route_distance_km": np.nan,

        "average_travel_time_s": np.nan,
        "average_travel_time_min": np.nan,
        "average_travel_time_source": "",

        "average_speed_kmh": np.nan,

```

```

        "tomtom_travel_time_s": np.nan,
        "tomtom_no_traffic_time_s": np.nan,
        "tomtom_historic_traffic_time_s": np.nan,
        "tomtom_live_traffic_time_s": np.nan,
        "tomtom_traffic_delay_s": np.nan,

        "route_wkt": "",
        "route_geojson": "",
        "api_error": api_error,
    }
}
# =====
# 11. Single API test section
# =====

def test_one_tomtom_request(test_mode: Optional[str] = "bus") -> None:
    """Test one TomTom request using the first OD link."""
    row = links.iloc[0]

    print("Testing one TomTom request")
    print("Link:", row["from_station_id"], "->", row["to_station_id"])
    print("Mode:", travel_mode_label(test_mode))

    response_json, error = calculate_tomtom_route(
        from_lat=float(row["from_lat"]),
        from_lon=float(row["from_lon"]),
        to_lat=float(row["to_lat"]),
        to_lon=float(row["to_lon"]),
        travel_mode=test_mode,
        api_key=TOMTOM_API_KEY,
    )

    if error:
        print("API test failed:")
        print(error)
        return

    routes = response_json.get("routes", [])
    print("API test successful.")

```

```

print("Number of routes returned:", len(routes))

if routes:
    summary = routes[0].get("summary", {})
    print("First route summary:")
    print(json.dumps(summary, indent=2)[:1500])

    selected_route, route_line, detail, conflicts =
select_valid_route_from_response(
    response_json,
    from_station_id=int(row["from_station_id"]),
    to_station_id=int(row["to_station_id"]),
    )

    print("GIS validation result:", "VALID" if selected_route is not None else
"INVALID")
    print("Validation detail:", detail)

# Run the single request test.
# Change "bus" to "car" or None if you want to test another mode.
test_one_tomtom_request(test_mode="bus")
# =====

# 12. Batch processing function
# =====

def process_one_link(link_row: pd.Series) -> Dict[str, Any]:
    """Process one OD link using bus first, then car/default fallback."""
    all_attempt_messages = []
    api_errors = []

    for travel_mode in TRAVEL_MODE_ATTEMPTS:
        response_json, error = calculate_tomtom_route(
            from_lat=float(link_row["from_lat"]),
            from_lon=float(link_row["from_lon"]),
            to_lat=float(link_row["to_lat"]),
            to_lon=float(link_row["to_lon"]),
            travel_mode=travel_mode,
            api_key=TOMTOM_API_KEY,

```

```

    )

    mode_label = travel_mode_label(travel_mode)

    if error:
        msg = f"{mode_label}: {error}"
        all_attempt_messages.append(msg)
        api_errors.append(msg)
        time.sleep(API_SLEEP_SECONDS)
        continue

    selected_route, route_line, detail, conflicts =
select_valid_route_from_response(
        response_json=response_json,
        from_station_id=int(link_row["from_station_id"]),
        to_station_id=int(link_row["to_station_id"]),
    )

    if selected_route is not None:
        return build_result_row_from_route(
            link_row=link_row,
            selected_route=selected_route,
            route_line_wgs84=route_line,
            travel_mode_used=travel_mode,
            validation_detail=detail,
        )

    all_attempt_messages.append(f"{mode_label}: {detail}")
    time.sleep(API_SLEEP_SECONDS)

    return build_blank_result_row(
        link_row=link_row,
        validation_detail=" | | ".join(all_attempt_messages),
        api_error=" | | ".join(api_errors),
    )

def run_batch_collection(links_df: pd.DataFrame) -> pd.DataFrame:
    """Run TomTom routing collection for all OD links."""

```



```

results = []

for _, row in tqdm(links_df.iterrows(), total=len(links_df), desc="Collecting
TomTom routes"):
    result = process_one_link(row)
    results.append(result)

return pd.DataFrame(results)

# =====
# 14. Export results: CSV, Excel, GeoJSON, and HTML map
# =====

def export_geojson(results_df: pd.DataFrame, path: Path) -> None:
    """Export valid route geometries as GeoJSON FeatureCollection."""
    features = []

    valid_df = results_df[
        (results_df["validation_status"] == "VALID")
        & (results_df["route_geojson"].astype(str).str.len() > 0)
    ].copy()

    for _, row in valid_df.iterrows():
        geom = json.loads(row["route_geojson"])
        props = row.drop(labels=["route_geojson"]).to_dict()

        # Keep GeoJSON properties compact.
        props.pop("route_wkt", None)

        features.append({
            "type": "Feature",
            "geometry": geom,
            "properties": props,
        })

    fc = {
        "type": "FeatureCollection",
        "features": features,
    }

```

```

with open(path, "w", encoding="utf-8") as f:
    json.dump(fc, f, ensure_ascii=False, indent=2)

def export_html_map(results_df: pd.DataFrame, stations_df: pd.DataFrame,
polygon_wgs84: Polygon, path: Path) -> None:
    """Export an HTML map for route checking."""
    centroid = polygon_wgs84.centroid
    m = folium.Map(location=[centroid.y, centroid.x], zoom_start=15,
tiles="OpenStreetMap")

    # Polygon layer
    poly_coords_latlon = [(lat, lon) for lon, lat in list(polygon_wgs84.exterior.coords)]
    folium.Polygon(
        locations=poly_coords_latlon,
        popup="VNU Polygon",
        fill=False,
        weight=2,
    ).add_to(m)

    # Station layer
    for _, st in stations_df.iterrows():
        folium.CircleMarker(
            location=[st["latitude"], st["longitude"]],
            radius=3,
            popup=f'{st["station_id"]}: {st.get("station_name", "")}',
            fill=True,
        ).add_to(m)

    # Route layer
    if len(results_df) > 0:
        valid_df = results_df[
            (results_df["validation_status"] == "VALID")
            & (results_df["route_geojson"].astype(str).str.len() > 0)
        ].copy()

        for _, row in valid_df.iterrows():
            geom = json.loads(row["route_geojson"])

```

```

coords_latlon = [(lat, lon) for lon, lat in geom["coordinates"]]

popup = (
    f'Link: {row["link_name"]} <br>'
    f'Mode: {row["travel_mode_used"]} <br>'
    f'Distance km: {row["gis_route_distance_km"]:.3f} <br>'
    f'Avg time min: {row["average_travel_time_min"]:.3f} <br>'
    f'Avg speed km/h: {row["average_speed_kmh"]:.2f}'
)

folium.PolyLine(
    locations=coords_latlon,
    weight=3,
    popup=popup,
).add_to(m)

m.save(str(path))

if len(results_df) > 0:
    csv_path = OUT_DIR / "tomtom_vnu_link_results.csv"
    xlsx_path = OUT_DIR / "tomtom_vnu_link_results.xlsx"
    geojson_path = OUT_DIR / "tomtom_vnu_link_results.geojson"
    html_path = OUT_DIR / "tomtom_vnu_link_map.html"

    results_df.to_csv(csv_path, index=False, encoding="utf-8-sig")
    results_df.to_excel(xlsx_path, index=False)
    export_geojson(results_df, geojson_path)
    export_html_map(results_df, stations, study_area_wgs84, html_path)

    print("Saved outputs:")
    print(csv_path)
    print(xlsx_path)
    print(geojson_path)
    print(html_path)
else:
    print("No results to export yet. Run full collection first.")

```

Appendix C

Grid Link Assignment – Colab Script

```
import os
from pathlib import Path
import numpy as np
import pandas as pd
from pyproj import Transformer

pd.set_option('display.max_columns', 120)
pd.set_option('display.width', 180)

# =====
# USER PARAMETERS
# =====
LINK_FILE = "Links_data.xlsx"
GRID_FILE = "VNU_100m_Grid_Data.csv"

# The new grid format should contain grid_population.
# Keep True to prevent accidental fallback to uniform-density calculation.
REQUIRE_GRID_POPULATION_FIELD = True

# Used only if REQUIRE_GRID_POPULATION_FIELD = False and grid_population is
# missing.
DEFAULT_CELL_SIZE_M = 100
DEFAULT_DENSITY_PEOPLE_PER_KM2 = 4554.6

# Maximum eligible distance from grid to candidate Forward link.
CATCHMENT_RADIUS_KM = 0.95

# Candidate rule options:
# "line"      = Sgij, shortest grid-centroid-to-link distance <= radius
# "midpoint" = grid centroid to link midpoint distance <= radius
# "either"    = line OR midpoint distance is within radius
CANDIDATE_RULE = "line"

# Chunk size controls memory usage.
CHUNK_SIZE = 2000
```

```

OUT_DIR = Path("./outputs")
OUT_DIR.mkdir(exist_ok=True)

# =====
# HELPER FUNCTIONS
# =====
def resolve_path(file_name):
    candidates = [Path(file_name), Path('/content') / file_name, Path('/mnt/data') /
file_name]
    for p in candidates:
        if p.exists():
            return p
    raise FileNotFoundError(f"Could not find {file_name}. Upload it or update
LINK_FILE/GRID_FILE.")

def first_existing(columns, candidates):
    normalized = {str(c).strip(): c for c in columns}
    for c in candidates:
        if c in normalized:
            return normalized[c]
    return None

def to_numeric_series(df, col, field_name):
    values = pd.to_numeric(df[col], errors='coerce')
    if values.isna().any():
        bad_count = int(values.isna().sum())
        raise ValueError(f"Column {col!r} contains {bad_count} non-numeric values
and cannot be used as {field_name}.")
    return values.astype(float)

def point_segment_dist_matrix(px, py, ax, ay, bx, by):
    # Distance from n points to m line segments. Returns n x m in meters.
    px = px[:, None]
    py = py[:, None]
    ax = ax[None, :]
    ay = ay[None, :]
    bx = bx[None, :]

```

```

by = by[None, :]

abx = bx - ax
aby = by - ay
apx = px - ax
apy = py - ay

denom = abx * abx + aby * aby
denom = np.where(denom == 0, 1e-12, denom)

t = (apx * abx + apy * aby) / denom
t = np.clip(t, 0.0, 1.0)

cx = ax + t * abx
cy = ay + t * aby
return np.sqrt((px - cx) ** 2 + (py - cy) ** 2)

def point_midpoint_dist_matrix(px, py, mx, my):
    # Distance from n points to m link midpoints. Returns n x m in meters.
    px = px[:, None]
    py = py[:, None]
    mx = mx[None, :]
    my = my[None, :]
    return np.sqrt((px - mx) ** 2 + (py - my) ** 2)

# =====
# LOAD DATA
# =====
LINK_PATH = resolve_path(LINK_FILE)
GRID_PATH = resolve_path(GRID_FILE)

links = pd.read_excel(LINK_PATH)
grids = pd.read_csv(GRID_PATH)
links.columns = [str(c).strip() for c in links.columns]
grids.columns = [str(c).strip() for c in grids.columns]

print("Link file:", LINK_PATH)
print("Grid file:", GRID_PATH)

```

```

print("Links shape:", links.shape)
print("Grid shape:", grids.shape)

# =====
# VALIDATE AND STANDARDIZE LINK INPUTS
# =====
required_link_cols = [
    'from_station_id', 'to_station_id', 'link_type',
    'from_lat', 'from_lon', 'to_lat', 'to_lon'
]
missing = [c for c in required_link_cols if c not in links.columns]
if missing:
    raise ValueError(f"Missing required link columns: {missing}")

if 'link_name' not in links.columns:
    links['link_name'] = links['from_station_id'].astype(int).astype(str) + '_' +
links['to_station_id'].astype(int).astype(str)

links['link_type'] = links['link_type'].astype(str).str.strip()
valid_types = {'Forward', 'Backward', 'U-turn'}
bad_types = sorted(set(links['link_type'].unique()) - valid_types)
if bad_types:
    raise ValueError(f"Unexpected link_type values: {bad_types}. Expected only
{valid_types}")

print("link_type counts:")
print(links['link_type'].value_counts(dropna=False))

# =====
# VALIDATE AND STANDARDIZE GRID INPUTS
# =====
# New-format preferred columns are placed first.
id_candidates = ['grid_id', 'Grid_ID', 'id']
lon_candidates = ['longitude_epsg4326', 'lon', 'longitude', 'centroid_lon']
lat_candidates = ['latitude_epsg4326', 'lat', 'latitude', 'centroid_lat']
x_candidates = ['x_epsg32648', 'x_utm', 'centroid_x_utm', 'x']
y_candidates = ['y_epsg32648', 'y_utm', 'centroid_y_utm', 'y']
pop_candidates = ['grid_population', 'pop_g', 'population', 'Popg', 'pop']

```

```

density_candidates = ['grid_population_density', 'population_density',
'density_people_per_km2']
area_candidates = ['grid_area_km2', 'area_km2', 'cell_area_km2']
size_candidates = ['grid_size_m', 'cell_size_m']

grid_id_col = first_existing(grids.columns, id_candidates)
lon_col = first_existing(grids.columns, lon_candidates)
lat_col = first_existing(grids.columns, lat_candidates)
x_col = first_existing(grids.columns, x_candidates)
y_col = first_existing(grids.columns, y_candidates)
pop_col = first_existing(grids.columns, pop_candidates)
density_col = first_existing(grids.columns, density_candidates)
area_col = first_existing(grids.columns, area_candidates)
size_col = first_existing(grids.columns, size_candidates)

if grid_id_col is None:
    grids['grid_id'] = np.arange(1, len(grids) + 1)
    grid_id_col = 'grid_id'

if pop_col is None and REQUIRE_GRID_POPULATION_FIELD:
    raise ValueError(
        "The new grid file must contain a grid_population column. "
        "Set REQUIRE_GRID_POPULATION_FIELD = False only if you intentionally
want the fallback density × area calculation."
    )

# =====
# PROJECT TO UTM 48N
# =====
transformer_to_utm = Transformer.from_crs('EPSG:4326', 'EPSG:32648',
always_xy=True)
transformer_to_wgs = Transformer.from_crs('EPSG:32648', 'EPSG:4326',
always_xy=True)

if x_col is None or y_col is None:
    if lon_col is None or lat_col is None:
        raise ValueError('Grid file must contain either UTM x/y columns or lon/lat
columns.')
```



```

    gx, gy = transformer_to_utm.transform(
        to_numeric_series(grids, lon_col, 'grid longitude').to_numpy(),
        to_numeric_series(grids, lat_col, 'grid latitude').to_numpy()
    )
    grids['x_utm'] = gx
    grids['y_utm'] = gy
else:
    grids['x_utm'] = to_numeric_series(grids, x_col, 'grid UTM x')
    grids['y_utm'] = to_numeric_series(grids, y_col, 'grid UTM y')

if lon_col is None or lat_col is None:
    glon, glat = transformer_to_wgs.transform(grids['x_utm'].to_numpy(float),
        grids['y_utm'].to_numpy(float))
    grids['lon'] = glon
    grids['lat'] = glat
else:
    grids['lon'] = to_numeric_series(grids, lon_col, 'grid longitude')
    grids['lat'] = to_numeric_series(grids, lat_col, 'grid latitude')

grids['grid_id_final'] = pd.to_numeric(grids[grid_id_col], errors='coerce')
if grids['grid_id_final'].isna().any():
    raise ValueError(f"Column {grid_id_col!r} contains non-numeric grid IDs.")
grids['grid_id_final'] = grids['grid_id_final'].astype(int)

fx, fy = transformer_to_utm.transform(links['from_lon'].to_numpy(float),
    links['from_lat'].to_numpy(float))
tx, ty = transformer_to_utm.transform(links['to_lon'].to_numpy(float),
    links['to_lat'].to_numpy(float))
links['from_x_utm'] = fx
links['from_y_utm'] = fy
links['to_x_utm'] = tx
links['to_y_utm'] = ty
links['mid_x_utm'] = (links['from_x_utm'] + links['to_x_utm']) / 2
links['mid_y_utm'] = (links['from_y_utm'] + links['to_y_utm']) / 2

# =====
# POPULATION PER GRID FROM NEW FORMAT
# =====

```

```

if pop_col is not None:
    grids['grid_population_used'] = to_numeric_series(grids, pop_col, 'grid
population')
    population_source = f"direct column: {pop_col}"
else:
    # Fallback path is intentionally explicit and should normally not be used for the
    new format.
    if area_col is not None:
        grids['grid_area_km2_used'] = to_numeric_series(grids, area_col, 'grid area
km2')
    elif size_col is not None:
        cell_size_values = to_numeric_series(grids, size_col, 'grid size m')
        grids['grid_area_km2_used'] = (cell_size_values ** 2) / 1_000_000
    else:
        grids['grid_area_km2_used'] = (DEFAULT_CELL_SIZE_M ** 2) / 1_000_000

    if density_col is not None:
        density_values = to_numeric_series(grids, density_col, 'grid population
density')
    else:
        density_values = DEFAULT_DENSITY_PEOPLE_PER_KM2

    grids['grid_population_used'] = density_values * grids['grid_area_km2_used']
    population_source = "fallback calculation: density × grid_area_km2"

if (grids['grid_population_used'] < 0).any():
    raise ValueError('grid_population contains negative values. Please check the grid
file.')

# Keep these fields for traceability in outputs.
if size_col is not None:
    grids['grid_size_m_used'] = to_numeric_series(grids, size_col, 'grid size m')
else:
    grids['grid_size_m_used'] = DEFAULT_CELL_SIZE_M

if area_col is not None:
    grids['grid_area_km2_used'] = to_numeric_series(grids, area_col, 'grid area
km2')

```

```

else:
    # Fallback only if grid_area_km2 is missing.
    # For full cells: area = grid_size_m × grid_size_m.
    grids['grid_area_km2_used'] = (grids['grid_size_m_used'] ** 2) / 1_000_000

if density_col is not None:
    grids['grid_population_density_used'] = to_numeric_series(grids, density_col,
'grid population density')
else:
    grids['grid_population_density_used'] = np.nan

print("Grid population source:", population_source)
print(f"Total grid population : {grids['grid_population_used'].sum():,.6f} people")
print(f"Mean grid population   : {grids['grid_population_used'].mean():,.6f}
people/grid")
print(f"Min/Max grid pop.      : {grids['grid_population_used'].min():,.6f} /
{grids['grid_population_used'].max():,.6f}")

# =====
# FORWARD ASSIGNMENT
# =====
forward =
links[links['link_type'].eq('Forward')].copy().reset_index().rename(columns={'index':
'original_index'})
if len(forward) == 0:
    raise ValueError('No Forward links found. Cannot assign grids.')

A_x = forward['from_x_utm'].to_numpy(float)
A_y = forward['from_y_utm'].to_numpy(float)
B_x = forward['to_x_utm'].to_numpy(float)
B_y = forward['to_y_utm'].to_numpy(float)
M_x = forward['mid_x_utm'].to_numpy(float)
M_y = forward['mid_y_utm'].to_numpy(float)

radius_m = CATCHMENT_RADIUS_KM * 1000
assign_records = []

for start in range(0, len(grids), CHUNK_SIZE):

```

```

end = min(start + CHUNK_SIZE, len(grids))
gchunk = grids.iloc[start:end]
px = gchunk['x_utm'].to_numpy(float)
py = gchunk['y_utm'].to_numpy(float)

sg_m = point_segment_dist_matrix(px, py, A_x, A_y, B_x, B_y)
mid_m = point_midpoint_dist_matrix(px, py, M_x, M_y)

if CANDIDATE_RULE == 'line':
    eligible = sg_m <= radius_m
elif CANDIDATE_RULE == 'midpoint':
    eligible = mid_m <= radius_m
elif CANDIDATE_RULE == 'either':
    eligible = (sg_m <= radius_m) | (mid_m <= radius_m)
else:
    raise ValueError("CANDIDATE_RULE must be 'line', 'midpoint', or 'either'.")

candidate_count = eligible.sum(axis=1)
masked_sg = np.where(eligible, sg_m, np.inf)
best_pos = np.argmin(masked_sg, axis=1)
best_sg_m = masked_sg[np.arange(len(gchunk)), best_pos]
best_mid_m = mid_m[np.arange(len(gchunk)), best_pos]

selected_prob = np.full(len(gchunk), np.nan)
for r in range(len(gchunk)):
    if candidate_count[r] > 0:
        valid_d_km = sg_m[r, eligible[r]] / 1000.0
        d_min = best_sg_m[r] / 1000.0
        # Probability of the selected nearest link under the logit expression.
        # Assignment remains deterministic: Fg,ij = 1 for the selected link only.
        selected_prob[r] = 1.0 / np.exp(-(valid_d_km - d_min)).sum()

has_assignment = candidate_count > 0
for local_idx, (_, grow) in enumerate(gchunk.iterrows()):
    grid_pop = float(grow['grid_population_used'])
    if has_assignment[local_idx]:
        fp = int(best_pos[local_idx])
        frow = forward.iloc[fp]

```

```

        assign_records.append({
            'grid_id': int(grow['grid_id_final']),
            'grid_lon': float(grow['lon']),
            'grid_lat': float(grow['lat']),
            'grid_x_utm': float(grow['x_utm']),
            'grid_y_utm': float(grow['y_utm']),
            'grid_size_m': float(grow['grid_size_m_used']) if
pd.notna(grow['grid_size_m_used']) else np.nan,
            'grid_area_km2': float(grow['grid_area_km2_used']) if
pd.notna(grow['grid_area_km2_used']) else np.nan,
            'grid_population_density':
float(grow['grid_population_density_used']) if
pd.notna(grow['grid_population_density_used']) else np.nan,
            'grid_population': grid_pop,
            'Fgij': 1,
            'Popgij': grid_pop,
            'assigned': 1,
            'candidate_forward_link_count': int(candidate_count[local_idx]),
            'assigned_forward_original_index': int(frow['original_index']),
            'assigned_forward_link_name': frow['link_name'],
            'assigned_forward_from_station_id': int(frow['from_station_id']),
            'assigned_forward_to_station_id': int(frow['to_station_id']),
            'Sgij_m': float(best_sg_m[local_idx]),
            'Sgij_km': float(best_sg_m[local_idx] / 1000.0),
            'grid_to_link_midpoint_m': float(best_mid_m[local_idx]),
            'grid_to_link_midpoint_km': float(best_mid_m[local_idx] /
1000.0),
            'logit_probability_selected': float(selected_prob[local_idx]),
            'catchment_rule': CANDIDATE_RULE,
            'catchment_radius_km': CATCHMENT_RADIUS_KM,
        })
    else:
        assign_records.append({
            'grid_id': int(grow['grid_id_final']),
            'grid_lon': float(grow['lon']),
            'grid_lat': float(grow['lat']),
            'grid_x_utm': float(grow['x_utm']),
            'grid_y_utm': float(grow['y_utm']),

```

```

        'grid_size_m': float(grow['grid_size_m_used']) if
pd.notna(grow['grid_size_m_used']) else np.nan,
        'grid_area_km2': float(grow['grid_area_km2_used']) if
pd.notna(grow['grid_area_km2_used']) else np.nan,
        'grid_population_density':
float(grow['grid_population_density_used']) if
pd.notna(grow['grid_population_density_used']) else np.nan,
        'grid_population': grid_pop,
        'Fgij': 0,
        'Popgij': 0.0,
        'assigned': 0,
        'candidate_forward_link_count': 0,
        'assigned_forward_original_index': -1,
        'assigned_forward_link_name': None,
        'assigned_forward_from_station_id': np.nan,
        'assigned_forward_to_station_id': np.nan,
        'Sgij_m': np.nan,
        'Sgij_km': np.nan,
        'grid_to_link_midpoint_m': np.nan,
        'grid_to_link_midpoint_km': np.nan,
        'logit_probability_selected': np.nan,
        'catchment_rule': CANDIDATE_RULE,
        'catchment_radius_km': CATCHMENT_RADIUS_KM,
    })

assignments = pd.DataFrame(assign_records)
print("Grid assignment completed.")
print("Assigned grids   :", int(assignments['assigned'].sum()))
print("Unassigned grids:", int((assignments['assigned'] == 0).sum()))
print("Assigned share   :", assignments['assigned'].mean())
print(f"Assigned population from Popgij: {assignments['Popgij'].sum():.6f} people")

# =====
# AGGREGATE FORWARD POPij and Sij
# =====
assigned_only = assignments[assignments['assigned'].eq(1)].copy()
if len(assigned_only) > 0:
    assigned_only['Sij_component_m'] = assigned_only['Sgij_m'] *

```

```

assigned_only['Popgij']
    forward_agg = (
        assigned_only.groupby('assigned_forward_original_index')
        .agg(
            assigned_grid_count=('grid_id', 'count'),
            assigned_grid_area_km2=('grid_area_km2', 'sum'),
            Popij=('Popgij', 'sum'),
            assigned_grid_population_total=('grid_population', 'sum'),
            Sij_weighted_sum_m=('Sij_component_m', 'sum'),
            mean_assignment_probability=('logit_probability_selected', 'mean'),
            mean_candidate_count=('candidate_forward_link_count', 'mean'),
            min_Sgij_m=('Sgij_m', 'min'),
            max_Sgij_m=('Sgij_m', 'max'),
        )
        .reset_index()
        .rename(columns={'assigned_forward_original_index':
'original_index'})
    )
    forward_agg['Sij_m'] = np.where(
        forward_agg['Popij'] > 0,
        forward_agg['Sij_weighted_sum_m'] / forward_agg['Popij'],
        0.0
    )
    forward_agg['Sij_km'] = forward_agg['Sij_m'] / 1000.0
    forward_agg['assigned_grid_area_m2'] =
forward_agg['assigned_grid_area_km2'] * 1_000_000
else:
    forward_agg = pd.DataFrame(columns=[
        'original_index', 'assigned_grid_count', 'Popij',
'assigned_grid_population_total',
        'Sij_weighted_sum_m', 'Sij_m', 'Sij_km', 'mean_assignment_probability',
        'mean_candidate_count', 'min_Sgij_m', 'max_Sgij_m'
    ])

# =====
# APPLY LINK TYPE RULES
# =====
result = links.copy().reset_index().rename(columns={'index': 'original_index'})

```

```

result = result.merge(forward_agg.drop(columns=['Sij_weighted_sum_m'],
errors='ignore'), on='original_index', how='left')

numeric_cols = [
    'assigned_grid_count',
    'assigned_grid_area_km2',
    'assigned_grid_area_m2',
    'Popij',
    'assigned_grid_population_total',
    'Sij_m',
    'Sij_km',
    'mean_assignment_probability',
    'mean_candidate_count',
    'min_Sgij_m',
    'max_Sgij_m'
]
for c in numeric_cols:
    if c not in result.columns:
        result[c] = 0
    result[c] = result[c].fillna(0)

result['assignment_rule'] = np.select(
    [result['link_type'].eq('Forward'), result['link_type'].eq('Backward'),
    result['link_type'].eq('U-turn')],
    ['Direct hard assignment from eligible grid centroids using grid_population',
    'Replicate nearest Forward link assignment by midpoint distance',
    'Zero grid assignment; routing only'],
    default='Unknown'
)
result['assignment_source_link_name'] = np.where(result['link_type'].eq('Forward'),
result['link_name'], "")
result['assignment_source_original_index'] =
np.where(result['link_type'].eq('Forward'), result['original_index'], np.nan)
result['mapping_midpoint_distance_m'] = np.nan

# Backward replication
backward = result[result['link_type'].eq('Backward')].copy()
forward_result = result[result['link_type'].eq('Forward')].copy()

```



```

map_records = []
if len(backward) > 0 and len(forward_result) > 0:
    b_mx = backward['mid_x_utm'].to_numpy(float)[: , None]
    b_my = backward['mid_y_utm'].to_numpy(float)[: , None]
    f_mx = forward_result['mid_x_utm'].to_numpy(float)[None, :]
    f_my = forward_result['mid_y_utm'].to_numpy(float)[None, :]
    d_mid = np.sqrt((b_mx - f_mx) ** 2 + (b_my - f_my) ** 2)
    nearest_pos = np.argmin(d_mid, axis=1)

    for row_i, (_, brow) in enumerate(backward.iterrows()):
        frow = forward_result.iloc[int(nearest_pos[row_i])]
        dist_m = float(d_mid[row_i, int(nearest_pos[row_i])])
        b_idx = result['original_index'].eq(brow['original_index'])

        for col in numeric_cols:
            result.loc[b_idx, col] = frow[col]
        result.loc[b_idx, 'assignment_source_link_name'] = frow['link_name']
        result.loc[b_idx, 'assignment_source_original_index'] =
frow['original_index']
        result.loc[b_idx, 'mapping_midpoint_distance_m'] = dist_m

    map_records.append({
        'backward_original_index': int(brow['original_index']),
        'backward_link_name': brow['link_name'],
        'backward_from_station_id': int(brow['from_station_id']),
        'backward_to_station_id': int(brow['to_station_id']),
        'nearest_forward_original_index': int(frow['original_index']),
        'nearest_forward_link_name': frow['link_name'],
        'nearest_forward_from_station_id': int(frow['from_station_id']),
        'nearest_forward_to_station_id': int(frow['to_station_id']),
        'midpoint_distance_m': dist_m,
        'copied_assigned_grid_count': int(frow['assigned_grid_count']),
        'copied_Popij': float(frow['Popij']),
        'copied_Sij_m': float(frow['Sij_m']),
        'copied_Sij_km': float(frow['Sij_km']),
    })
backward_mapping = pd.DataFrame(map_records)

```

```

# U-turn force zero assignment
uturn_mask = result['link_type'].eq('U-turn')
for c in numeric_cols:
    result.loc[uturn_mask, c] = 0
result.loc[uturn_mask, 'assignment_source_link_name'] = ''
result.loc[uturn_mask, 'assignment_source_original_index'] = np.nan
result.loc[uturn_mask, 'mapping_midpoint_distance_m'] = np.nan

result['unique_population_counted'] = result['link_type'].eq('Forward')
result['final_link_population_note'] = np.select(
    [result['link_type'].eq('Forward'), result['link_type'].eq('Backward'),
    result['link_type'].eq('U-turn')],
    ['Unique population assignment based on direct grid_population',
    'Replicated from nearest Forward link; do not add to unique coverage total',
    'No population assignment'],
    default='Unknown'
)

# =====
# VALIDATION SUMMARY
# =====
unique_forward_assigned_population = result.loc[result['link_type'].eq('Forward'),
'Popij'].sum()
all_grid_population = grids['grid_population_used'].sum()
all_grid_area_km2 = grids['grid_area_km2_used'].sum()
unique_forward_assigned_area_km2 = result.loc[result['link_type'].eq('Forward'),
'assigned_grid_area_km2'].sum()
assigned_grid_count = int(assignments['assigned'].sum())
unassigned_grid_count = int((assignments['assigned'] == 0).sum())
summary = pd.DataFrame([
    {'metric': 'link_file', 'value': LINK_PATH.name},
    {'metric': 'grid_file', 'value': GRID_PATH.name},
    {'metric': 'grid_population_source', 'value': population_source},
    {'metric': 'link_rows', 'value': len(links)},
    {'metric': 'grid_rows', 'value': len(grids)},
    {'metric': 'catchment_radius_km', 'value': CATCHMENT_RADIUS_KM},
    {'metric': 'candidate_rule', 'value': CANDIDATE_RULE},
    {'metric': 'forward_links', 'value': int((links['link_type'] == 'Forward').sum())},

```

```

        {'metric': 'backward_links', 'value': int((links['link_type'] == 'Backward').sum())},
        {'metric': 'uturn_links', 'value': int((links['link_type'] == 'U-turn').sum())},
        {'metric': 'assigned_grid_count', 'value': assigned_grid_count},
        {'metric': 'unassigned_grid_count', 'value': unassigned_grid_count},
        {'metric': 'assigned_grid_share', 'value': assigned_grid_count / len(grids)},
        {'metric': 'total_grid_population_from_input', 'value': all_grid_population},
        {'metric': 'unique_forward_assigned_population', 'value':
unique_forward_assigned_population},
        {'metric': 'unassigned_population', 'value': all_grid_population -
unique_forward_assigned_population},
        {'metric': 'assigned_population_share', 'value':
unique_forward_assigned_population / all_grid_population if all_grid_population
else np.nan},
        {'metric': 'final_link_level_population_sum_with_backward_replication', 'value':
result['Popij'].sum()}},
        {'metric': 'total_grid_area_km2_from_input', 'value': all_grid_area_km2},
        {'metric': 'unique_forward_assigned_area_km2', 'value':
unique_forward_assigned_area_km2},
        {'metric': 'unassigned_grid_area_km2', 'value': all_grid_area_km2 -
unique_forward_assigned_area_km2},
        {'metric': 'assigned_area_share', 'value': unique_forward_assigned_area_km2 /
all_grid_area_km2 if all_grid_area_km2 else np.nan},
        {'metric': 'final_link_level_area_sum_with_backward_replication', 'value':
result['assigned_grid_area_km2'].sum()}},
    ])

# =====
# EXPORT OUTPUTS
# =====
preferred_cols = [
    'original_index', 'link_name', 'link_type',
    'from_station_id', 'to_station_id',
    'from_lat', 'from_lon', 'to_lat', 'to_lon',
    'from_station_name', 'to_station_name',
    'gis_route_distance_m', 'gis_route_distance_km',
    'average_travel_time_s', 'average_travel_time_min', 'average_speed_kmh',
    'assigned_grid_count',
    'assigned_grid_area_km2',

```

```

'assigned_grid_area_m2',
'Popij',
'assigned_grid_population_total',
'Sij_m',
'Sij_km',
    'assignment_rule', 'assignment_source_link_name',
'assignment_source_original_index',
    'mapping_midpoint_distance_m',
    'unique_population_counted', 'final_link_population_note',
    'mean_assignment_probability', 'mean_candidate_count', 'min_Sgij_m',
'max_Sgij_m'
]
existing_preferred = [c for c in preferred_cols if c in result.columns]
remaining = [c for c in result.columns if c not in existing_preferred]
result_export = result[existing_preferred + remaining].copy()

main_csv = OUT_DIR / 'vnu_link_popij_sij_linktype_gridpop_catchment_950m.csv'
grid_csv = OUT_DIR / 'vnu_grid_assignment_forward_gridpop_catchment_950m.csv'
map_csv = OUT_DIR /
'vnu_backward_to_forward_mapping_gridpop_catchment_950m.csv'
summary_csv = OUT_DIR / 'vnu_popij_sij_assignment_summary_gridpop_950m.csv'
excel_file = OUT_DIR / 'vnu_popij_sij_linktype_gridpop_catchment_950m.xlsx'

result_export.to_csv(main_csv, index=False)
assignments.to_csv(grid_csv, index=False)
backward_mapping.to_csv(map_csv, index=False)
summary.to_csv(summary_csv, index=False)

with pd.ExcelWriter(excel_file, engine='openpyxl') as writer:
    result_export.to_excel(writer, sheet_name='link_popij_sij', index=False)
    assignments.to_excel(writer, sheet_name='grid_assignments', index=False)
    backward_mapping.to_excel(writer, sheet_name='backward_mapping',
index=False)
    summary.to_excel(writer, sheet_name='summary', index=False)

print("\n===== VALIDATION SUMMARY =====")
print(summary.to_string(index=False))
print("\n===== SAVED OUTPUTS =====")

```

```
print(main_csv)
print(grid_csv)
print(map_csv)
print(summary_csv)
print(excel_file)
```

Appendix D

PULSE ALGORITHM

```
# =====  
# Pulse Algorithm for Feeder Bus Route Optimization  
# Case: VNU-HCM first-mile feeder route to Metro Station National University  
#  
# Research basis:  
# - Hemdan et al. (2026): feeder bus design as a constrained route problem  
#   using potential demand, priority index, and Pulse Algorithm logic.  
# - Lozano & Medaglia (2013): Pulse Algorithm for Constrained Shortest Path.  
# - Zhu et al. (2017): circular feeder-bus route structure and potential demand  
#   logic based on demand, passenger access distance, and rail-station distance.  
#  
# Objective:  
#   Maximize Total Priority Index (PI)  
#  
# Main constraints:  
#   1. Route starts and ends at START_STATION_ID.  
#   2. Total route travel time <= TMAX_MINUTES.  
#   3. The final route cannot contain both an original forward demand link  
#       and its backward alternative.  
#   4. Each original forward demand link can be represented by either its  
#       forward or backward direction, but not both.  
#   5. Route is treated as a simple circular route: no repeated station  
#       except returning to the start station at the final step.  
#  
# Outputs:  
#   1. Total_route_travel_time  
#   2. Total PI  
#   3. Total Potential Demand  
#   4. SAp  
#   5. Program Running Time  
#   6. Route_map files for Google My Maps  
# =====
```

```
import os  
import math
```

```

import time
import json
import heapq
import html
import zipfile
import xml.sax.saxutils as saxutils
from pathlib import Path
from collections import defaultdict

import numpy as np
import pandas as pd

try:
    import folium
except Exception:
    folium = None

# =====
# 1. USER SETTINGS
# =====
# In Google Colab, upload INPUT DATA.csv to /content, or keep it in your runtime.
INPUT_FILE = "/content/INPUT DATA.csv"

# Local fallback for this generated notebook/script.
LOCAL_FALLBACK_INPUT_FILE = "/mnt/data/INPUT DATA.csv"

# Main route settings.
START_STATION_ID = 31

# Manually change this value when you want another maximum route time.
# Example: 15.0, 18.0, 20.0, 25.0
TMAX_MINUTES = 18

# Search controls.
# For your current network, the Pulse search is usually fast. These limits are
# kept as safeguards for Colab. If the result says SEARCH_LIMIT_REACHED, increase
# them.
MAX_RUNTIME_SECONDS = 360

```

```
MAX_EXPANSIONS = 20_000_000
```

```
# Output folder.
```

```
OUTPUT_DIR = "/content/pulse_feeder_bus_outputs"
```

```
LOCAL_FALLBACK_OUTPUT_DIR = "/mnt/data/pulse_feeder_bus_outputs"
```

```
# Earth radius for haversine distance.
```

```
EARTH_RADIUS_M = 6_371_000.0
```

```
# =====
```

```
# 2. PATH HANDLING
```

```
# =====
```

```
def resolve_input_path():
```

```
    """Resolve INPUT DATA.csv in Colab or local execution."""
```

```
    if os.path.exists(INPUT_FILE):
```

```
        return INPUT_FILE
```

```
    if os.path.exists(LOCAL_FALLBACK_INPUT_FILE):
```

```
        return LOCAL_FALLBACK_INPUT_FILE
```

```
    try:
```

```
        from google.colab import files
```

```
        print("INPUT DATA.csv was not found. Please upload INPUT DATA.csv.")
```

```
        uploaded = files.upload()
```

```
        if "INPUT DATA.csv" in uploaded:
```

```
            return "/content/INPUT DATA.csv"
```

```
    except Exception:
```

```
        pass
```

```
    raise FileNotFoundError(
```

```
        "INPUT DATA.csv was not found. Upload it to Colab or update INPUT_FILE."
```

```
    )
```

```
def resolve_output_dir():
```

```
    """Resolve output directory for Colab or local execution."""
```

```
    if os.path.exists("/content"):
```

```
        out = Path(OUTPUT_DIR)
```



```

else:
    out = Path(LOCAL_FALLBACK_OUTPUT_DIR)
    out.mkdir(parents=True, exist_ok=True)
    return out

```

```

# =====

```

3. BASIC GEOSPATIAL FUNCTION

```

# =====

```

```

def haversine_m(lat1, lon1, lat2, lon2):
    """Great-circle distance in meters."""
    phi1 = math.radians(float(lat1))
    phi2 = math.radians(float(lat2))
    dphi = math.radians(float(lat2) - float(lat1))
    dlamba = math.radians(float(lon2) - float(lon1))

    a = (
        math.sin(dphi / 2.0) ** 2
        + math.cos(phi1) * math.cos(phi2) * math.sin(dlamba / 2.0) ** 2
    )
    a = max(0.0, min(1.0, a))
    return 2.0 * EARTH_RADIUS_M * math.asin(math.sqrt(a))

```

```

# =====

```

4. DATA PREPARATION: PD, PI, DEMAND KEY, AND SAp BASIS

```

# =====

```

```

def load_and_prepare_input(input_path, start_station_id):

```

```

    """

```

```

    Prepare the input network.

```

```

    Main calculation:

```

```

        Potential Demand:

```

```

             $PD_{ij} = Pop_{ij} / S_{ij} * L_{ij}$ 

```

```

        Lij:

```

```

            straight-line distance from the midpoint of link (i,j)
            to the target metro station.

```

PI_{ij}:

rank of positive PD_{ij} among all unique demand links.

Higher PD receives higher PI.

Forward/backward rule:

- Forward link: demand_key = link_name.
- Backward link: demand_key = forward_source_link_name.
- U-turn link: PI = 0 and used only as routing connector.

"""

```
raw = pd.read_csv(input_path)
```

```
required_columns = [  
    "link_name",  
    "link_type",  
    "from_station_id",  
    "to_station_id",  
    "from_lat",  
    "from_lon",  
    "to_lat",  
    "to_lon",  
    "from_station_name",  
    "to_station_name",  
    "gis_route_distance_m",  
    "average_travel_time_s",  
    "assigned_grid_area_km2",  
    "assigned_grid_area_m2",  
    "Popij",  
    "Sij_m",  
    "forward_source_link_name",  
]
```

```
missing = [c for c in required_columns if c not in raw.columns]
```

```
if missing:
```

```
    raise ValueError(f"Missing required columns in INPUT DATA.csv: {missing}")
```

```
df = raw.copy()
```

```

# Standardize fields.
df["link_name"] = df["link_name"].astype(str).str.strip()
df["link_type"] = df["link_type"].astype(str).str.strip()

integer_columns = ["from_station_id", "to_station_id"]
for c in integer_columns:
    df[c] = pd.to_numeric(df[c], errors="coerce").astype(int)

numeric_columns = [
    "from_lat",
    "from_lon",
    "to_lat",
    "to_lon",
    "gis_route_distance_m",
    "average_travel_time_s",
    "average_speed_kmh",
    "assigned_grid_area_km2",
    "assigned_grid_area_m2",
    "Popij",
    "Sij_m",
]

for c in numeric_columns:
    if c in df.columns:
        df[c] = pd.to_numeric(df[c], errors="coerce").fillna(0.0)

# Build station table.
station_records = []
for _, r in df.iterrows():
    station_records.append(
        {
            "station_id": int(r["from_station_id"]),
            "latitude": float(r["from_lat"]),
            "longitude": float(r["from_lon"]),
            "station_name": str(r["from_station_name"]),
        }
    )
    station_records.append(

```

```

        {
            "station_id": int(r["to_station_id"]),
            "latitude": float(r["to_lat"]),
            "longitude": float(r["to_lon"]),
            "station_name": str(r["to_station_name"]),
        }
    )

stations = (
    pd.DataFrame(station_records)
    .drop_duplicates("station_id")
    .sort_values("station_id")
    .reset_index(drop=True)
)

if start_station_id not in set(stations["station_id"]):
    raise ValueError(f"START_STATION_ID={start_station_id} is not found in the
station table.")

start_row = stations.loc[stations["station_id"].eq(start_station_id)].iloc[0]
start_lat = float(start_row["latitude"])
start_lon = float(start_row["longitude"])

# Link midpoint and station-distance factor Lij.
df["mid_lat"] = (df["from_lat"] + df["to_lat"]) / 2.0
df["mid_lon"] = (df["from_lon"] + df["to_lon"]) / 2.0

df["Lij_to_station_m"] = [
    haversine_m(mlat, mlon, start_lat, start_lon)
    for mlat, mlon in zip(df["mid_lat"], df["mid_lon"])
]

# Base potential demand per row.
is_urn = df["link_type"].str.lower().eq("u-turn")

valid_pd = (~is_urn) & (df["Popij"] > 0) & (df["Sij_m"] > 0)
df["PD_base"] = 0.0
df.loc[valid_pd, "PD_base"] = (

```

```

        df.loc[valid_pd, "Popij"]
        / df.loc[valid_pd, "Sij_m"]
        * df.loc[valid_pd, "Lij_to_station_m"]
    )

    # Demand-key definition.
    df["demand_key"] = df["link_name"]
    df["metric_source_link_name"] = df["link_name"]

    source_col = df["forward_source_link_name"].fillna("").astype(str).str.strip()
    is_backward = df["link_type"].str.lower().eq("backward") & source_col.ne("")

    df.loc[is_backward, "demand_key"] = source_col[is_backward]
    df.loc[is_backward, "metric_source_link_name"] = source_col[is_backward]

    df.loc[is_urn, "demand_key"] = "UTURN_" + df.loc[is_urn,
"link_name"].astype(str)
    df.loc[is_urn, "metric_source_link_name"] = df.loc[is_urn,
"link_name"].astype(str)

    # Backward links inherit the source forward-link demand metrics.
    source_df = df.set_index("link_name", drop=False)

    inherited_metrics = []
    for _, r in df.iterrows():
        link_type = str(r["link_type"]).lower()
        source_name = str(r["metric_source_link_name"])

        if source_name in source_df.index:
            source_row = source_df.loc[source_name]
            if isinstance(source_row, pd.DataFrame):
                source_row = source_row.iloc[0]
        else:
            source_row = r

        if link_type == "u-turn":
            inherited_metrics.append(
                {

```

```

        "PDij": 0.0,
        "Pop_key": 0.0,
        "Area_key_km2": 0.0,
        "Area_key_m2": 0.0,
    }
)
else:
    inherited_metrics.append(
        {
            "PDij": float(source_row["PD_base"]),
            "Pop_key": float(source_row["Popij"]),
            "Area_key_km2":
float(source_row["assigned_grid_area_km2"]),
            "Area_key_m2":
float(source_row["assigned_grid_area_m2"]),
        }
    )

df = pd.concat([df, pd.DataFrame(inherited_metrics)], axis=1)

# PI ranking by unique demand key.
non_urn_unique = (
    df[~is_urn]
    .drop_duplicates("demand_key")["demand_key", "PDij"]
    .copy()
)

positive_keys = (
    non_urn_unique[non_urn_unique["PDij"] > 0]
    .sort_values(["PDij", "demand_key"], ascending=[True, True])
    .reset_index(drop=True)
)

positive_keys["Plij"] = np.arange(1, len(positive_keys) + 1, dtype=int)

df = df.merge(positive_keys[["demand_key", "Plij"]], on="demand_key",
how="left")
df["Plij"] = df["Plij"].fillna(0).astype(int)

```

```

# U-turn rows have zero demand contribution.
is_urn = df["link_type"].str.lower().eq("u-turn")
df.loc[is_urn, ["PDij", "Pop_key", "Area_key_km2", "Area_key_m2", "Plij"]] = 0

# Edge id.
df = df.reset_index(drop=True)
df["edge_id"] = df.index.astype(int)

return df, stations

# =====
# 5. FEEDER GRAPH CLASS
# =====

class FeederGraph:
    def __init__(self, edges_df, stations_df, start_station_id):
        self.edges_df = edges_df.copy()
        self.stations_df = stations_df.copy()
        self.start = int(start_station_id)

        self.adj = defaultdict(list)
        self.rev = defaultdict(list)
        self.edge_by_id = {}
        self.nodes = set()

    for _, r in self.edges_df.iterrows():
        e = {
            "edge_id": int(r["edge_id"]),
            "link_name": str(r["link_name"]),
            "link_type": str(r["link_type"]),
            "u": int(r["from_station_id"]),
            "v": int(r["to_station_id"]),
            "from_station_name": str(r["from_station_name"]),
            "to_station_name": str(r["to_station_name"]),
            "from_lat": float(r["from_lat"]),
            "from_lon": float(r["from_lon"]),
            "to_lat": float(r["to_lat"]),
            "to_lon": float(r["to_lon"]),

```

```

        "time_s": float(r["average_travel_time_s"]),
        "distance_m": float(r["gis_route_distance_m"]),
        "PI": int(r["Plij"]),
        "PD": float(r["PDij"]),
        "Pop": float(r["Pop_key"]),
        "Area_km2": float(r["Area_key_km2"]),
        "Area_m2": float(r["Area_key_m2"]),
        "demand_key": str(r["demand_key"]),
    }

```

```

self.adj[e["u"]].append(e)
self.rev[e["v"]].append(e)
self.edge_by_id[e["edge_id"]] = e
self.nodes.add(e["u"])
self.nodes.add(e["v"])

```

Explore high-value links earlier. This does not change exactness.

for u in self.adj:

```

    self.adj[u].sort(
        key=lambda x: (
            x["PI"] / max(x["time_s"], 1e-9),
            x["PI"],
            -x["time_s"],
        ),
        reverse=True,
    )

```

Unique demand-key totals for fair coverage denominator.

```

unique_keys = (
    self.edges_df[self.edges_df["Plij"] > 0]
    .drop_duplicates("demand_key")
    .copy()
)

```

```

self.total_available_PI = int(unique_keys["Plij"].sum())
self.total_available_PD = float(unique_keys["PDij"].sum())
self.total_available_population = float(unique_keys["Pop_key"].sum())
self.total_available_area_km2 = float(unique_keys["Area_key_km2"].sum())

```



```

self.total_available_area_m2 = float(unique_keys["Area_key_m2"].sum())
self.n_unique_positive_demand_links =
int(unique_keys["demand_key"].nunique())

```

```

self.min_time_to_start = self.compute_min_time_to_start()

```

```

def compute_min_time_to_start(self):

```

```

    """

```

```

    Reverse Dijkstra from the start station.

```

```

    min_time_to_start[n] is the shortest travel time from node n back to start.

```

```

    Used for Pulse infeasibility pruning.

```

```

    """

```

```

    dist = {n: math.inf for n in self.nodes}

```

```

    dist[self.start] = 0.0

```

```

    pq = [(0.0, self.start)]

```

```

    while pq:

```

```

        d, node = heapq.heappop(pq)

```

```

        if d > dist[node] + 1e-9:

```

```

            continue

```

```

        for e in self.rev.get(node, []):

```

```

            nd = d + e["time_s"]

```

```

            if nd < dist[e["u"]] - 1e-9:

```

```

                dist[e["u"]] = nd

```

```

                heapq.heappush(pq, (nd, e["u"]))

```

```

    return dist

```

```

def route_nodes(self, route):

```

```

    if not route:

```

```

        return []

```

```

    nodes = [route[0]["u"]]

```

```

    nodes.extend(e["v"] for e in route)

```

```

    return nodes

```

```

def is_feasible(self, route, tmax_s, enforce_simple_route=True):

```

```

    if not route:

```

```

        return False, "empty_route"

    if route[0]["u"] != self.start:
        return False, "does_not_start_at_target_station"

    if route[-1]["v"] != self.start:
        return False, "does_not_end_at_target_station"

    elapsed = 0.0
    previous_node = self.start
    used_edges = set()
    used_demand_keys = set()
    visited_nodes = {self.start}

    for idx, e in enumerate(route):
        if e["u"] != previous_node:
            return False, "route_not_continuous"

        elapsed += e["time_s"]
        if elapsed > tmax_s + 1e-9:
            return False, "travel_time_exceeds_Tmax"

        if e["edge_id"] in used_edges:
            return False, "repeated_directed_edge"
        used_edges.add(e["edge_id"])

        if e["PI"] > 0:
            if e["demand_key"] in used_demand_keys:
                return False,
"forward_backward_or_duplicate_demand_key"
            used_demand_keys.add(e["demand_key"])

        next_node = e["v"]

    if enforce_simple_route:
        if next_node == self.start:
            if idx != len(route) - 1:
                return False, "returned_to_start_before_final_step"

```

```

        else:
            if next_node in visited_nodes:
                return False, "repeated_station"
            visited_nodes.add(next_node)

        previous_node = next_node

    return True, "OK"

def metrics(self, route):
    used_keys = set()

    total_pi = 0
    total_pd = 0.0
    total_pop = 0.0
    total_area_km2 = 0.0
    total_area_m2 = 0.0

    for e in route:
        if e["PI"] <= 0:
            continue

        key = e["demand_key"]
        if key in used_keys:
            continue

        used_keys.add(key)
        total_pi += e["PI"]
        total_pd += e["PD"]
        total_pop += e["Pop"]
        total_area_km2 += e["Area_km2"]
        total_area_m2 += e["Area_m2"]

    total_time_s = sum(e["time_s"] for e in route)
    total_distance_m = sum(e["distance_m"] for e in route)

    sap_percent = (
        total_area_km2 / self.total_available_area_km2 * 100.0

```

```

        if self.total_available_area_km2 > 0
        else 0.0
    )

    return {
        "Total_route_travel_time_s": total_time_s,
        "Total_route_travel_time_min": total_time_s / 60.0,
        "Total_route_distance_m": total_distance_m,
        "Total_PI": int(total_pi),
        "Total_Potential_Demand": float(total_pd),
        "Total_Served_Population": float(total_pop),
        "SAp_km2": float(total_area_km2),
        "SAp_m2": float(total_area_m2),
        "SAp_percent": float(sap_percent),
        "Num_route_links": len(route),
        "Num_nodes_including_return": len(self.route_nodes(route)),
        "Route_node_sequence": " -> ".join(map(str, self.route_nodes(route))),
        "Route_link_sequence": " | ".join(e["link_name"] for e in route),
    }

```

```
# =====
```

6. PULSE ALGORITHM

```
# =====
```

```
class PulseSolver:
```

```
    """
```

Pulse-style exact search for the constrained route problem.

The classical CSP minimizes path cost under a resource constraint.

Here, the problem is adapted as:

maximize Total PI

subject to total travel time $\leq$ Tmax

Pulse components implemented:

- DFS pulse propagation from the start node.
- Cycle pruning for simple circular route construction.
- Resource infeasibility pruning using travel time.
- Return-feasibility pruning using shortest time back to start.

- Bound pruning using a fractional upper bound on remaining PI.
- Forward/backward exclusion through demand_key memory.

"""

```
def __init__(
    self,
    graph,
    tmax_minutes,
    max_runtime_seconds=120.0,
    max_expansions=5_000_000,
):
    self.graph = graph
    self.tmax_s = float(tmax_minutes) * 60.0
    self.max_runtime_seconds = float(max_runtime_seconds)
    self.max_expansions = int(max_expansions)

    self.best_route = None
    self.best_metrics = None
    self.best_pi = -1
    self.best_pd = -1.0
    self.best_sap = -1.0
    self.best_time_s = math.inf

    self.expansions = 0
    self.prunes = {
        "time_limit": 0,
        "max_expansions": 0,
        "time_resource": 0,
        "return_infeasible": 0,
        "upper_bound": 0,
        "repeated_station": 0,
        "repeated_edge": 0,
        "duplicate_demand_key": 0,
    }

    self.limit_reached = False
    self.start_time = None
    self.upper_bound_items = self.build_upper_bound_items()
```

```

def build_upper_bound_items(self):
    """
    Build an optimistic fractional-knapsack upper bound.

    One item represents one positive unique demand key.
    Item value = PI
    Item time = minimum travel time of any directed representation of that key.

    The bound ignores connectivity and therefore overestimates possible
    remaining PI.
    That makes it safe for pruning.
    """
    key_best = {}

    for e in self.graph.edge_by_id.values():
        if e["PI"] <= 0:
            continue

        key = e["demand_key"]
        if key not in key_best or e["time_s"] < key_best[key]["time_s"]:
            key_best[key] = {
                "demand_key": key,
                "PI": e["PI"],
                "time_s": max(e["time_s"], 1e-9),
            }

    items = list(key_best.values())
    items.sort(key=lambda x: x["PI"] / x["time_s"], reverse=True)
    return items

def fractional_pi_upper_bound(self, remaining_time_s, used_demand_keys):
    """
    Optimistic upper bound on extra PI that can still be collected.

    The bound is intentionally optimistic because it ignores route continuity.
    It is only used to safely prune branches that cannot beat the incumbent.
    """

```

```

if remaining_time_s <= 0:
    return 0.0

ub = 0.0
rt = float(remaining_time_s)

for item in self.upper_bound_items:
    if item["demand_key"] in used_demand_keys:
        continue

    if item["time_s"] <= rt:
        ub += item["PI"]
        rt -= item["time_s"]
    else:
        ub += item["PI"] * (rt / item["time_s"])
        break

return ub

def incumbent_is_better(self, candidate_route):
    """
    Main ranking:
    1. Higher Total PI
    2. Higher Total Potential Demand
    3. Higher SAp
    4. Lower travel time
    """
    m = self.graph.metrics(candidate_route)

    cand_tuple = (
        m["Total_PI"],
        m["Total_Potential_Demand"],
        m["SAp_percent"],
        -m["Total_route_travel_time_s"],
    )

    best_tuple = (
        self.best_pi,

```

```

        self.best_pd,
        self.best_sap,
        -self.best_time_s,
    )

    return cand_tuple > best_tuple

def update_best(self, route):
    ok, reason = self.graph.is_feasible(route, self.tmax_s)
    if not ok:
        return

    if self.incumbent_is_better(route):
        self.best_route = list(route)
        self.best_metrics = self.graph.metrics(route)
        self.best_pi = self.best_metrics["Total_PI"]
        self.best_pd = self.best_metrics["Total_Potential_Demand"]
        self.best_sap = self.best_metrics["SAp_percent"]
        self.best_time_s = self.best_metrics["Total_route_travel_time_s"]

def check_global_limits(self):
    if self.expansions >= self.max_expansions:
        self.prunes["max_expansions"] += 1
        self.limit_reached = True
        return True

    if time.perf_counter() - self.start_time > self.max_runtime_seconds:
        self.prunes["time_limit"] += 1
        self.limit_reached = True
        return True

    return False

def pulse(self, node, elapsed_s, current_pi, path, visited_nodes,
used_demand_keys, used_edge_ids):
    """
    Recursive pulse propagation.
    """

```



```

self.expansions += 1

if self.check_global_limits():
    return

# Resource and return-feasibility pruning.
if elapsed_s > self.tmax_s + 1e-9:
    self.prunes["time_resource"] += 1
    return

min_return = self.graph.min_time_to_start.get(node, math.inf)
if elapsed_s + min_return > self.tmax_s + 1e-9:
    self.prunes["return_infeasible"] += 1
    return

# Bound pruning: cannot beat the best incumbent PI.
remaining_time = self.tmax_s - elapsed_s
optimistic_pi = current_pi + self.fractional_pi_upper_bound(
    remaining_time, used_demand_keys
)

if optimistic_pi < self.best_pi - 1e-9:
    self.prunes["upper_bound"] += 1
    return

# Propagate through outgoing links.
for e in self.graph.adj.get(node, []):
    edge_id = e["edge_id"]
    next_node = e["v"]
    new_elapsed = elapsed_s + e["time_s"]

    if edge_id in used_edge_ids:
        self.prunes["repeated_edge"] += 1
        continue

    if new_elapsed > self.tmax_s + 1e-9:
        self.prunes["time_resource"] += 1
        continue

```

```

if e["PI"] > 0 and e["demand_key"] in used_demand_keys:
    self.prunes["duplicate_demand_key"] += 1
    continue

# Return to start: only allowed as final route closure.
if next_node == self.graph.start:
    if len(path) == 0:
        continue

    candidate = path + [e]
    self.update_best(candidate)
    continue

# Simple route rule: no repeated station before returning to start.
if next_node in visited_nodes:
    self.prunes["repeated_station"] += 1
    continue

# Check if next node can still return to start in remaining time.
if new_elapsed + self.graph.min_time_to_start.get(next_node,
math.inf) > self.tmax_s + 1e-9:
    self.prunes["return_infeasible"] += 1
    continue

new_used_keys = set(used_demand_keys)
if e["PI"] > 0:
    new_used_keys.add(e["demand_key"])

# Pre-branch upper bound pruning.
next_pi = current_pi + e["PI"]
next_remaining = self.tmax_s - new_elapsed
optimistic_next = next_pi + self.fractional_pi_upper_bound(
    next_remaining, new_used_keys
)

if optimistic_next < self.best_pi - 1e-9:
    self.prunes["upper_bound"] += 1

```

```

        continue

    visited_nodes.add(next_node)
    used_edge_ids.add(edge_id)

    self.pulse(
        next_node,
        new_elapsed,
        next_pi,
        path + [e],
        visited_nodes,
        new_used_keys,
        used_edge_ids,
    )

    used_edge_ids.remove(edge_id)
    visited_nodes.remove(next_node)

def solve(self):
    self.start_time = time.perf_counter()

    self.pulse(
        node=self.graph.start,
        elapsed_s=0.0,
        current_pi=0,
        path=[],
        visited_nodes={self.graph.start},
        used_demand_keys=set(),
        used_edge_ids=set(),
    )

    runtime = time.perf_counter() - self.start_time

    if self.best_route is None:
        status = "NO_FEASIBLE_ROUTE_FOUND"
    elif self.limit_reached:
        status = "SEARCH_LIMIT_REACHED_BEST_FEASIBLE_RETURNED"
    else:

```

```
status = "CERTIFIED_EXACT_WITHIN_SEARCH_SPACE"
```

```
diagnostics = {  
    "status": status,  
    "runtime_s": runtime,  
    "expansions": self.expansions,  
    "prunes": self.prunes,  
    "TMAX_minutes": self.tmax_s / 60.0,  
    "max_runtime_seconds": self.max_runtime_seconds,  
    "max_expansions": self.max_expansions,  
}
```

```
return self.best_route, diagnostics
```

```
# =====
```

```
# 7. EXPORT FUNCTIONS
```

```
# =====
```

```
def route_links_dataframe(graph, route):
```

```
    rows = []
```

```
    cumulative_time_s = 0.0
```

```
    for seq, e in enumerate(route, start=1):
```

```
        cumulative_time_s += e["time_s"]
```

```
    rows.append(  
        {
```

```
            {
```

```
                "sequence": seq,
```

```
                "link_name": e["link_name"],
```

```
                "link_type": e["link_type"],
```

```
                "from_station_id": e["u"],
```

```
                "to_station_id": e["v"],
```

```
                "from_station_name": e["from_station_name"],
```

```
                "to_station_name": e["to_station_name"],
```

```
                "from_lat": e["from_lat"],
```

```
                "from_lon": e["from_lon"],
```

```
                "to_lat": e["to_lat"],
```

```
                "to_lon": e["to_lon"],
```

```

        "travel_time_s": e["time_s"],
        "travel_time_min": e["time_s"] / 60.0,
        "cumulative_time_s": cumulative_time_s,
        "cumulative_time_min": cumulative_time_s / 60.0,
        "distance_m": e["distance_m"],
        "PI": e["PI"],
        "Potential_Demand": e["PD"],
        "Served_Population": e["Pop"],
        "Served_Area_km2": e["Area_km2"],
        "Served_Area_m2": e["Area_m2"],
        "demand_key": e["demand_key"],
    }
)

return pd.DataFrame(rows)

def route_nodes_dataframe(graph, route):
    station_dict = graph.stations_df.set_index("station_id").to_dict("index")
    rows = []

    for seq, node in enumerate(graph.route_nodes(route), start=1):
        s = station_dict.get(node, {})
        rows.append(
            {
                "sequence": seq,
                "station_id": node,
                "station_name": s.get("station_name", ""),
                "latitude": s.get("latitude", np.nan),
                "longitude": s.get("longitude", np.nan),
            }
        )

    return pd.DataFrame(rows)

def route_coordinates(route):
    if not route:
        return []

```

```

coords = [(route[0]["from_lon"], route[0]["from_lat"], 0.0)]
for e in route:
    coords.append((e["to_lon"], e["to_lat"], 0.0))
return coords

def export_kml(route, output_path):
    coords = route_coordinates(route)
    coord_text = " ".join(f"{lon},{lat},{alt}" for lon, lat, alt in coords)

    placemarks = []

    # Route line.
    placemarks.append(
        f"""
        <Placemark>
            <name>Pulse optimal feeder bus route</name>
            <Style>
                <LineStyle>
                    <color>ff0000ff</color>
                    <width>4</width>
                </LineStyle>
            </Style>
            <LineString>
                <tessellate>1</tessellate>
                <coordinates>{coord_text}</coordinates>
            </LineString>
        </Placemark>
        """
    )

    # Link markers.
    for seq, e in enumerate(route, start=1):
        name = saxutils.escape(f"{seq}. {e['link_name']} ({e['u']->{e['v']}})")
        desc = saxutils.escape(
            f"Type: {e['link_type']} | Pl: {e['Pl']} | "
            f"PD: {e['PD']:.3f} | Time: {e['time_s']/60:.3f} min | "
            f"Demand key: {e['demand_key']}"
        )

```

```

        placemarks.append(
            f"""
            <Placemark>
                <name>{name}</name>
                <description>{desc}</description>
                <Point>
                    <coordinates>{e['to_lon']},{e['to_lat']},0</coordinates>
                </Point>
            </Placemark>
            """
        )

kml = f"""<?xml version="1.0" encoding="UTF-8"?>
<kml xmlns="http://www.opengis.net/kml/2.2">
<Document>
    <name>Pulse Feeder Bus Route</name>
    {"".join(placemarks)}
</Document>
</kml>
"""

with open(output_path, "w", encoding="utf-8") as f:
    f.write(kml)

def export_geojson(route, output_path):
    coords = [[lon, lat] for lon, lat, _ in route_coordinates(route)]

    features = [
        {
            "type": "Feature",
            "properties": {
                "name": "Pulse optimal feeder bus route",
                "model": "Pulse Algorithm",
            },
            "geometry": {
                "type": "LineString",
                "coordinates": coords,
            }
        }
    ]

```

```

        },
    }
]

```

```

for seq, e in enumerate(route, start=1):
    features.append(
        {
            "type": "Feature",
            "properties": {
                "sequence": seq,
                "link_name": e["link_name"],
                "link_type": e["link_type"],
                "from_station_id": e["u"],
                "to_station_id": e["v"],
                "PI": e["PI"],
                "Potential_Demand": e["PD"],
                "travel_time_min": e["time_s"] / 60.0,
                "demand_key": e["demand_key"],
            },
            "geometry": {
                "type": "Point",
                "coordinates": [e["to_lon"], e["to_lat"]],
            },
        }
    )

```

```

geojson = {
    "type": "FeatureCollection",
    "features": features,
}

```

```

with open(output_path, "w", encoding="utf-8") as f:
    json.dump(geojson, f, ensure_ascii=False, indent=2)

```

```

def export_html_map(route, output_path):
    if folium is None:
        print("folium is not installed. HTML map is skipped.")
        return

```



```

points = [(route[0]["from_lat"], route[0]["from_lon"])]
points.extend((e["to_lat"], e["to_lon"]) for e in route)

center_lat = float(np.mean([p[0] for p in points]))
center_lon = float(np.mean([p[1] for p in points]))

fmap = folium.Map(location=[center_lat, center_lon], zoom_start=15)

folium.PolyLine(
    points,
    tooltip="Pulse optimal feeder bus route",
    weight=5,
).add_to(fmap)

for seq, e in enumerate(route, start=1):
    popup = html.escape(
        f"Seq: {seq}<br>"
        f"Link: {e['link_name']}<br>"
        f"Type: {e['link_type']}<br>"
        f"From: {e['u']} - {e['from_station_name']}<br>"
        f"To: {e['v']} - {e['to_station_name']}<br>"
        f"PI: {e['PI']}<br>"
        f"PD: {e['PD']:.3f}<br>"
        f"Time: {e['time_s']/60:.3f} min<br>"
        f"Demand key: {e['demand_key']}"
    )

    folium.CircleMarker(
        location=[e["to_lat"], e["to_lon"]],
        radius=5,
        tooltip=f"{seq}. {e['link_name']}",
        popup=popup,
        fill=True,
    ).add_to(fmap)

fmap.save(output_path)

```

```

def write_outputs(graph, route, diagnostics, output_dir):
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    metrics = graph.metrics(route)

    summary = {
        "Model": "Pulse Algorithm",
        "Objective": "Maximize Total PI",
        "Start_End_Station_ID": graph.start,
        "TMAX_minutes": diagnostics["TMAX_minutes"],
        "Optimality_Status": diagnostics["status"],
        **metrics,
        "Program_Running_Time_s": diagnostics["runtime_s"],
        "Pulse_Expansions": diagnostics["expansions"],
        "Total_available_PI": graph.total_available_PI,
        "Total_available_PD": graph.total_available_PD,
        "Total_available_population": graph.total_available_population,
        "Total_available_area_km2": graph.total_available_area_km2,
        "Total_available_area_m2": graph.total_available_area_m2,
        "Unique_positive_demand_links": graph.n_unique_positive_demand_links,
    }

    summary_df = pd.DataFrame([summary])
    links_df = route_links_dataframe(graph, route)
    nodes_df = route_nodes_dataframe(graph, route)

    summary_path = output_dir / "pulse_summary.csv"
    links_path = output_dir / "pulse_route_links.csv"
    nodes_path = output_dir / "pulse_route_nodes_for_google_mymaps.csv"
    kml_path = output_dir / "pulse_route_map.kml"
    geojson_path = output_dir / "pulse_route_map.geojson"
    html_path = output_dir / "pulse_route_map.html"
    diagnostics_path = output_dir / "pulse_search_diagnostics.json"

    summary_df.to_csv(summary_path, index=False, encoding="utf-8-sig")
    links_df.to_csv(links_path, index=False, encoding="utf-8-sig")
    nodes_df.to_csv(nodes_path, index=False, encoding="utf-8-sig")

```

```

export_kml(route, kml_path)
export_geojson(route, geojson_path)
export_html_map(route, html_path)

with open(diagnostics_path, "w", encoding="utf-8") as f:
    json.dump(diagnostics, f, ensure_ascii=False, indent=2)

# Zip all output files.
zip_path = output_dir / "pulse_feeder_bus_outputs.zip"
with zipfile.ZipFile(zip_path, "w", compression=zipfile.ZIP_DEFLATED) as zf:
    for file_path in [
        summary_path,
        links_path,
        nodes_path,
        kml_path,
        geojson_path,
        html_path,
        diagnostics_path,
    ]:
        if file_path.exists():
            zf.write(file_path, arcname=file_path.name)

return {
    "summary": summary_path,
    "route_links": links_path,
    "route_nodes": nodes_path,
    "kml": kml_path,
    "geojson": geojson_path,
    "html": html_path,
    "diagnostics": diagnostics_path,
    "zip": zip_path,
}, summary_df, links_df, nodes_df

# =====
# 8. MAIN EXECUTION
# =====

```

```

def main():
    input_path = resolve_input_path()
    output_dir = resolve_output_dir()

    print("=" * 72)
    print("PULSE ALGORITHM FOR FEEDER BUS ROUTE OPTIMIZATION")
    print("=" * 72)
    print(f"Input file: {input_path}")
    print(f"Output directory: {output_dir}")
    print(f"Start/end station: {START_STATION_ID}")
    print(f"TMAX_MINUTES: {TMAX_MINUTES}")
    print()

    prepared_edges, stations = load_and_prepare_input(input_path,
START_STATION_ID)
    graph = FeederGraph(prepared_edges, stations, START_STATION_ID)

    print("Input data summary")
    print("-" * 72)
    print(f"Number of directed links: {len(prepared_edges):,}")
    print(f"Number of stations: {stations['station_id'].nunique():,}")
    print(f"Unique positive demand links:
{graph.n_unique_positive_demand_links:,}")
    print(f"Total available PI: {graph.total_available_PI:,.0f}")
    print(f"Total available PD: {graph.total_available_PD:,.3f}")
    print(f"Total available assigned area: {graph.total_available_area_km2:,.6f} km2")
    print()

    solver = PulseSolver(
        graph=graph,
        tmax_minutes=TMAX_MINUTES,
        max_runtime_seconds=MAX_RUNTIME_SECONDS,
        max_expansions=MAX_EXPANSIONS,
    )

    route, diagnostics = solver.solve()

    if route is None:

```

```

    print("No feasible route was found. Try increasing TMAX_MINUTES.")
    return None

paths, summary_df, links_df, nodes_df = write_outputs(
    graph=graph,
    route=route,
    diagnostics=diagnostics,
    output_dir=output_dir,
)

print("Pulse route result")
print("-" * 72)
display_cols = [
    "Optimality_Status",
    "Total_route_travel_time_min",
    "Total_PI",
    "Total_Potential_Demand",
    "SAp_percent",
    "Program_Running_Time_s",
    "Route_node_sequence",
]

print(summary_df[display_cols].T)
print()

print("Generated files")
print("-" * 72)
for key, value in paths.items():
    print(f"{key}: {value}")

print()
print("Route links")
print("-" * 72)
try:
    from IPython.display import display
    display(links_df)
except Exception:
    print(links_df)

```

```
    return {
        "graph": graph,
        "route": route,
        "diagnostics": diagnostics,
        "summary": summary_df,
        "route_links": links_df,
        "route_nodes": nodes_df,
        "paths": paths,
    }

if __name__ == "__main__":
    RESULTS = main()
```